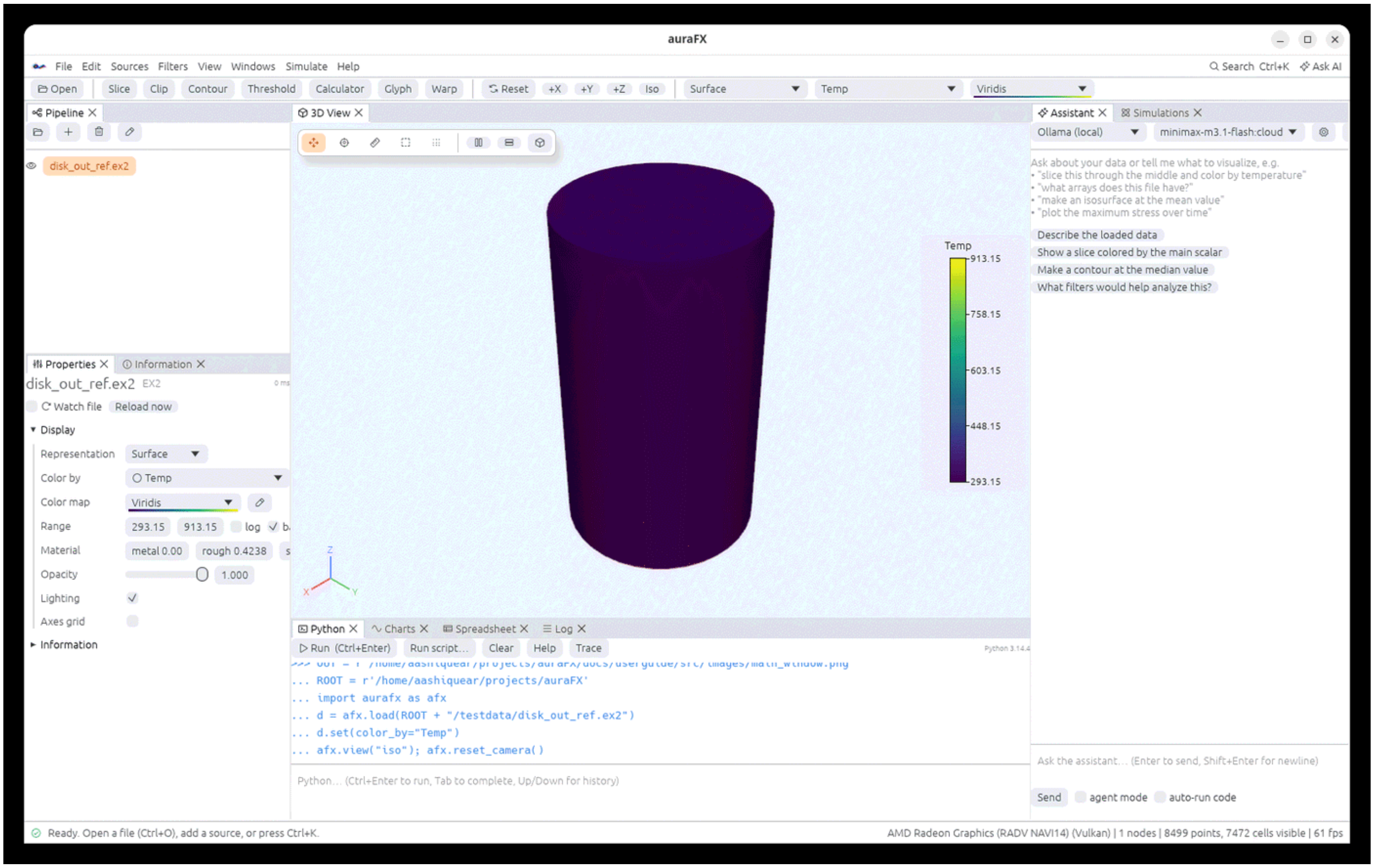
auraFX User Guide

auraFX is a scientific visualization, analysis and simulation workbench. It opens the files your solvers write (Exodus, VTK, OpenFOAM, LAMMPS, CGNS, EnSight, NetCDF and many more), shows them in a 3D view drawn by your GPU, lets you cut, contour, threshold, trace and measure them with the usual filters, and adds three things next to the 3D view:

- a **Python console** with a one-line-per-idea API (`afx.load("run.e").slice().color_by("T")`),
- an **AI assistant** that sees your pipeline and writes runnable code for you,
- a **simulation runner** that launches solvers locally, over SSH or through Slurm and opens the results when they finish.

This guide walks through every feature. Each chapter says where a feature is in the window, how to call it from Python, and what its limits are.


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Temp
Viridis
Pipeline
disk_out_ref.ex2
3D View
Temp
913.15
758.15
603.15
448.15
293.15
Z
X
Y
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Properties
Information
Watch file
Reload now
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

disk_out_ref.ex2 EX2

Display

Representation: Surface

Color by: Temp

Color map: Viridis

Range: 293.15 913.15 ☐ log ☑

Material: metal 0.00 rough 0.4238

Opacity: 1.000

Lighting: ☑

Axes grid: ☐

Information

```
>>> OUT = r'/home/aashiquear/projects/auraFX/docs/userguide/src/images/main_window.png
... ROOT = r'/home/aashiquear/projects/auraFX'
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... d.set(color_by="Temp")
... afx.view("iso"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 8499 points, 7472 cells visible | 61 fps

How this guide is organised

Part	What it covers
Basics	opening files, the pipeline, filters, coloring, views, selection, time and animation
Rendering	lighting, shadows, ambient occlusion, ray tracing, materials, volumes, point clouds
Scripting	the Python API, the AI assistant, writing plugins
Data	every file format read and written, exporting pictures and scenes, running simulations, working on remote machines
Reference	settings and keyboard shortcuts

Formats of this guide

The guide is kept as Markdown in the repository (`docs/userguide/src`), published as an online book, and printed to a PDF (`docs/userguide/auraFX-user-guide.pdf`). All three come from the same source files, so they always say the same thing.

Getting started

Installing

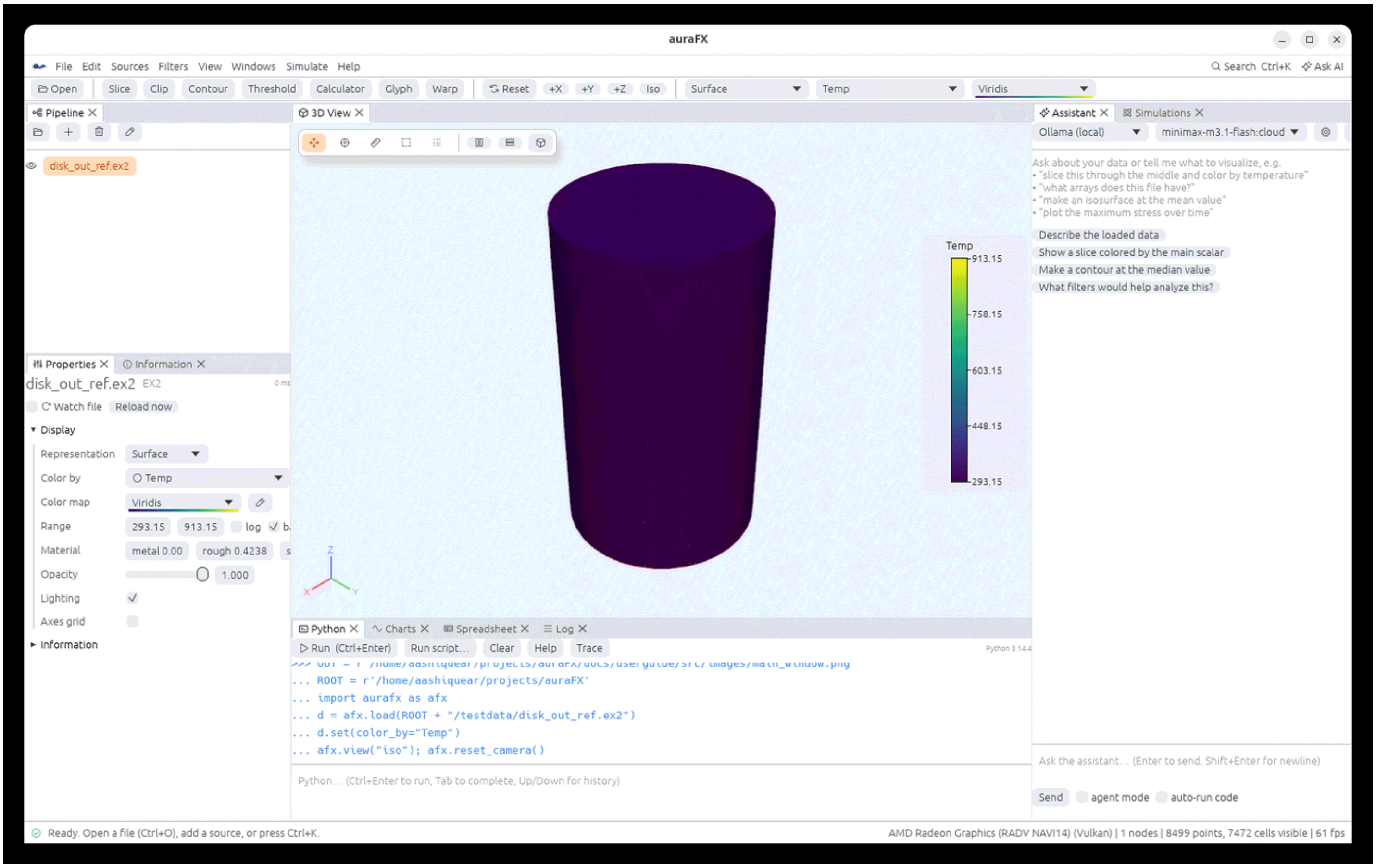
Prebuilt packages for Linux, macOS and Windows are attached to each release, and a one-line installer puts the app in your path:

```
curl -fsSL
https://raw.githubusercontent.com/aashiquear/auraFX/main/scripts/install.sh | sh
# Linux, macOS
irm https://raw.githubusercontent.com/aashiquear/auraFX/main/scripts/install.ps1 |
iex          # Windows PowerShell
```

Desktop packages bundle Python 3.12 with NumPy, so the Python console works without setup. To build from source you need Rust 1.85 or newer, Python 3.10+ with development headers, and a Vulkan (Linux, Windows) or Metal (macOS) GPU driver:

```
cargo build --release
./target/release/aurafx                    # start
./target/release/aurafx run.e              # open a file
./target/release/aurafx --script setup.py # run a Python script at startup
```

The window


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Temp
Viridis
Pipeline
disk_out_ref.ex2
3D View
Temp
913.15
758.15
603.15
448.15
293.15
Z
X
Y
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Properties | Information

disk_out_ref.ex2 EX2

☐ Watch file | Reload now

▾ Display

Representation	Surface
Color by	Temp
Color map	Viridis
Range	293.15 913.15 ☐ log ☑
Material	metal 0.00 rough 0.4238
Opacity	1.000
Lighting	☑
Axes grid	☐

▸ Information

Ollama (local) | minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

```
>>> OUT = r'/home/aashiquear/projects/aurafx/docs/userguide/src/images/main_window.png
... ROOT = r'/home/aashiquear/projects/auraFX'
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... d.set(color_by="Temp")
... afx.view("iso"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 8499 points, 7472 cells visible | 61 fps

The window has five areas; every panel can be closed, reopened from the **Windows** menu, and dragged to another place.

Area	Purpose
Pipeline (top left)	the files you opened and the filters below them; click to select, click the eye to show or hide
Properties / **Information** (bottom left)	the selected object’s parameters and display settings; the Information tab shows arrays, ranges, bounds, cell types and time steps
3D View (middle)	the picture; drag to rotate, right-drag or wheel to zoom, middle-drag (or Shift-drag) to pan, double-click to fit
Python, **Charts**, **Spreadsheet**, **Log** (bottom)	the console, plots, the data as a table, messages
Assistant, **Simulations** (right)	the AI chat and the job runner

The toolbar has the most used filters (Slice, Clip, Contour, Threshold, Calculator, Glyph, Warp), the camera buttons (Reset, +X, +Y, +Z, Iso) and, for the selected object, its representation, the array it is colored by and the color map. The status bar at the bottom names the GPU in use and counts the visible points and cells.

Opening a file

- **File › Open...** (Ctrl+O), or drop a file onto the window, or start the app with the file name.
- **File › Open Recent** keeps the last files; the three sample datasets at the bottom of that menu (a heated disk, a wavelet volume, a sphere) are good for trying things out.
- Numbered files (`run_0001.vtu`, `run_0002.vtu`, ...), `.pvd` collections, Exodus and OpenFOAM cases open as time series: the time toolbar appears under the main toolbar.

When a file opens, the object is colored by its first array and the camera fits the data. The file is watched: if a solver rewrites it, auraFX reloads it (**Watch file** in Properties).

```
d = afx.load("run.e")                # Python does the same
d.set(color_by="Temp")
afx.view("iso"); afx.reset_camera()
```

Applying a filter

Select an object, then pick a filter from the toolbar, the **Filters** menu, or the command palette (**Ctrl+K**, type part of its name). The filter runs at once with sensible defaults (a slice through the center, a contour at the middle value) and appears below its input in the Pipeline. Change its parameters in Properties; they apply as you type. **Re-run** forces a recomputation and **Reset to defaults** restores the defaults.

Filters can be stacked: a Contour of a Clip of a file. Delete an object with the trash button or the Delete key; the objects below it go too.

Saving your work

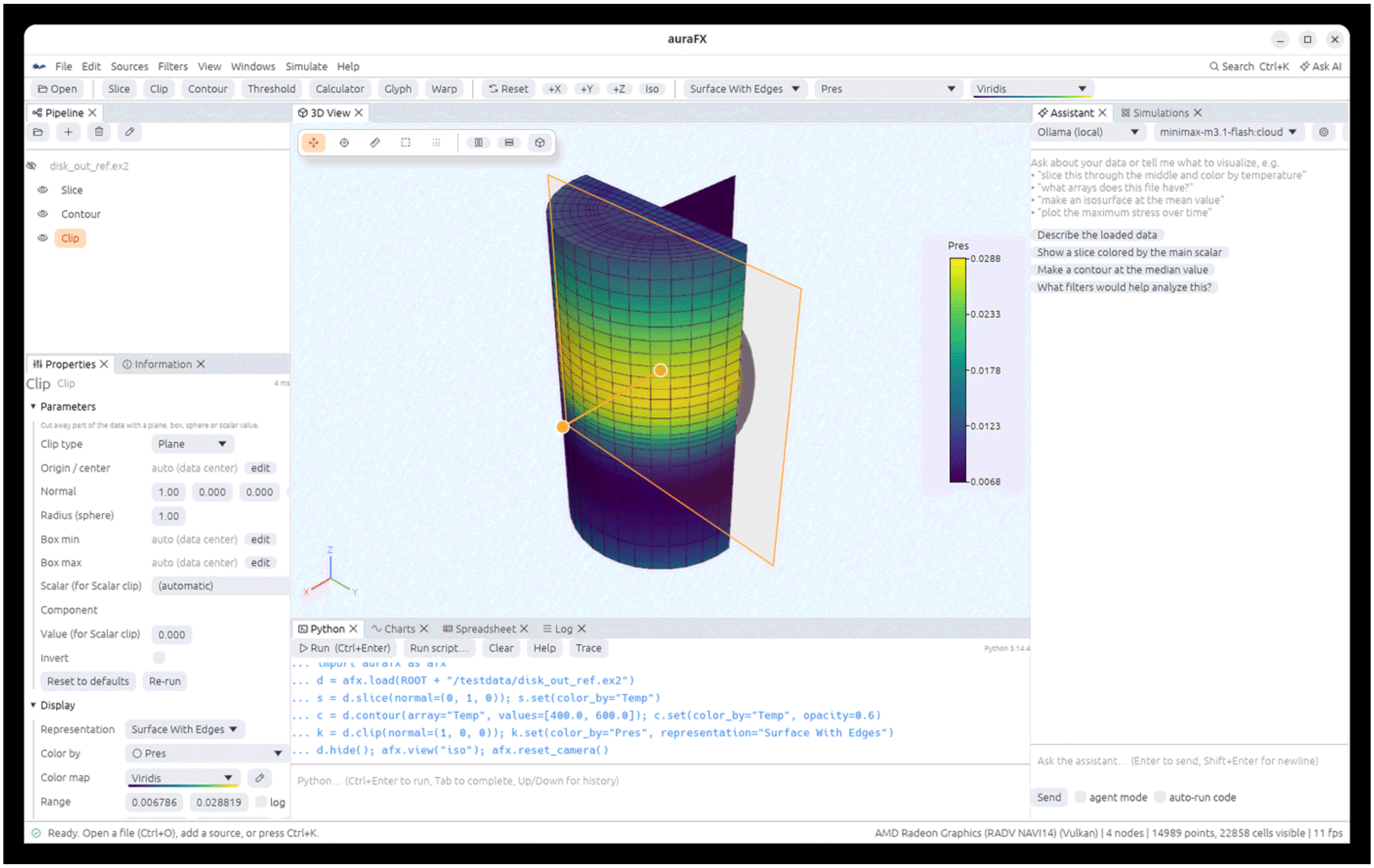
- **File › Save State** (Ctrl+S) writes an `.afx` file holding the pipeline, every parameter, the display settings, the camera and any keyframes. **File › Open** reopens it, including objects that live on a remote server.
- On the next start, **Restore last session** in the empty view brings back the last state.
- Right-click an object › **Save data...** writes its output to a file (see File formats).
- **Edit › Undo / Redo** cover every change to the pipeline and display settings.

Getting help

- **Help › Keyboard shortcuts** and **Help › Python API cheat sheet** open inside the app.
- The command palette (Ctrl+K) finds filters, sources and actions by name.
- The AI assistant answers questions about the loaded data and the app (see AI assistant).

Filters

A filter takes the output of the object above it and makes a new object. Select an input, choose the filter, adjust it in Properties. Every filter is also a method in Python (`node.slice(...)`), and the assistant knows all of them.


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
Pres
Viridis
Pipeline
3D View
Assistant
Simulations
disk_out_ref.ex2
Slice
Contour
Clip
Pres
0.0288
0.0233
0.0178
0.0123
0.0068
Z
X
Y
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Ollama (local)
minimax-m3.1-flash:cloud

Clip Clip 4 ms

▾ Parameters

Cut away part of the data with a plane, box, sphere or scalar value.

Clip type	Plane
Origin / center	auto (data center) edit
Normal	1.00 0.000 0.000
Radius (sphere)	1.00
Box min	auto (data center) edit
Box max	auto (data center) edit
Scalar (for Scalar clip)	(automatic)
Component	
Value (for Scalar clip)	0.000
Invert	☐

Reset to defaults Re-run

▾ Display

Representation	Surface With Edges
Color by	○ Pres
Color map	Viridis
Range	0.006786 0.028819 ☐ log

```
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... s = d.slice(normal=(0, 1, 0)); s.set(color_by="Temp")
... c = d.contour(array="Temp", values=[400.0, 600.0]); c.set(color_by="Temp", opacity=0.6)
... k = d.clip(normal=(1, 0, 0)); k.set(color_by="Pres", representation="Surface With Edges")
... d.hide(); afx.view("iso"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

Send ☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 4 nodes | 14989 points, 22858 cells visible | 11 fps

Common filters (toolbar)

Filter	What it does	Main parameters
Slice	the data on a plane or a sphere, with the values interpolated onto it	type, origin, normal (or radius); the plane can be dragged in the view
Clip	keeps one side of a plane, the inside/outside of a box or sphere, or the cells above/below a scalar value; **crinkle** keeps whole cells	type, origin, normal, box corners, radius, scalar and value, invert

Filter	What it does	Main parameters
Contour	isosurfaces (or isolines on 2-D data) at one or more values	array, values; computes normals
Threshold	the cells whose values lie in a range	array, min, max, all/any point
Calculator	a new array from an expression over other arrays (`sqrt(u*u+v*v)` , `mag(V)` , vectors with `iHat` ...)	expression, result name, point or cell
Glyph	arrows, cones, spheres, cubes or lines at the points, scaled and oriented by arrays	shape, orientation and scale arrays, factor, maximum glyphs
Warp By Vector / Scalar	moves points by a vector array (displacements) or along the normal by a scalar	array, factor
Stream Tracer	streamlines of a vector field from a line or a point cloud of seeds; **Tube** makes them solid	vectors, seed type, seed line or cloud, integration direction and length
Plot Over Line	samples the data along a line and plots it in Charts	the two end points (draggable)
Probe Location	the values at one point, shown in Information and the Spreadsheet	location (draggable)

Analysis

Resample To Image, Resample With Dataset, Histogram, Descriptive Statistics, Integrate Variables (areas, volumes and integrals of every array), Gradient (with divergence, vorticity and Q-criterion), Cell Size, Mesh Quality, Elevation, Cell Centers, Connectivity, Find Data, Extract Selection, Temporal Statistics (mean, min, max and standard deviation over all time steps), Temporal Interpolator, Temporal Shift Scale, Temporal Array Operator, and the particle filters (Particle Tracer, Particle Path, Streak Line) that follow particles through a time-varying flow. Texture Map to Plane / Cylinder / Sphere, Perlin Noise and Spatio Temporal Harmonics are described below.

Geometry

Extract Surface, Extract Edges, Feature Edges, Outline, Transform, Reflect, Shrink, Clean, Surface Normals, Smooth, Triangulate, Tetrahedralize, Tessellate (curved cells into straight pieces), Mask Points, Merge Blocks, Extract Block, Extract Subset (a sub-volume of an image), Append Datasets / Geometry / Attributes, Cell Data to Point Data and back, Extract Component, Rename Array, Pass Arrays, and OpenFOAM Boundary Patches.

Sources

Sources adds objects without a file: Sphere, Cone, Cylinder, Box, Plane, Line, Point Cloud, Wavelet (a test volume), Spatio Temporal Harmonics (a test volume of waves with its own time steps), AMR Gaussian Pulse (a small AMR dataset), Hyper Tree Grid and Random Hyper Tree Grid (hyper tree grids), Cell Grid (one DG cell with test fields) and Unstructured Cell Types (one of every cell type, for tests).

Textures and made-up data

Showing an image on a surface

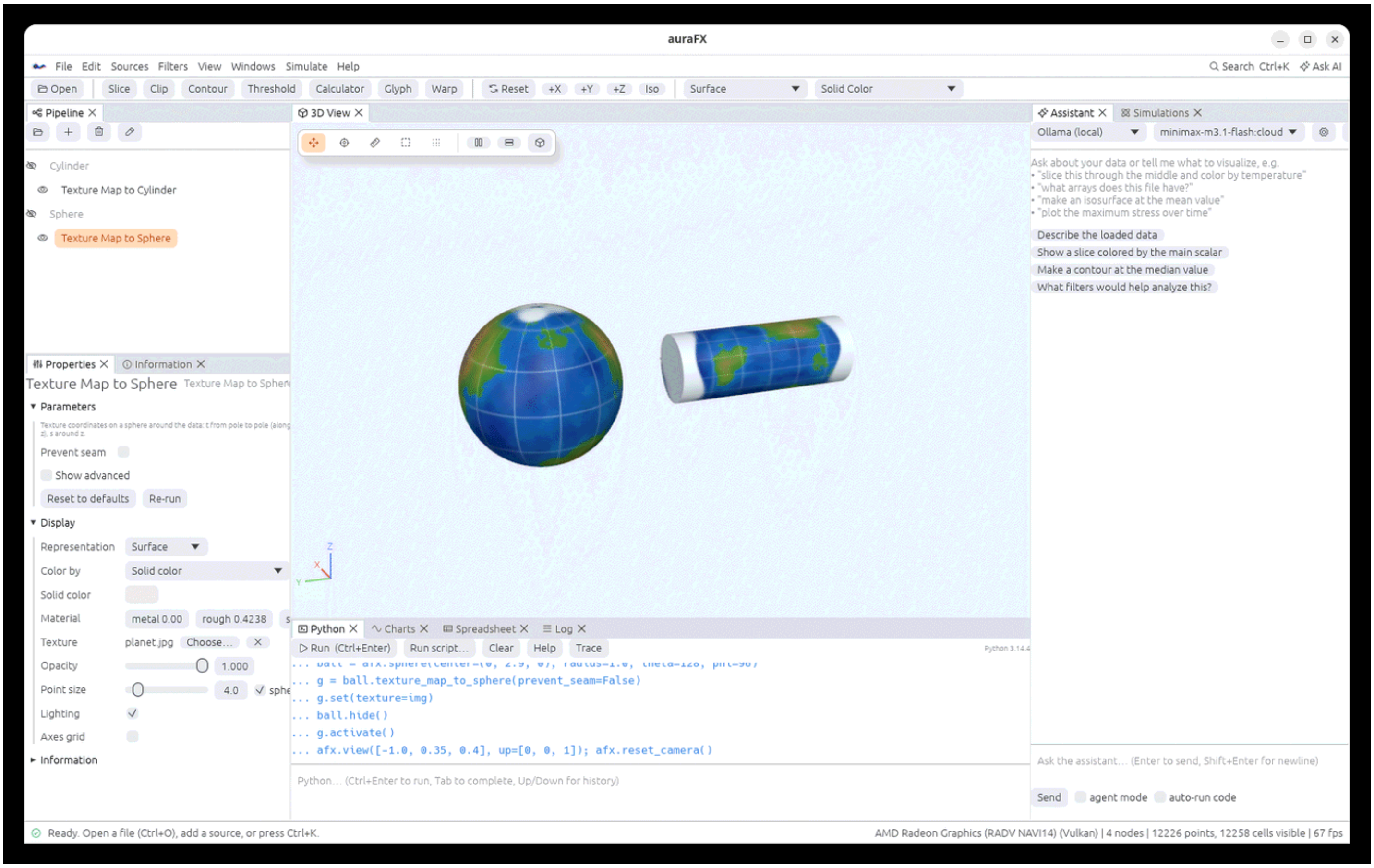
An image (PNG or JPEG) can be wrapped onto a surface, the way a world map goes onto a globe. The surface needs **texture coordinates**, which say where each point falls in the image. Some files and the Plane source come with them. For anything else, add one of these filters (**Filters › Texture Map to Plane / Cylinder / Sphere**):

Filter	How the image is laid on	Parameters
Texture Map to Plane	projected straight onto the surface from a plane, like a slide projector	**Automatic** fits the plane to the points and stretches the image over the bounds. Turn it off and give **Origin** (image corner), **Point 1** (the image's right edge) and **Point 2** (its top edge).
Texture Map to Cylinder	rolled around an axis: up the axis is up the image, around it is across	**Automatic** puts the axis along the object's longest side. Turn it off and give **Point 1** and **Point 2**, the axis ends.

Filter	How the image is laid on	Parameters
Texture Map to Sphere	wrapped around a center: the image's top at the +z pole, its bottom at the −z pole	**Center** (under Show advanced; empty = the middle of the points)

Prevent seam (cylinder and sphere, on by default) runs the image across half the way round and back mirrored. That avoids a visible join, but shows the image twice. Turn it off to wrap the image once, as for a world map. A thin seam then shows where the edges meet.

Then pick the image in **Properties › Display › Texture › Choose...** (× removes it). The row appears when the object has texture coordinates. The image replaces the array colors and the solid color; lighting and the material still apply. Coordinates outside 0 to 1 repeat the image.


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Solid Color
Pipeline
Cylinder
Texture Map to Cylinder
Sphere
Texture Map to Sphere
3D View
Z
X
Y
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Texture Map to Sphere

Parameters

Texture coordinates on a sphere around the data: t from pole to pole (along z), s around z.

Prevent seam ☐

☐ Show advanced

Display

Representation	Surface
Color by	Solid color
Material	metal 0.00 rough 0.4238
Texture	planet.jpg
Opacity	1.000
Point size	4.0 ✓ sphe
Lighting	✓
Axes grid	☐

```
... ball = afx.sphere(center=(0, 2.9, 0), radius=1.0, theta=128, phi=96)
... g = ball.texture_map_to_sphere(prevent_seam=False)
... g.set(texture=img)
... ball.hide()
... g.activate()
... afx.view([-1.0, 0.35, 0.4], up=[0, 0, 1]); afx.reset_camera()
```

Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant... (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 4 nodes | 12226 points, 12258 cells visible | 67 fps

```
ball = afx.sphere(radius=1, theta=128, phi=96)
globe = ball.texture_map_to_sphere(prevent_seam=False)
globe.set(texture="/path/to/world.jpg")       # texture=None removes it
p = d.texture_map_to_plane(automatic=False, origin=(0, 0, 0), point1=(10, 0, 0),
point2=(0, 5, 0))
c = d.texture_map_to_cylinder(point1=(0, 0, -1), point2=(0, 0, 1),
automatic=False)
```

The new array is called **Texture Coordinates**. The coordinates can also be colored by like any other array.

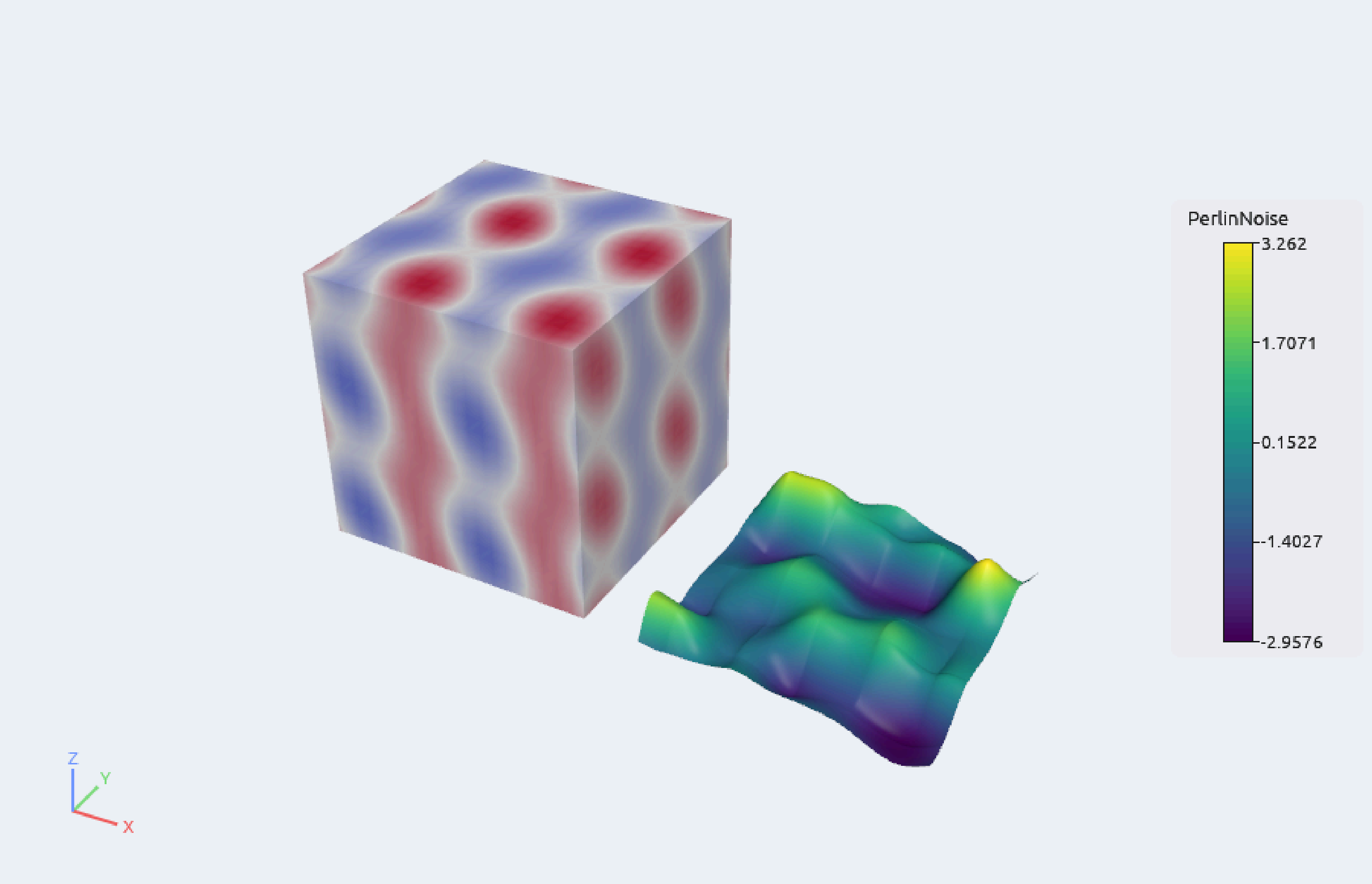
Limits:

- The image is shown on surfaces (Surface and Surface With Edges) and on lines. Points and volumes don’t show it.
- Ray tracing, scene export and pictures drawn on a server show the solid color instead of the image.
- The coordinates are stored at half precision for drawing (about 1/2000 of the image width), which only matters for images repeated many times.

Made-up data for tests and demos

Filter or source	What it makes	Parameters
Perlin Noise (filter)	a smooth random-looking value at every point (array **PerlinNoise**), for test data or rough-looking surfaces	**Frequency** (bumps per unit along x, y, z), **Phase** (shifts the pattern), **Amplitude**, result name
Spatio Temporal Harmonics (filter)	a sum of travelling sine waves at every point (array **SpatioTemporalHarmonics**)	**Harmonics**: 6 numbers per wave (amplitude, temporal frequency, wave vector x y z, phase). The value is the sum of amplitude × sin(frequency × time + wave vector · point + phase).
Spatio Temporal Harmonics (source, **Sources** menu)	an image volume with those waves and **20 time steps of its own** (one period), so it animates without a file	**Whole extent** (x min, x max, y min, y max, z min, z max; spacing 1), **Harmonics**, **Time steps**

The source’s time steps join the time toolbar like a file’s. The filter follows the time of its input. On data without time steps it uses time 0. The default frequencies round π/5 to 0.6283.



```
h = afx.spatio_temporal_harmonics_grid()     # waves in a 21×21×21 volume, 20 time steps
h.set(harmonics=[2, 1, 0.3, 0, 0, 0])         # one wave along x
afx.time(5)
n = afx.plane(xres=200, yres=200).perlin_noise(frequency=(4, 4, 4), amplitude=0.1)
n.warp_by_scalar(array="PerlinNoise")
```

Programmable filters

Programmable Filter, **Programmable Source** and **Python Calculator** run Python that is stored in the pipeline and in state files. The filter's code gets `data` (points and arrays as NumPy) and returns it changed, or a new dataset built with `afx.make_*`. For code you want to keep across sessions, write a plugin.

Filters with several inputs

Append Datasets, Append Geometry, Append Attributes, Resample With Dataset and Cell Grid Probe take several inputs. Ctrl+click the inputs in the Pipeline, then choose the filter; the

Properties panel gets an **Inputs** section where the order can be changed.

```
afx.apply_multi("append_datasets", [a, b, c])
r = probe_mesh.resample_with_dataset(source)
```

Parameters that say “auto”

Many parameters start as **auto (data center)** or similar: the value is taken from the data each time it runs (the center of the bounds, the middle of a range, 5 % of the size). Press **edit** to type a value instead. Auto values follow the data through time steps and across pieces of a parallel server.

Speed

Slice, Contour, Clip, Threshold, Extract Surface, Stream Tracer, Gradient and Integrate Variables run on every core and give exactly the same result on any number of threads. A slice of 4 million cells takes about 80 ms. The thread count can be limited in **Settings › Filter threads** (0 = all cores) or with `AURAFX_THREADS`.

From Python

```
afx.filters()                   # every filter with its parameters
afx.help_filter("clip")         # one filter explained
s = d.slice(normal=(0, 1, 0))
c = d.contour(array="Temp", values=[400, 600])
k = d.clip(type="Box", box_min=(-3, -3, -2), box_max=(3, 3, 4))
t = d.threshold(array="Pres", min=0.01)
s.set(origin=(0, 1, 0))         # change a parameter; the filter re-runs
s.update()                      # wait for it
```

Coloring and color maps

Color by an array

The **Color by** box in Properties (and in the toolbar) lists every point and cell array of the object. Vectors are colored by magnitude unless a **Component** is chosen. **Solid color** uses one color for the whole object. New files are colored by their first array automatically.

Range is taken from the data; type values to fix it (useful when comparing time steps or objects), press **auto** to return to the data range, tick **log** for a logarithmic scale and **bar** to show the legend in the view. Values that are not a number (NaN, fill values of NetCDF files) are drawn grey.

```
d.set(color_by="Temp", cmap="Inferno", range=(300, 900), log_scale=False)
d.set(color_by="V", component=0)        # x component of a vector
d.set(color_by=None, solid_color=(0.8, 0.2, 0.1))
```

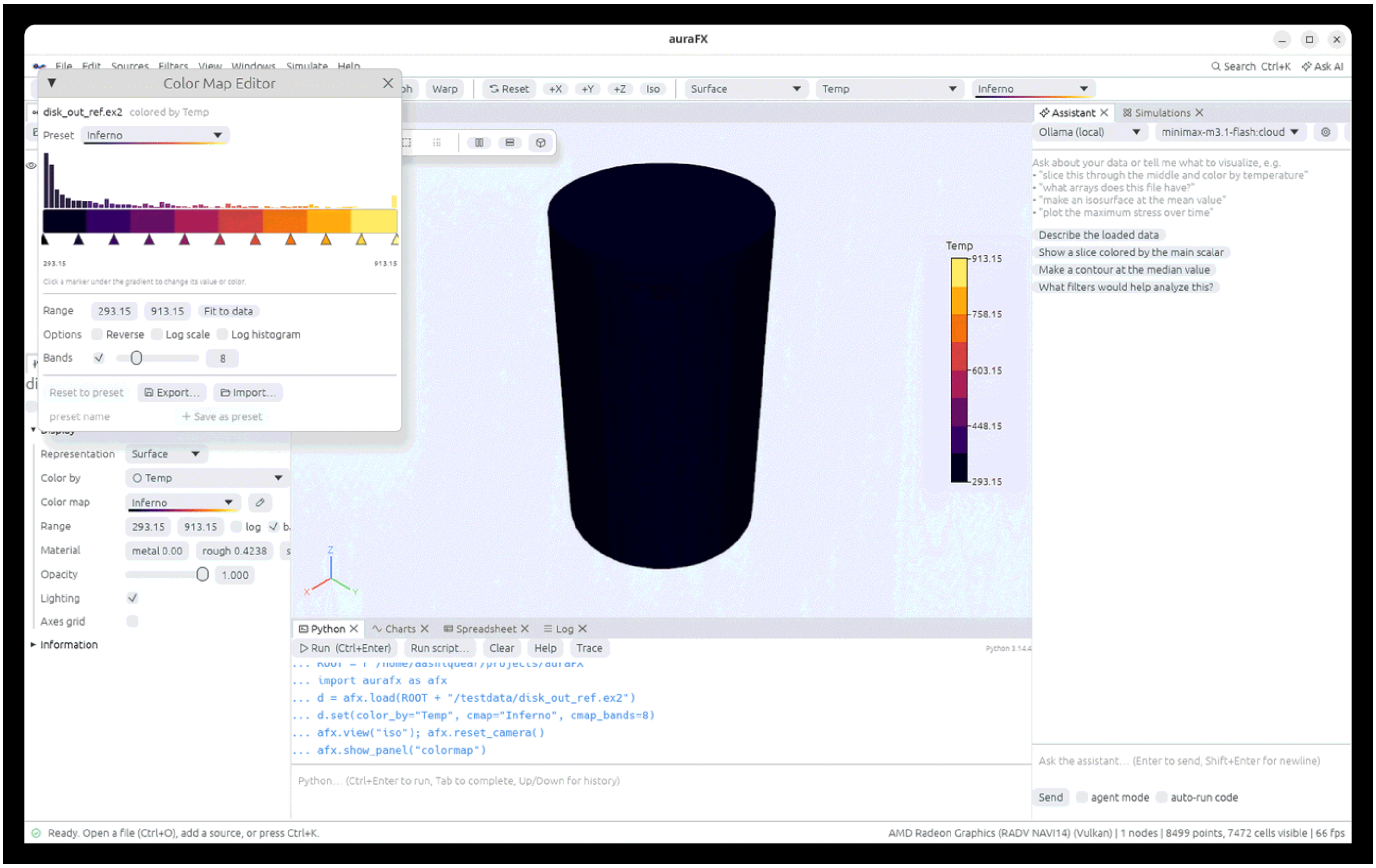
An array of 8-bit values with 3 or 4 components (red, green, blue and maybe opacity, such as the `IPFColor` array of Orientation Colors) is drawn with its own colors, without a legend. Clear **Colors: the array's own colors** under Color by, or use `d.set(direct_colors=False)`, to map it through the color map instead.

To show an image on a surface instead of colors (a map on a globe), see Textures.

Color maps

Eighteen maps ship with the app (Viridis, Inferno, Magma, Plasma, Cool to Warm, Jet, Rainbow, Grayscale, ColorBrewer sets and more); the **Color map** box shows each with a preview and marks the ones safe for color-blind readers. **Reverse** flips a map and **Bands** turns it into steps.

The color map editor


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Warp
Reset
+X
+Y
+Z
Iso
Color Map Editor
Temp
913.15
758.15
603.15
448.15
293.15
Z
X
Y
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
Fit to data
Reset to preset
Export…
Import…
+ Save as preset
Information

disk_out_ref.ex2 colored by Temp

Preset: Inferno

293.15 913.15

Click a marker under the gradient to change its value or color.

Range: 293.15 913.15

Options: ☐ Reverse ☐ Log scale ☐ Log histogram

Bands: ☑ 8

preset name

Representation: Surface

Color by: Temp

Color map: Inferno

Range: 293.15 913.15 ☐ log

Material: metal 0.00 rough 0.4238

Opacity: 1.000

Lighting: ☑

Axes grid: ☐

Ollama (local) | minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

```
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... d.set(color_by="Temp", cmap="Inferno", cmap_bands=8)
... afx.view("iso"); afx.reset_camera()
... afx.show_panel("colormap")
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 8499 points, 7472 cells visible | 66 fps

View › Color Map Editor… (or the pencil next to the map) opens the editor for the selected object:

- the histogram of the array behind the map shows where the values lie;
- drag the markers to move colors, double-click to change a color, right-click to delete one, click between markers to add one;
- **Bands**, **Reverse**, and the range fields as in Properties;
- **Save as preset** keeps your map under a name for other objects and sessions;
- **Import…** reads ParaView color map files (`.json`) as presets; **Export…** writes them.

```
afx.colormaps()                                         # names
afx.define_colormap("hot steel", [[0, 0, 0, 0], [0.5, 1, 0.3, 0], [1, 1, 1, 1]])
afx.import_colormaps("paraview_maps.json")
d.set(cmap="hot steel", cmap_reverse=True, cmap_bands=8)
d.set(cmap_points=[[0, 0, 0, 1], [1, 1, 0, 0]])     # a one-off map on this object
```

The legend

The legend (scalar bar) shows the map, its range and the array name for the selected object, or for the first object with a legend when nothing is selected. Untick **bar** to hide it. It is included in screenshots and animations, and drawn as real text in SVG exports.

Views, layouts and the window

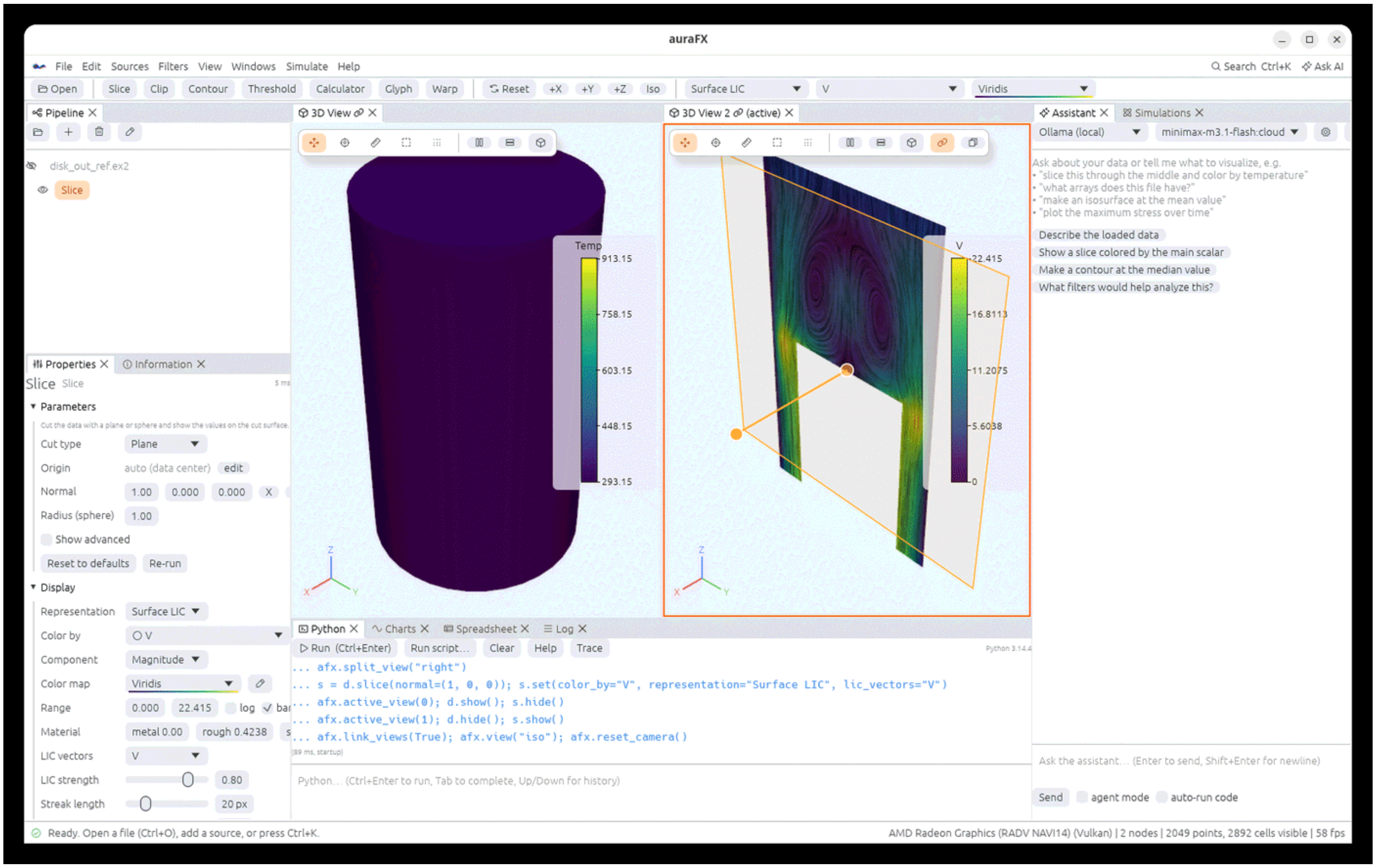
The 3D view

Action	Mouse / key
rotate	drag
pan	middle-drag, or Shift+drag
zoom	wheel, right-drag, or Ctrl+drag; pinch on a trackpad
fit the data	double-click, **R**, or the **Reset** button
look along an axis	**+X**, **+Y**, **+Z**, **Iso** in the toolbar, or **View › Look from…**
orthographic projection	**View › Orthographic projection**
orientation axes (bottom left)	**View › Orientation axes**

The small toolbar inside the view switches the mouse mode: navigate (**Esc**), pick (**P**), ruler (**M**), select cells (**S**) and select points (**D**); see Selection, picking and widgets.

```
afx.view("iso"); afx.view("+x"); afx.view([1, 0.5, 0.3])   # a direction from the data to the camera
afx.reset_camera()
```

Several views


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface LIC
V
Viridis
Pipeline
disk_out_ref.ex2
Slice
3D View
3D View 2 (active)
Temp
913.15
758.15
603.15
448.15
293.15
V
22.415
16.8113
11.2075
5.6038
0
Z
X
Y
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Slice Slice 5 ms

Parameters

Cut the data with a plane or sphere and show the values on the cut surface.

Cut type: Plane
Origin: auto (data center)
Normal: 1.00 0.000 0.000
Radius (sphere): 1.00
☐ Show advanced

Display

Representation: Surface LIC
Color by: ○ V
Component: Magnitude
Color map: Viridis
Range: 0.000 22.415 ☐ log ✓ bar
Material: metal 0.00 rough 0.4238
LIC vectors: V
LIC strength: 0.80
Streak length: 20 px

```
... afx.split_view("right")
... s = d.slice(normal=(1, 0, 0)); s.set(color_by="V", representation="Surface LIC", lic_vectors="V")
... afx.active_view(0); d.show(); s.hide()
... afx.active_view(1); d.hide(); s.show()
... afx.link_views(True); afx.view("iso"); afx.reset_camera()
(89 ms, startup)
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 2049 points, 2892 cells visible | 58 fps

The split buttons in the view toolbar (or **Windows › Split view right / down**, `afx.split_view("right")`) add a 3D view next to the active one. Each view has its own camera and its own list of visible objects; click a view to make it active (orange frame), then the eye icons in the Pipeline and `node.show()` / `hide()` apply to it. From Python, `afx.active_view(1)` makes a view active (0 is the first one) and returns once it is, so the lines after it act on that view:

```
afx.split_view("right")                # returns the new view's number (1)
afx.active_view(0); d.show(); s.hide()  # left: the data
afx.active_view(1); d.hide(); s.show()  # right: the slice
```

- **Link** (the chain button, `afx.link_views(True)`) keeps the cameras of the linked views together; **Match** (`afx.match_views()`) copies the active camera to the others once.
- Right-click an object › **Send to view** moves it to another view or into a new one (`node.send_to_view(1)` or `"right"`, `keep=True` to keep it in the current view too).
- Close a view with the × on its tab; its objects stay in the pipeline.

Panels and layouts

The **Windows** menu lists every panel (Pipeline, Properties, Information, 3D views, Python, Charts, Spreadsheet, Log, Assistant, Simulations) with a tick to show or hide it, and four layout presets:

Preset	Shows
Visualize	pipeline, properties and a large 3D view
Analyze	the view with charts and the spreadsheet
Script	the view with a tall Python console
Simulate	the simulations panel, the log and the view

Reset layout returns to the default. Panels can be dragged by their tabs into new positions or into tabs of each other; the arrangement is remembered between sessions.

```
afx.layout("analyze")
afx.show_panel("Charts"); afx.hide_panel("Log")
```

Theme and scale

Settings › Theme chooses dark, light or follow-the-system; the 3D background follows the theme (and can be set per theme). **UI scale** enlarges the whole interface. From Python: `afx.set_theme("light")`.

Command palette

Ctrl+K (or Ctrl+Space, or the **Search** button) opens the palette: type a few letters of a filter, a source, a panel or an action (open, save screenshot, reset camera, undo, light theme, ask the AI…) and press Enter. It is the fastest way to reach anything in the app.

Undo, redo and state

Edit › Undo / Redo (Ctrl+Z, Ctrl+Y) cover opening files, adding and deleting filters, parameter and display changes. **File › Save State** writes everything to an `.afx` file (see Getting started); `afx.save_state("work.afx")` and `afx.load_state("work.afx")` do the same from Python.

Selection, picking and widgets

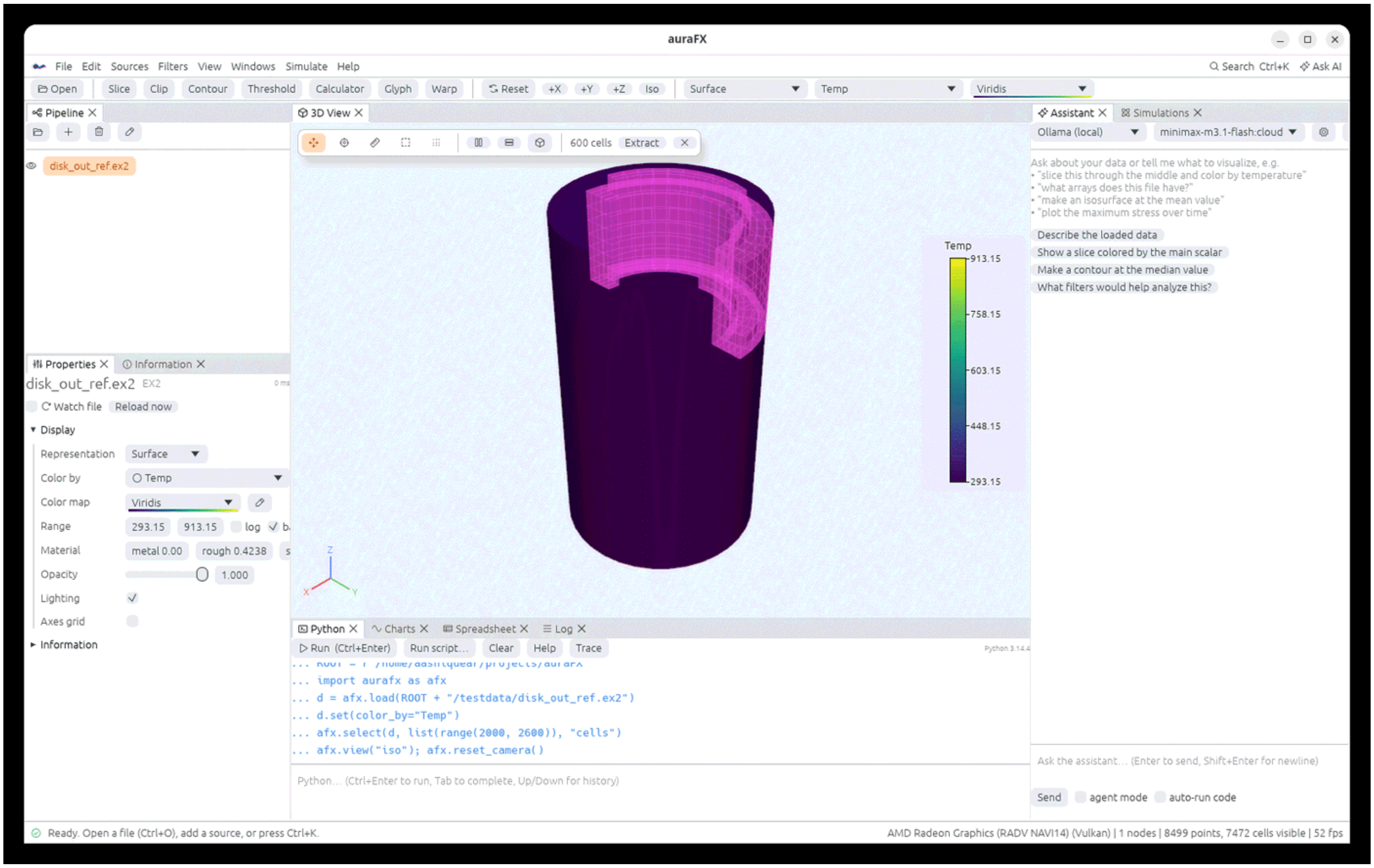
Picking values

Press **P** (or choose the pick mode in the view toolbar, or Ctrl+click in any mode) and click a surface: the point is marked and the Information panel lists every array's value there, with the object's name. `afx.probe(node, (x, y, z))` returns the same values from Python.

The ruler

Press **M**, click two points on surfaces: the distance is drawn and printed between them. Press M again (or Esc) to leave the mode.

Selecting cells and points


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Temp
Viridis
Pipeline
3D View
600 cells
Extract
disk_out_ref.ex2
Properties
Information
Temp
913.15
758.15
603.15
448.15
293.15
Z
X
Y
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

disk_out_ref.ex2 EX2

☐ Watch file · Reload now

Display

- Representation: Surface
- Color by: Temp
- Color map: Viridis
- Range: 293.15 913.15 ☐ log
- Material: metal 0.00 rough 0.4238
- Opacity: 1.000
- Lighting: ✓
- Axes grid: ☐

Information

```
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... d.set(color_by="Temp")
... afx.select(d, list(range(2000, 2600)), "cells")
... afx.view("iso"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) · minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

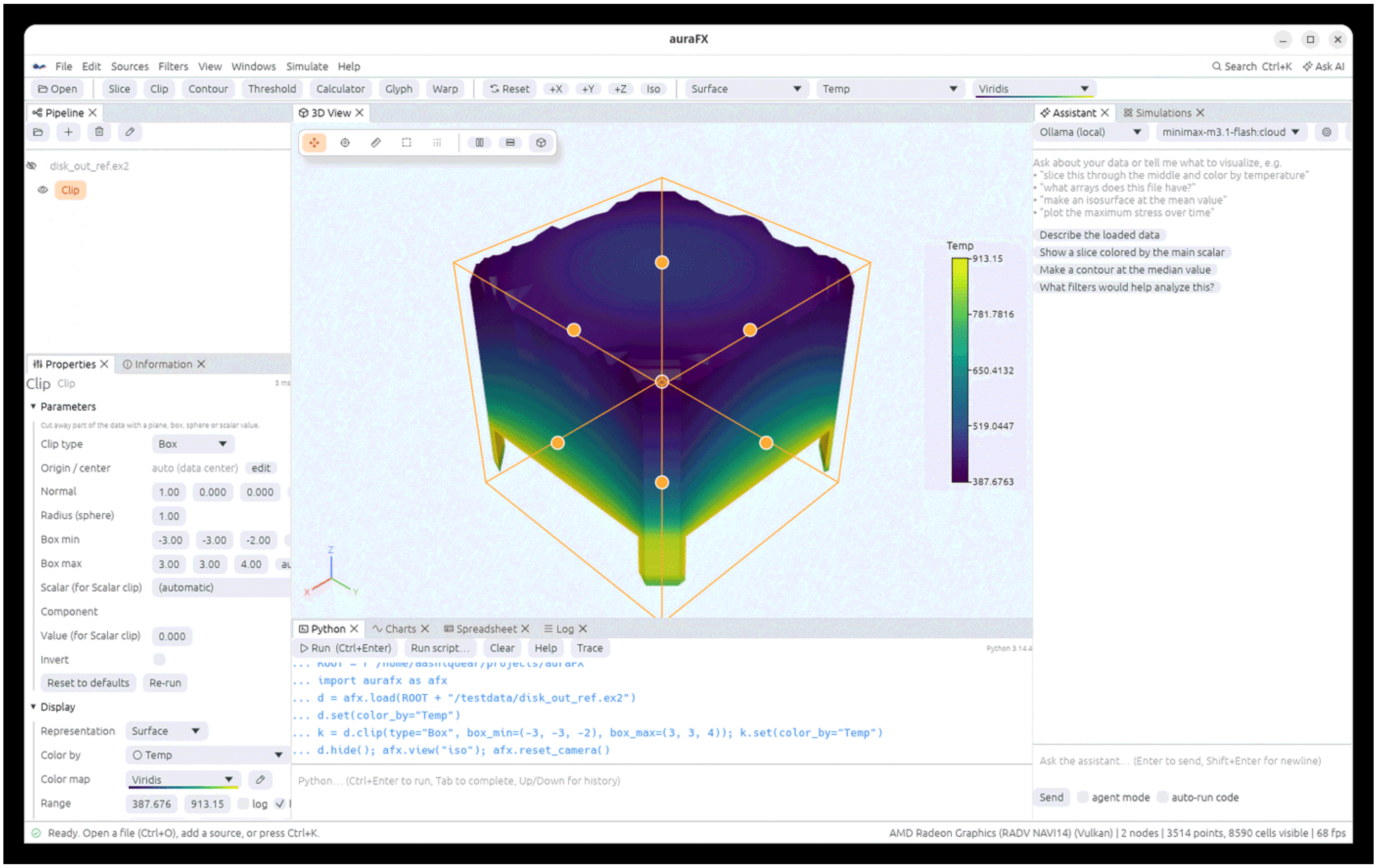
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 8499 points, 7472 cells visible | 52 fps

Press **S** (cells) or **D** (points) and drag a rectangle over the view. The selected cells are drawn highlighted through the surface. **Shift** adds to the selection, **Ctrl** removes from it. By default only the cells you can see are selected; **Select through** in the view toolbar takes every cell inside the rectangle, hidden ones included.

- **Create an Extract Selection node** (the button in the view toolbar, `afx.extract_selection()`) turns the selection into a new object you can filter further.
- **Find Data** (Filters) selects by a query such as `Temp > 600` or the N largest values.
- The Spreadsheet shows the selected rows; `afx.selection()` returns them, `afx.select(node, ids, "cells")` sets them, `afx.clear_selection()` clears.

Widgets: dragging filter shapes in the view


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp Reset +X +Y +Z Iso Surface Temp Viridis
Pipeline
disk_out_ref.ex2
Clip
3D View
Temp
913.15
781.7816
650.4132
519.0447
387.6763
Z
X
Y
Assistant
Simulations
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Clip Clip

Parameters

Cut away part of the data with a plane, box, sphere or scalar value.

Clip type	Box
Origin / center	auto (data center)
Normal	1.00 0.000 0.000
Radius (sphere)	1.00
Box min	-3.00 -3.00 -2.00
Box max	3.00 3.00 4.00
Scalar (for Scalar clip)	(automatic)
Component	
Value (for Scalar clip)	0.000
Invert	

Display

Representation	Surface
Color by	Temp
Color map	Viridis
Range	387.676 913.15

```
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... d.set(color_by="Temp")
... k = d.clip(type="Box", box_min=(-3, -3, -2), box_max=(3, 3, 4)); k.set(color_by="Temp")
... d.hide(); afx.view("iso"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 3514 points, 8590 cells visible | 68 fps

When the selected filter has a shape, it is drawn in the view with orange handles:

Filter	Widget
Slice, Clip (plane)	a plane: drag the center along the normal, drag the tip to tilt

Filter	Widget
Clip (box), Box source	a box: drag the center to move it, a face handle to resize that side
Slice/Clip (sphere), Stream Tracer point-cloud seeds, Sphere source	a sphere: center and radius handles
Plot Over Line, Stream Tracer line seeds, particle tracers, Line source	a line: either end, or the middle to move both
Probe Location	a point

The parameters in Properties follow the drag, and typing values moves the widget. Custom lights can be aimed the same way; see Lighting.

Annotations

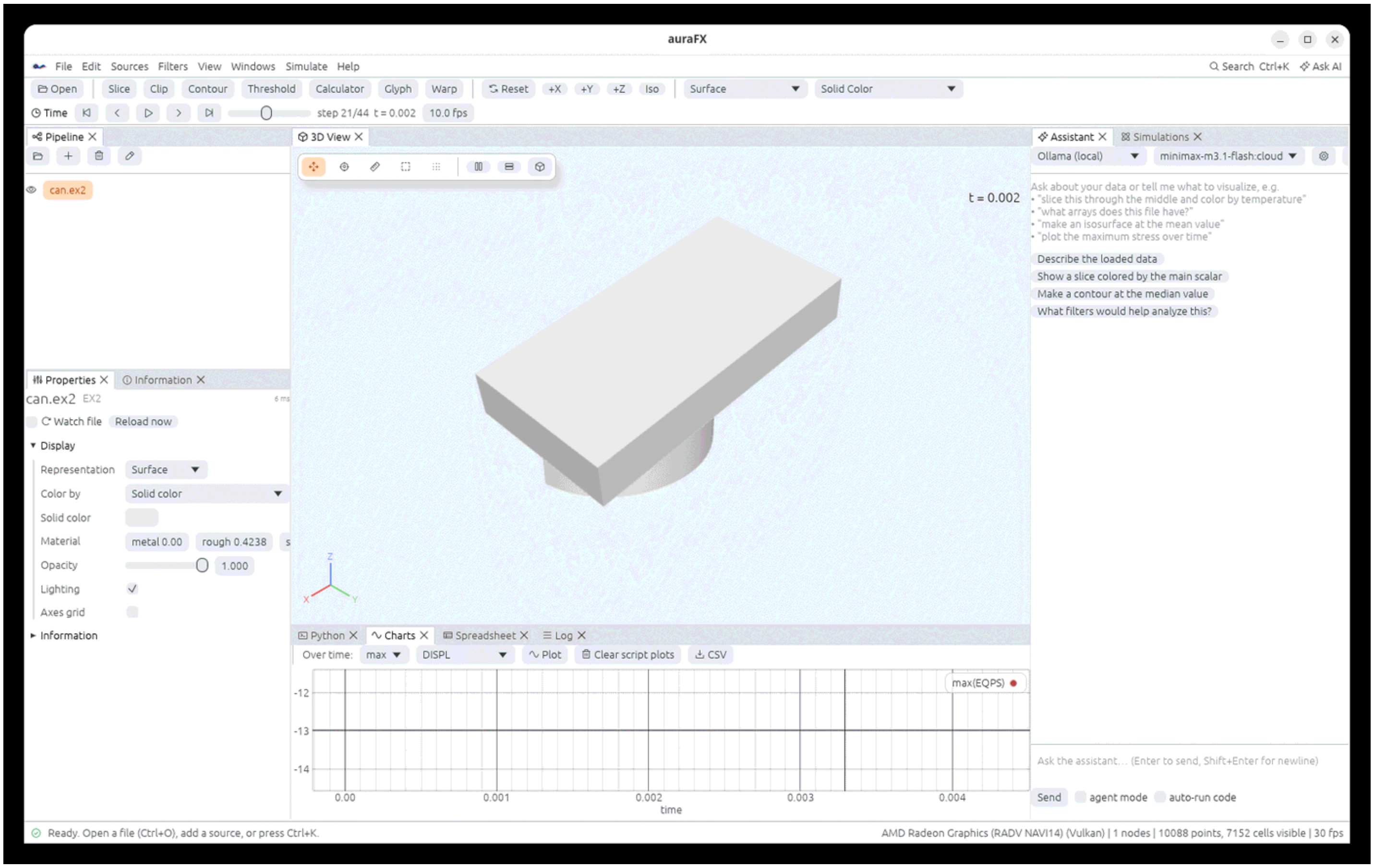
Filters › Annotation, `afx.annotate("Run 42", corner="upper left", size=18)` puts text in a corner of the view; `afx.time_label(True)` (or **View › Time label**) shows the current time. Both are drawn in screenshots and animations. `afx.clear_annotations()` removes them.

Time, charts and animation

Time steps

Files with several time steps (Exodus, `.pvd`, OpenFOAM cases, LAMMPS dumps, numbered file series, VTKHDF, EnSight, Xdmf, NetCDF, CGNS), and the Spatio Temporal Harmonics source, show the **time toolbar**: play/pause, first, previous, next, last, a slider, the step number, the time value and the playback speed. Every filter follows the time step; filters that look across time (Temporal Statistics, Particle Tracer, Temporal Interpolator…) read the steps they need.

```
afx.times()            # the time values
afx.time(20)           # go to step 20
```


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp Reset +X +Y +Z Iso
Surface
Solid Color
Time
step 21/44 t = 0.002
10.0 fps
Pipeline
can.ex2
3D View
t = 0.002
Properties
Information
can.ex2 EX2
Watch file
Reload now
Display
Representation Surface
Color by Solid color
Solid color
Material metal 0.00 rough 0.4238
Opacity 1.000
Lighting
Axes grid
Information
Python Charts Spreadsheet Log
Over time: max DISPL
Plot
Clear script plots
CSV
max(EQPS)
-12
-13
-14
0.00
0.001
0.002
0.003
0.004
time
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Send
agent mode
auto-run code

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 10088 points, 7152 cells visible | 30 fps

Charts

The **Charts** panel plots lines with zoom, legends and log axes:

- **Plot Over Line** (a filter) plots the arrays along a line;
- **Edit › Plot Over Time...** or `afx.over_time(node, "EQPS", stat="max")` plots a statistic (max, min, mean, sum, or the value of a global variable) over every time step;
- `afx.plot(x, y, label="T")` plots your own NumPy arrays; `afx.clear_plots()` empties the panel.

The **Spreadsheet** panel shows the selected object's points, cells or field data as a table with sorting, column choice and export to CSV.

Animation

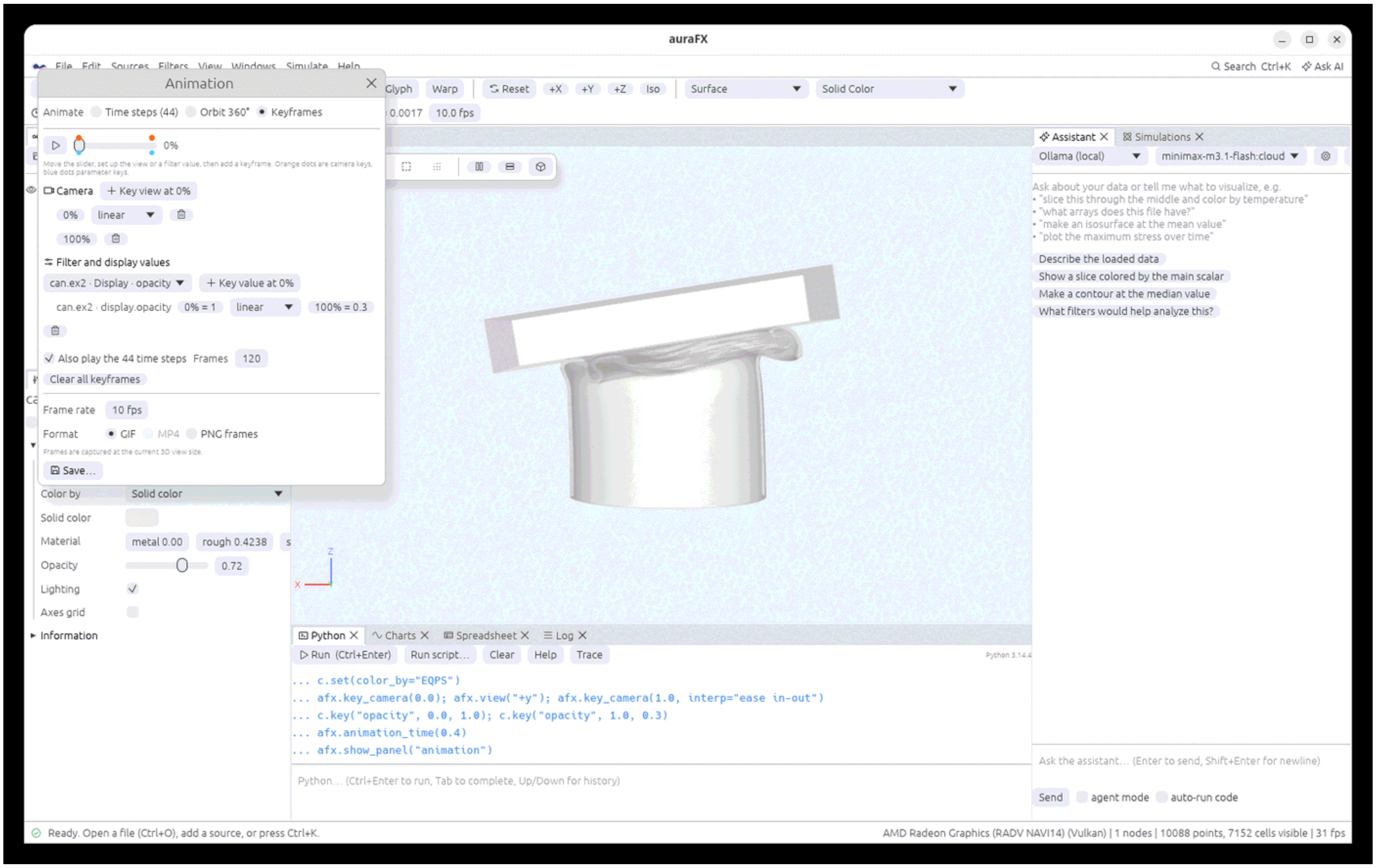
File › Animation... records one of three things:

Mode	What it plays
Time steps	every time step of the data
Orbit 360°	the camera turning once around the data (choose the number of frames)
Keyframes	the keyframe timeline described below

Pick the frame rate and the format: **GIF**, **MP4** (needs `ffmpeg` on the PATH) or a folder of **PNG frames**. Frames are captured at the 3D view's size, with the legend, labels and annotations.

```
afx.save_animation("run.gif", fps=10)          # time steps (or keyframes if any)
afx.save_animation("turn.mp4", orbit=72)       # a turntable
```

Keyframes


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Solid Color
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Send
Information
Save…
Clear all keyframes
z
x

Animation

Animate ○ Time steps (44) ○ Orbit 360° ● Keyframes

0%

Move the slider, set up the view or a filter value, then add a keyframe. Orange dots are camera keys, blue dots parameter keys.

Camera + Key view at 0%

0% linear

100%

Filter and display values

can.ex2 · Display · opacity + Key value at 0%

can.ex2 · display.opacity 0% = 1 linear 100% = 0.3

✓ Also play the 44 time steps Frames 120

Frame rate 10 fps

Format ● GIF ○ MP4 ○ PNG frames

Frames are captured at the current 3D view size.

Color by Solid color

Material metal 0.00 rough 0.4238

Opacity 0.72

Lighting ✓

Axes grid

0.0017 10.0 fps

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

```
... c.set(color_by="EQPS")
... afx.key_camera(0.0); afx.view("+y"); afx.key_camera(1.0, interp="ease in-out")
... c.key("opacity", 0.0, 1.0); c.key("opacity", 1.0, 0.3)
... afx.animation_time(0.4)
... afx.show_panel("animation")
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

○ agent mode ○ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 10088 points, 7152 cells visible | 31 fps

The **Keyframes** tab of the Animation window is a timeline from 0 % to 100 %:

1. Move the slider, set up the view, press **Key view at …** to record the camera.
2. Select a filter or display value in the list (any numeric filter parameter, or opacity, point size, line width, colors…), set it, press **Key value at …**.
3. Each key has an interpolation menu for the way it moves on to the next key: **linear**, **step** (hold), **ease in**, **ease out**, **ease in-out**, **exponential** (constant ratio, for zooms and scales) or **wave** (there and back, a sinusoid).
4. Press play to preview; tick **Also play the time steps** to run the data's time along the animation; set the number of **Frames**; **Save…** records it.

The camera moves on a sphere around the focal point, so turns look even and zooms do not rush. Orange dots on the timeline are camera keys, blue dots value keys; click a key to jump to it, right-click to delete it.

```
afx.key_camera(0.0); afx.view("+y"); afx.key_camera(1.0, interp="ease in-out")
c.key("opacity", 0.0, 1.0); c.key("opacity", 1.0, 0.3)
s.key("origin", 0.5, (0, 2, 0), interp="wave")
afx.animation(frames=120, time=True)
afx.animation_time(0.4)                          # preview a moment
afx.save_animation("keys.mp4", fps=24)
afx.clear_keyframes()
```

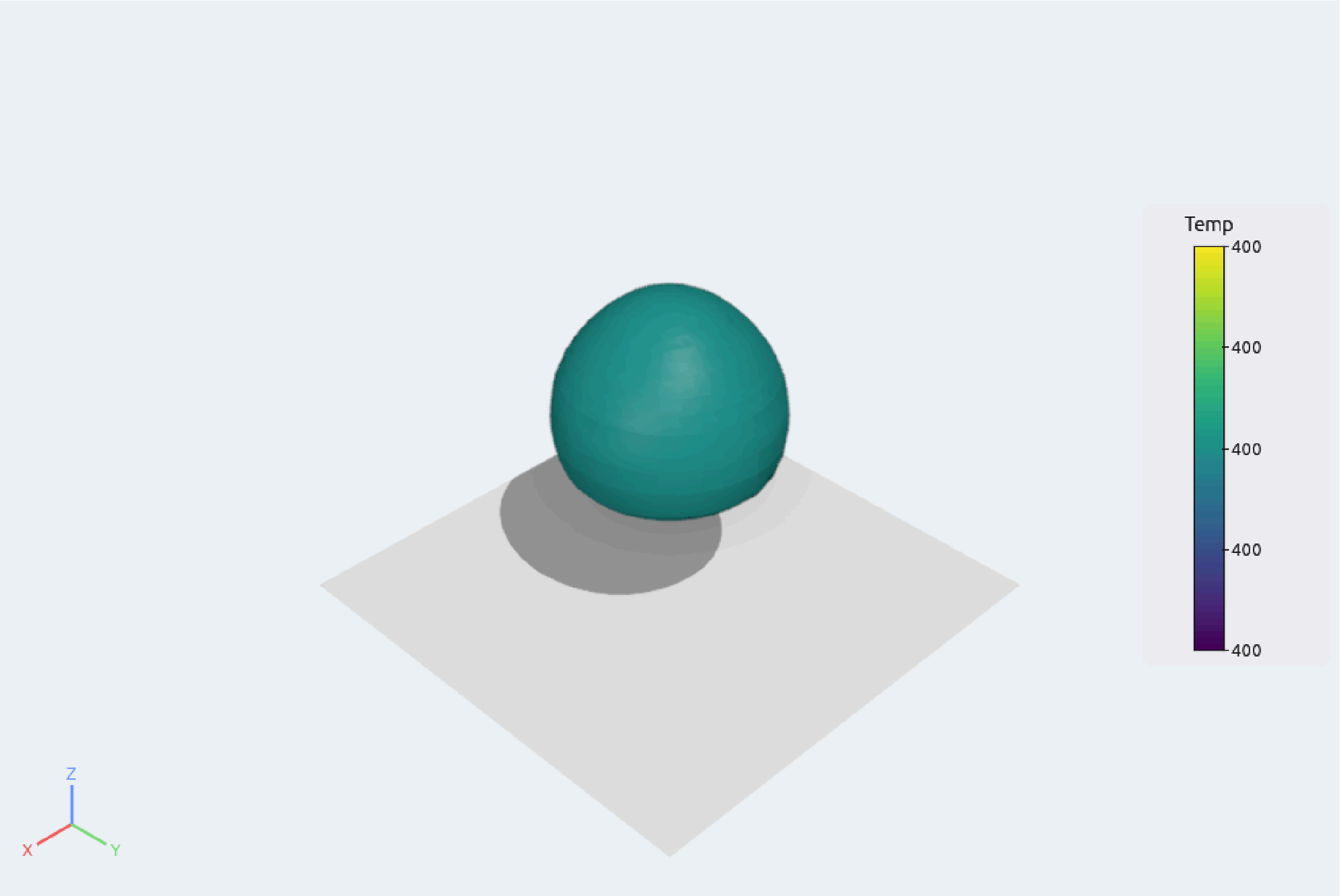
Animations driven by a function

`afx.save_animation("a.mp4", cue=f, frames=60)` calls your function `f(t)` with the animation time `t` from 0 to 1 before each frame (after the keyframes are applied), so anything can change between frames: filter values, colors, the data loaded. The call returns when the file is written.

```
def cue(t):
    c.set(values=[350 + 500 * t])
afx.save_animation("contour_sweep.gif", fps=8, cue=cue, frames=40)
```

Lighting, shadows and effects

Everything here is in **View › Lighting**, and in Python through `afx.lighting(...)`. Calling `afx.lighting()` without arguments returns the current defaults; unknown options raise an error, so typos are caught.



Lights

- **Light kit** (on by default): a key light from the upper right, a fill light from the lower left and a head light. Untick it for a single head light.
- **Custom lights**: tick it to edit the lights yourself. Each light has a color, an intensity, an azimuth and an elevation, follows the camera or stays fixed in the scene, and can cast shadows. **Add light** adds one; **ambient** sets the light from everywhere.
- **Show lights in the view** draws each custom light as a small sun around the data; drag a sun to aim its light.

```
afx.lighting(light_kit=False)
afx.lighting(lights=[{"direction": (0.5, 0.5, 1), "intensity": 0.8},
                     {"direction": (-0.7, -0.2, 0.5), "intensity": 0.3, "color":
(0.6, 0.7, 1), "camera_relative": False}],
             ambient=0.15)
```

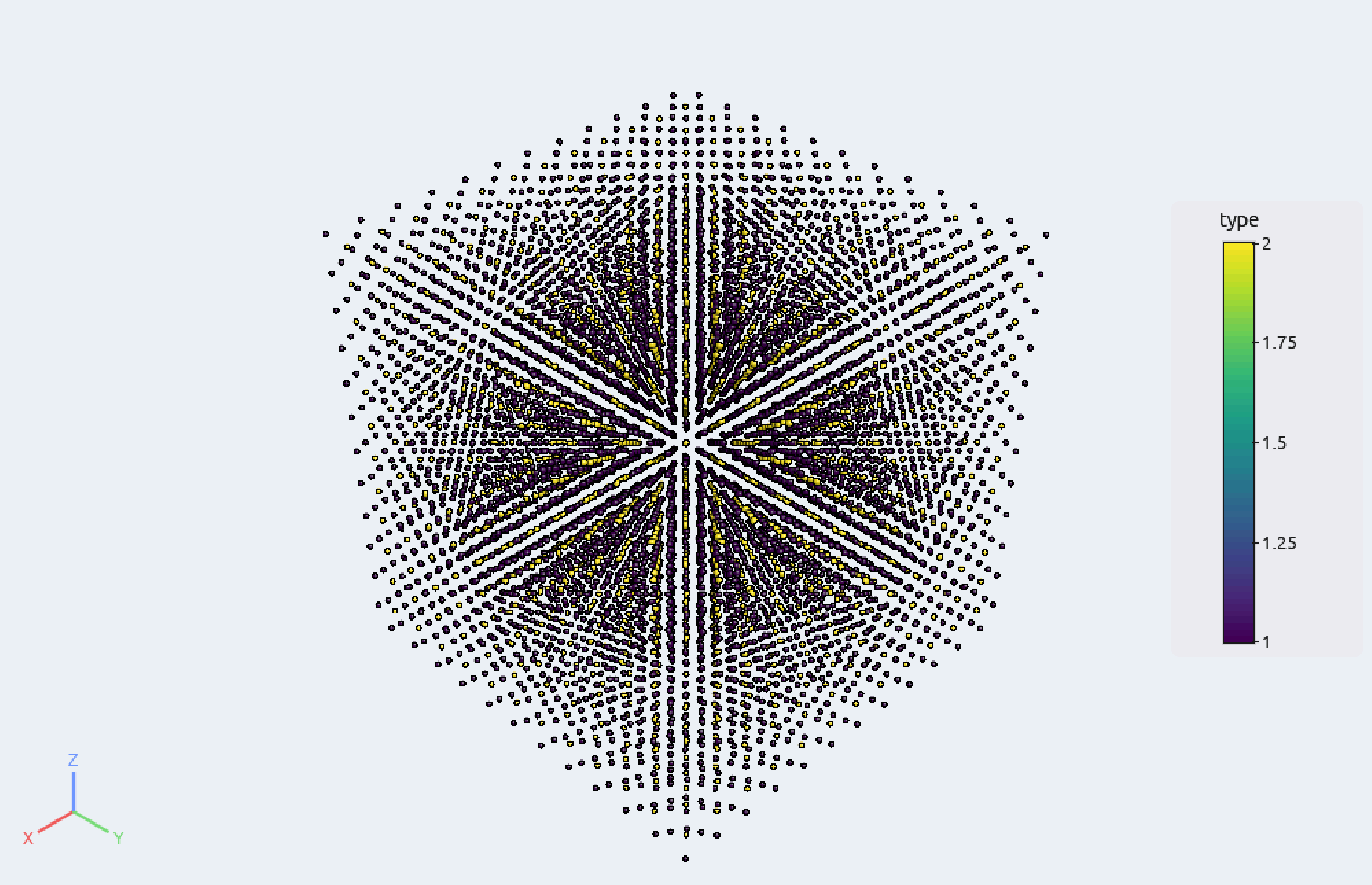
Shadows

Shadows draws the shadow of the key light (or of the first custom light with shadows ticked) on every opaque surface: a shadow map fitted to the visible objects, softened at the edges. Objects drawn by a remote server are shadowed on that machine, so they do not cast shadows on local objects.

Ambient occlusion

Ambient occlusion darkens creases, cavities and the contact between objects, which makes depth easier to read. The sliders set its strength and radius (a fraction of the data size). It works from the picture’s depth, so it costs the same whatever the mesh size.

Eye-Dome Lighting



Eye-Dome Lighting shades every pixel by how much it stands out from its neighbours in depth: outlines and depth cues for point clouds, particles and dense meshes without any normals. The slider sets the strength. `afx.lighting(edl=True)`.

Anti-aliasing, tone mapping and exposure

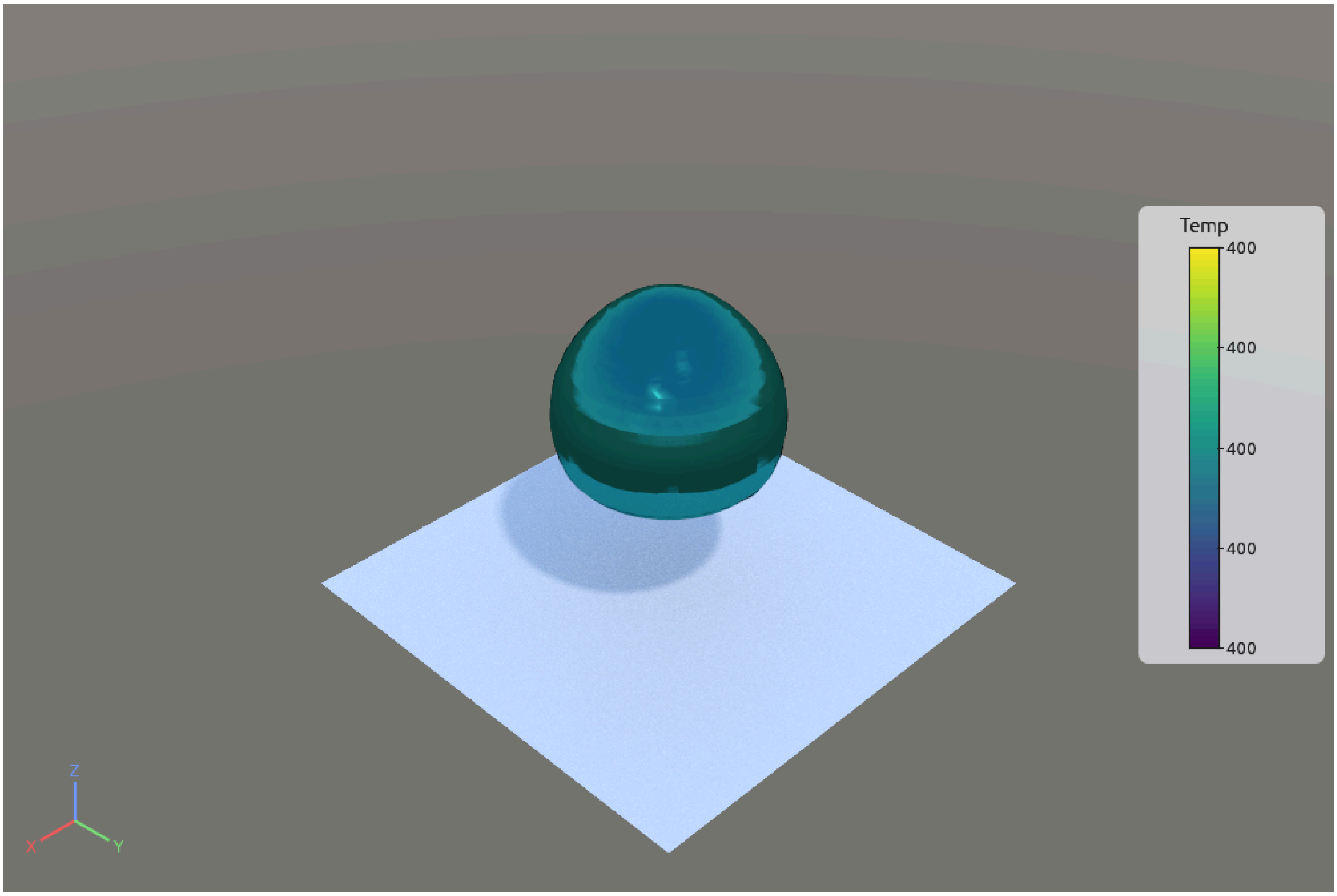
Edges are smoothed by multisampling (4 samples). **FXAA** adds a cheap post pass that smooths the remaining jagged edges of thin lines. **Tone mapping** compresses bright pictures with the **Reinhard** or **ACES filmic** curves, and **exposure** scales brightness before it.

```
afx.lighting(fxaa=True, tone_mapping="aces", exposure=1.2)
```

Transparency

Translucent objects (**Opacity** below 1 in Properties) are blended in a way that does not depend on the order they are drawn in, so several overlapping translucent surfaces look right from any angle. Translucent objects are always drawn on this computer, even when the rest of the data is drawn by a server.

Materials and environment maps



Material in Properties gives each surface a **metal** factor (0 plastic-like, 1 metal: reflections take the object’s color), a **rough**ness (small: sharp highlights and mirror-like reflections; 1: matte) and a highlight strength. The defaults reproduce the plain shading.

View › Lighting › Environment lights the scene with a picture of the surroundings: the built-in **sky**, **studio** and **white** environments, or your own equirectangular `.hdr` / `.png` / `.jpg` (z up). It replaces the flat ambient light with soft directional light, adds reflections on shiny and metallic surfaces, and **as background** shows it behind the data. While an environment is on, the built-in lights are dimmed by half.

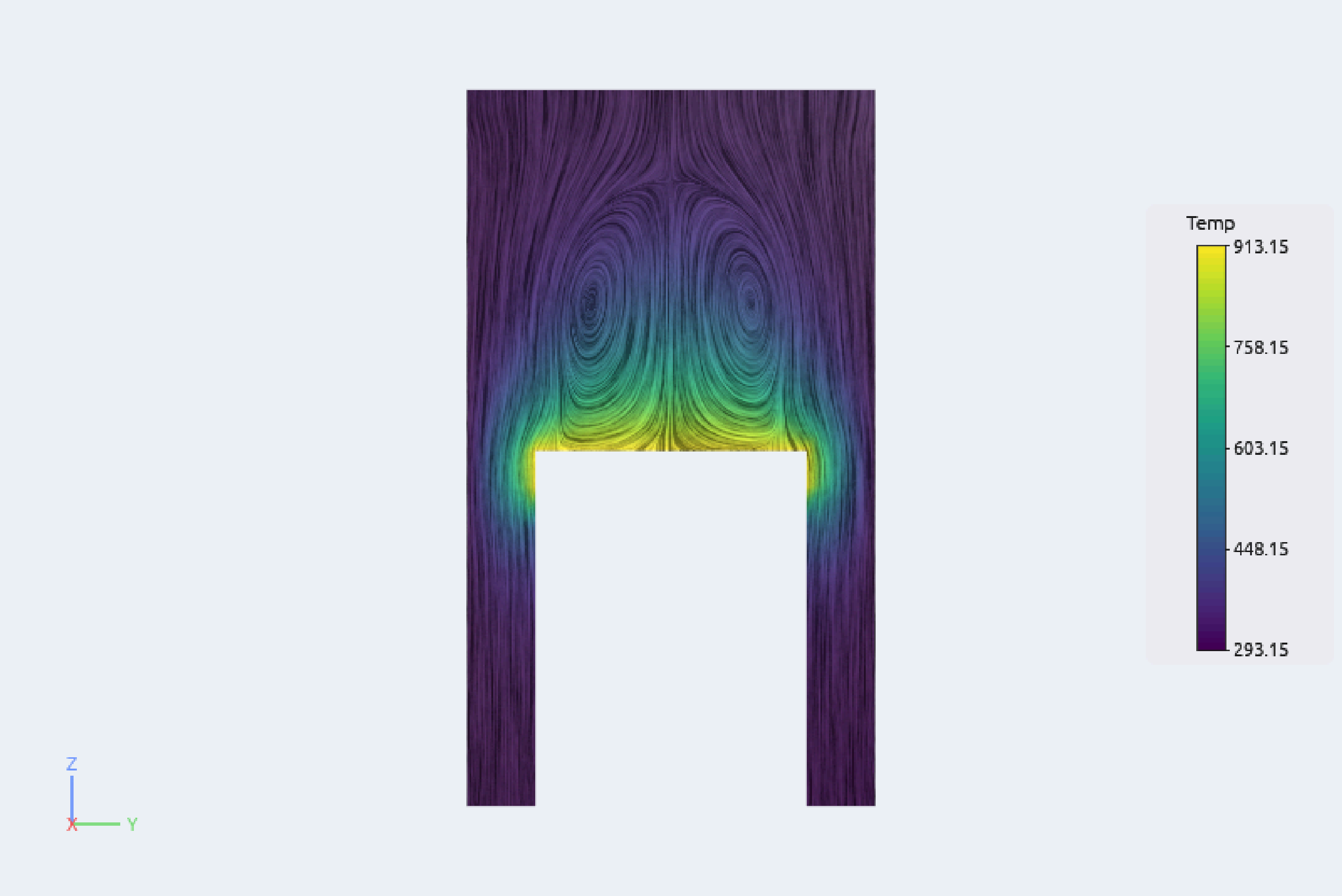
```
c.set(metallic=1.0, roughness=0.25, specular=0.5)
afx.lighting(environment="sky", environment_intensity=1.2, show_environment=True)
afx.lighting(environment="/data/hdri/studio_small.hdr")
```

Ray tracing

View › Lighting › Ray tracing path-traces the opaque surfaces: soft shadows, light from the sky or environment, and light bouncing between surfaces. It refines while the camera is still (the rasterizer draws while you move) and the status corner shows the samples done. Set the number of **samples** per pixel to stop at, the number of **bounces**, and where it runs: **auto** (the GPU when the data fits in its memory, else the CPU), **GPU** or **CPU**. Edges, points, lines, translucent objects, volumes and the legend are still drawn on top. Screenshots wait for the tracer to finish and are traced at the requested size.

```
afx.lighting(ray_tracing=True, ray_samples=256, ray_bounces=3, ray_device="auto")
afx.screenshot("traced.png", width=3840)
```

Surface LIC



The **Surface LIC** representation (Properties › Representation) draws a vector field as streaks on any surface while the surface stays colored by any array: the classic way to see a flow on a slice. Choose the vector array (**LIC vectors**), the streak **strength** and the **streak length** in pixels.

```
s.set(representation="Surface LIC", lic_vectors="V", color_by="Temp",
lic_length=30)
```

Volumes, point clouds and curved cells

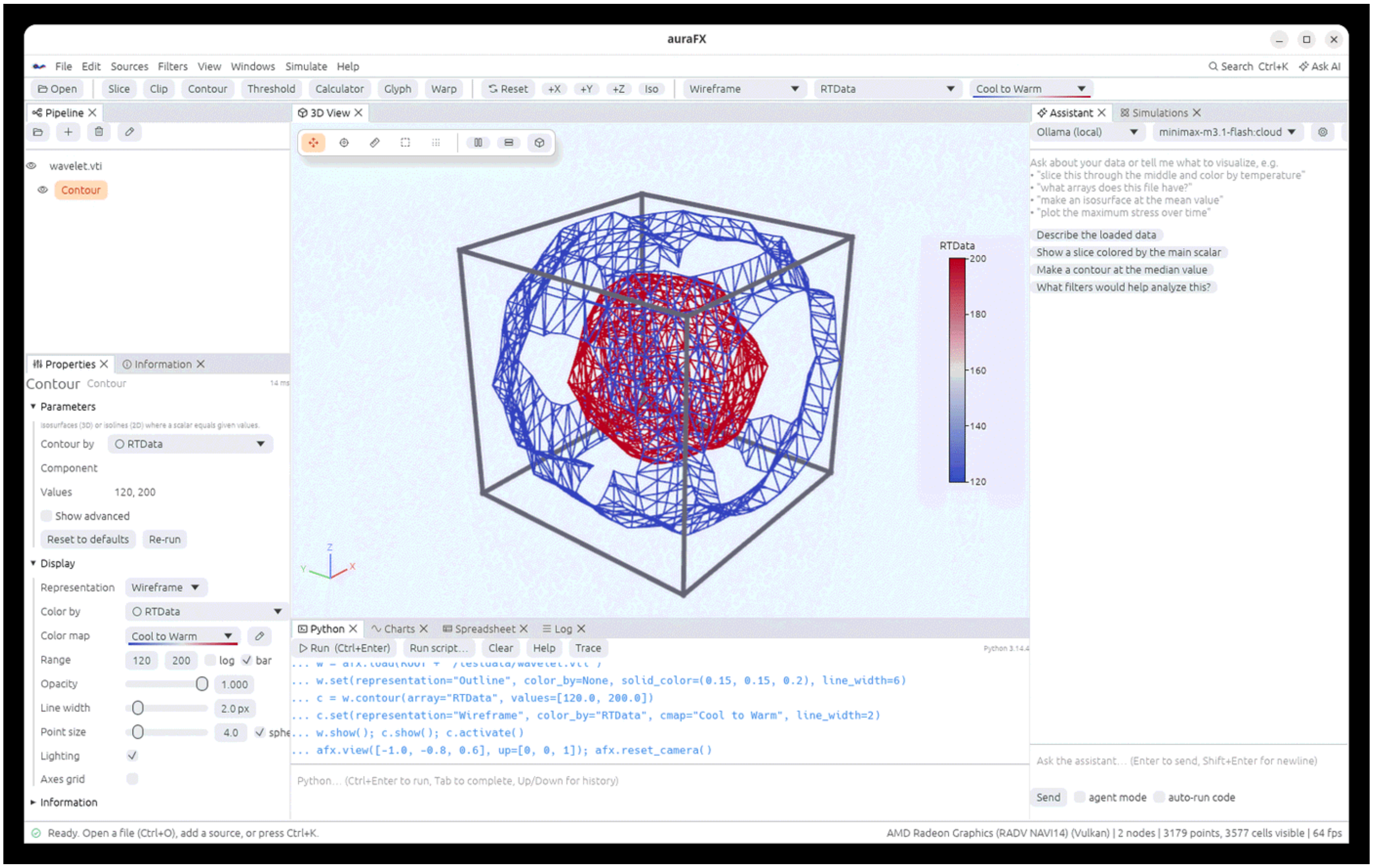
Representations

Representation in Properties (and the toolbar) chooses how an object is drawn:

Representation	Use
Surface	the outer surface, smooth-shaded (hard edges stay sharp: a 30° crease angle)
Surface With Edges	the surface with cell edges (**Edge color** in Properties, `edge_color=` in Python)
Wireframe	edges only
Points	the points, as flat dots or shaded spheres (**Point size**, **spheres**)
Outline	the bounding box (every box of AMR data)
Volume	GPU ray casting of image data (other meshes are resampled first)
Surface LIC	a vector field as streaks (see Lighting)
Molecule	atoms as spheres and bonds as cylinders (see Molecules)

Lighting can be switched off per object (flat colors, for 2-D data). **Axes grid** draws a labelled grid around the object with tick values.

Line width (Properties › Display, shown for Wireframe, Surface With Edges, Outline and line data; `line_width=` in Python) sets how many pixels wide lines, wireframes and cell edges are drawn, from 1 to 10 (any width from Python). It is a width on the screen, so lines keep their width when you zoom. Wider lines are drawn as flat bands with square ends (reaching half the width past each point), which fills the joints of bent lines. Wide lines show in screenshots and in pictures drawn on a server; picking still uses 1-pixel lines. For lines with a thickness in the scene (shaded, and growing as you zoom in), use the **Tube** filter.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp Reset +X +Y +Z Iso
Pipeline
wavelet.vti
Contour
3D View
RTData
200
180
160
140
120
Z
Y
X
Assistant
Simulations
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Send
agent mode
auto-run code
Properties
Information
Parameters
Show advanced
Reset to defaults
Re-run
Display
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4

Ollama (local) · minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Contour Contour 14 ms

Isosurfaces (3D) or isolines (2D) where a scalar equals given values.

Contour by: RTData
Component
Values: 120, 200
Representation: Wireframe
Color by: RTData
Color map: Cool to Warm
Range: 120 200 ☐ log ☑ bar
Opacity: 1.000
Line width: 2.0 px
Point size: 4.0 ☑ sphe
Lighting: ☑
Axes grid: ☐

```
... w = afx.load(ROOT + "/testdata/wavelet.vti")
... w.set(representation="Outline", color_by=None, solid_color=(0.15, 0.15, 0.2), line_width=6)
... c = w.contour(array="RTData", values=[120.0, 200.0])
... c.set(representation="Wireframe", color_by="RTData", cmap="Cool to Warm", line_width=2)
... w.show(); c.show(); c.activate()
... afx.view([-1.0, -0.8, 0.6], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

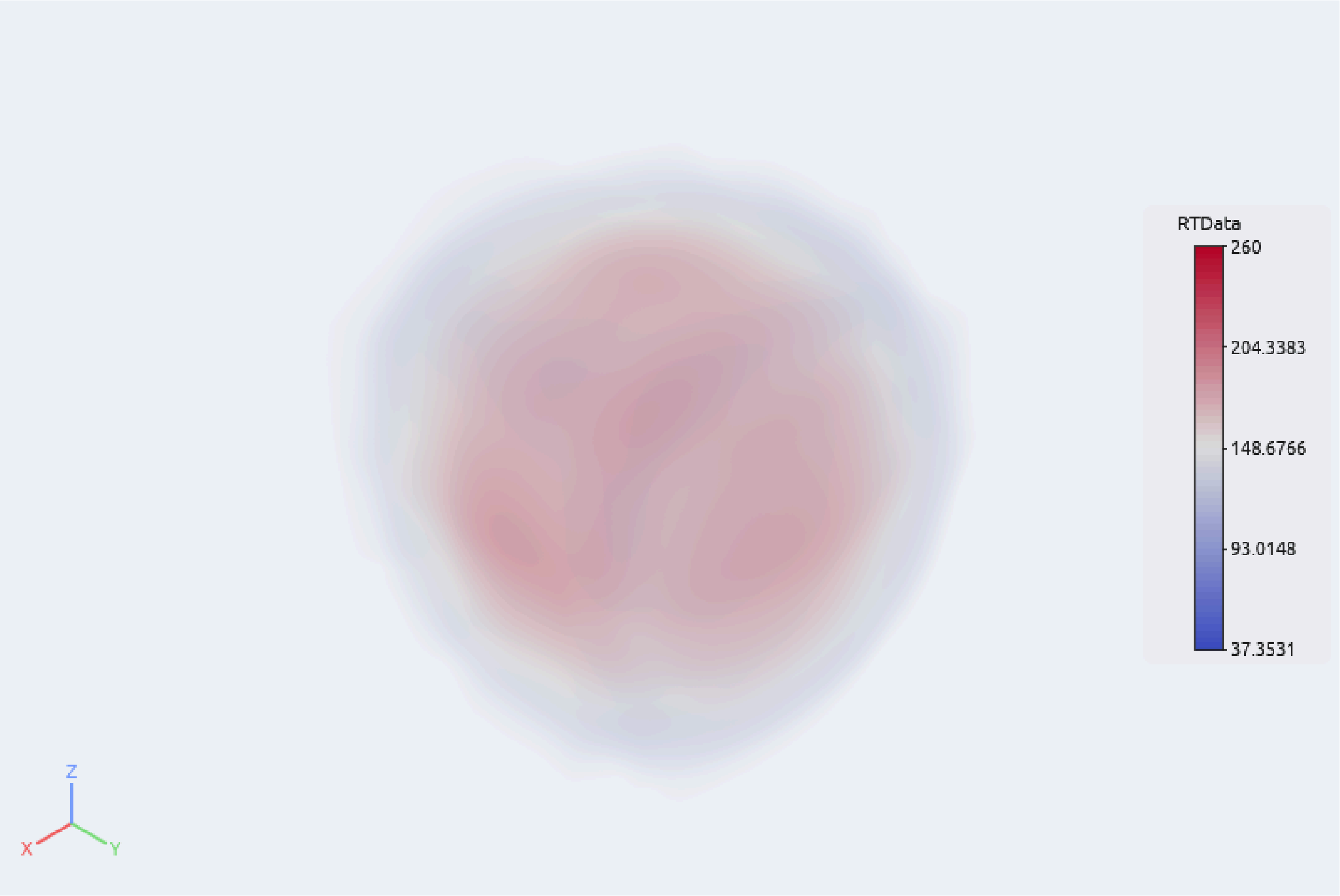
Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 3179 points, 3577 cells visible | 64 fps

```
c.set(representation="Wireframe", line_width=2)
w.set(representation="Outline", line_width=6)
```

Volume rendering



The **Volume** representation draws image data as a translucent cloud colored by the array: **Volume density** sets how opaque it is and **Transparent below** makes values under a fraction of the range invisible, which clears away the noise around a feature. Volumes are depth-merged with the surfaces around them.

```
w.set(representation="Volume", color_by="RTData", volume_opacity=2.0,
volume_threshold=0.3)
```

Point clouds and particles

LAMMPS dumps, XYZ files, PDB files and point sources open as points. Draw them as spheres sized by **Point size**, color them by type, and switch on Eye-Dome Lighting to see the structure of a dense cloud. The **Glyph** filter draws arrows or spheres scaled by an array; **Tube** gives lines a thickness.

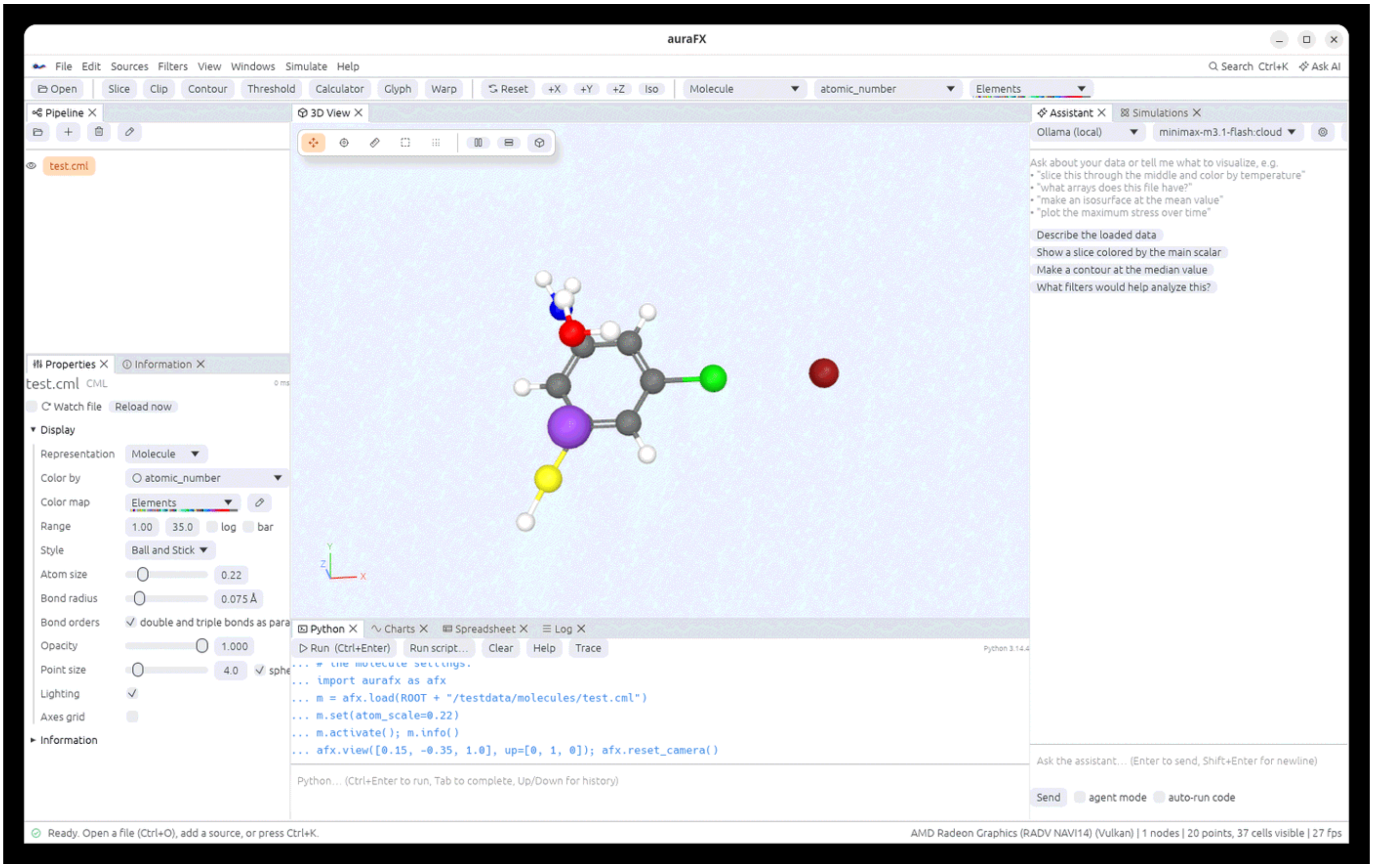
Molecules

Molecules are points (the atoms) with an atomic number array (`atomic_number` , or VTK's `Atomic Numbers`), and lines (the bonds). The PDB, XYZ, VASP and CML readers produce them (and the atoms block of a Gaussian cube file). A molecule opens in the **Molecule** representation, colored by element with the **Elements** color map (Jmol's colors). Files with more than 100 000 atoms open as points instead; switch to Molecule when you need it.

Settings (Properties › Display, or `node.set(...)`):

Setting	Python	Default	What it does
Style	`molecule_style`	Ball and Stick	**Ball and Stick**: atoms at a fraction of their van der Waals radius, bonds as thin sticks. **Van der Waals**: atoms at their full radius, no bonds (space filling). **Liquorice**: atoms and bonds as tubes of one width
Atom size	`atom_scale`	0.3	fraction of the van der Waals radius (Ball and Stick)
Bond radius	`bond_radius`	0.075 Å	radius of the sticks
Bond orders	`bond_orders`	on	double and triple bonds (a `bond_order` array, from CML files) drawn as two or three sticks

Each half of a bond takes the color of its atom. Color by any other atom array (charge, residue, B factor…) with **Color by** as usual. The radii and colors come from VTK's periodic table.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp Reset +X +Y +Z Iso
Molecule
atomic_number
Elements
Pipeline
test.cml
3D View
Y
Z
X
Assistant
Simulations
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

test.cml CML

☐ Watch file Reload now

Display

- Representation: Molecule
- Color by: atomic_number
- Color map: Elements
- Range: 1.00 35.0 ☐ log ☐ bar
- Style: Ball and Stick
- Atom size: 0.22
- Bond radius: 0.075 Å
- Bond orders: ☑ double and triple bonds as para
- Opacity: 1.000
- Point size: 4.0 ☑ sphe
- Lighting: ☑
- Axes grid: ☐

Information

```
... # the molecule settings:
... import aurafx as afx
... m = afx.load(ROOT + "/testdata/molecules/test.cml")
... m.set(atom_scale=0.22)
... m.activate(); m.info()
... afx.view([0.15, -0.35, 1.0], up=[0, 1, 0]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

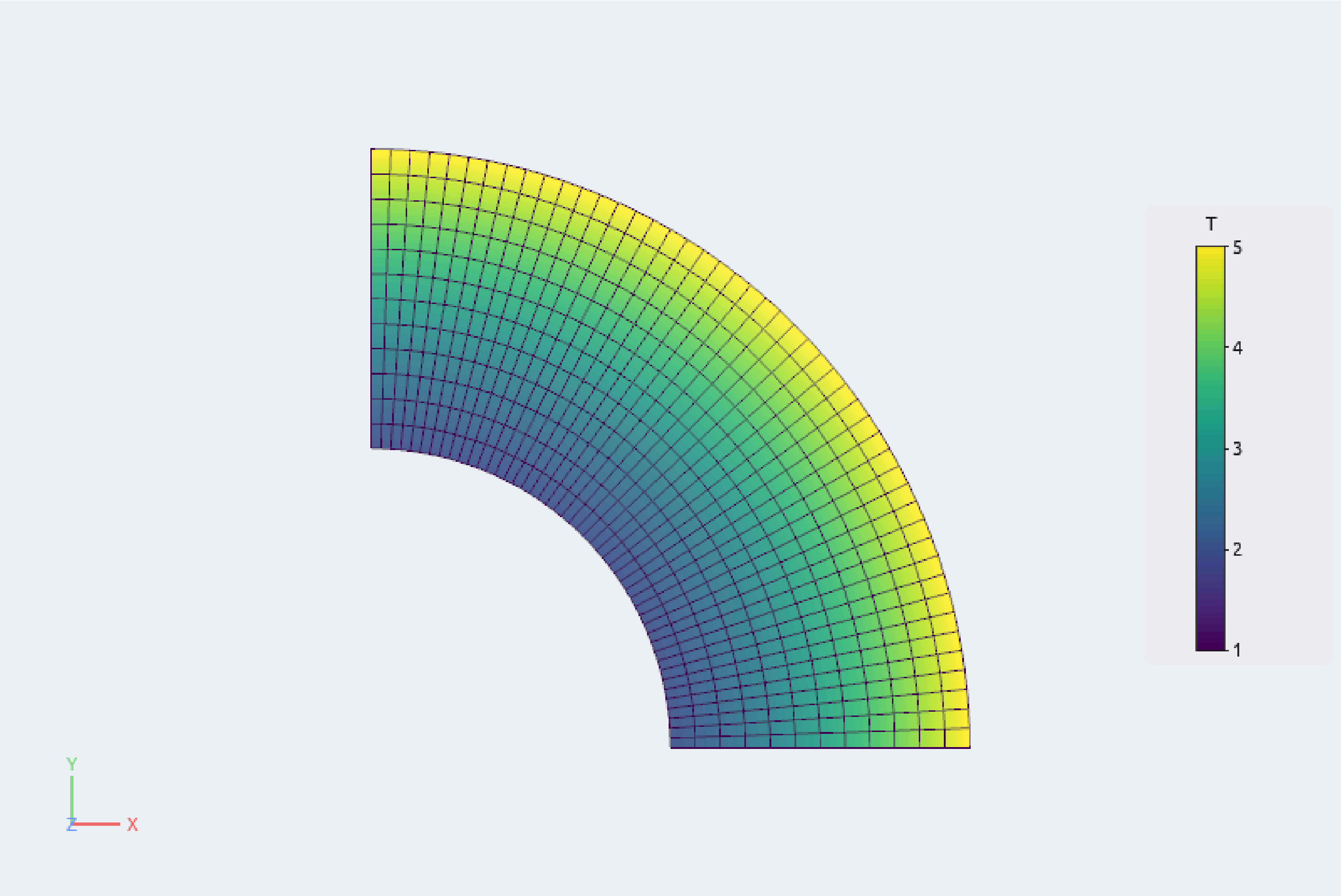
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 20 points, 37 cells visible | 27 fps

```
m = afx.load("molecule.cml")                 # Molecule representation, colored by
element
m.set(molecule_style="Van der Waals")        # or "Liquorice", "Ball and Stick"
m.set(atom_scale=0.25, bond_radius=0.1, bond_orders=False)
b = m.compute_molecule_bonds(tolerance=0.45) # bonds from distances
```

Filters › Compute Molecule Bonds finds bonds from distances: two atoms are bonded when they are closer than the sum of their covalent radii, each increased by half the **Tolerance** (0.45 Å). With **Absolute tolerance** off, each radius is multiplied by the tolerance instead. Bonds already there stay; only new ones are added. The bonds were verified on a test molecule with each tolerance setting.

Large structures (crystals, MD snapshots) draw fastest as **Points** with **spheres**. The Molecule representation builds geometry for every atom, which is fine up to a few hundred thousand atoms.

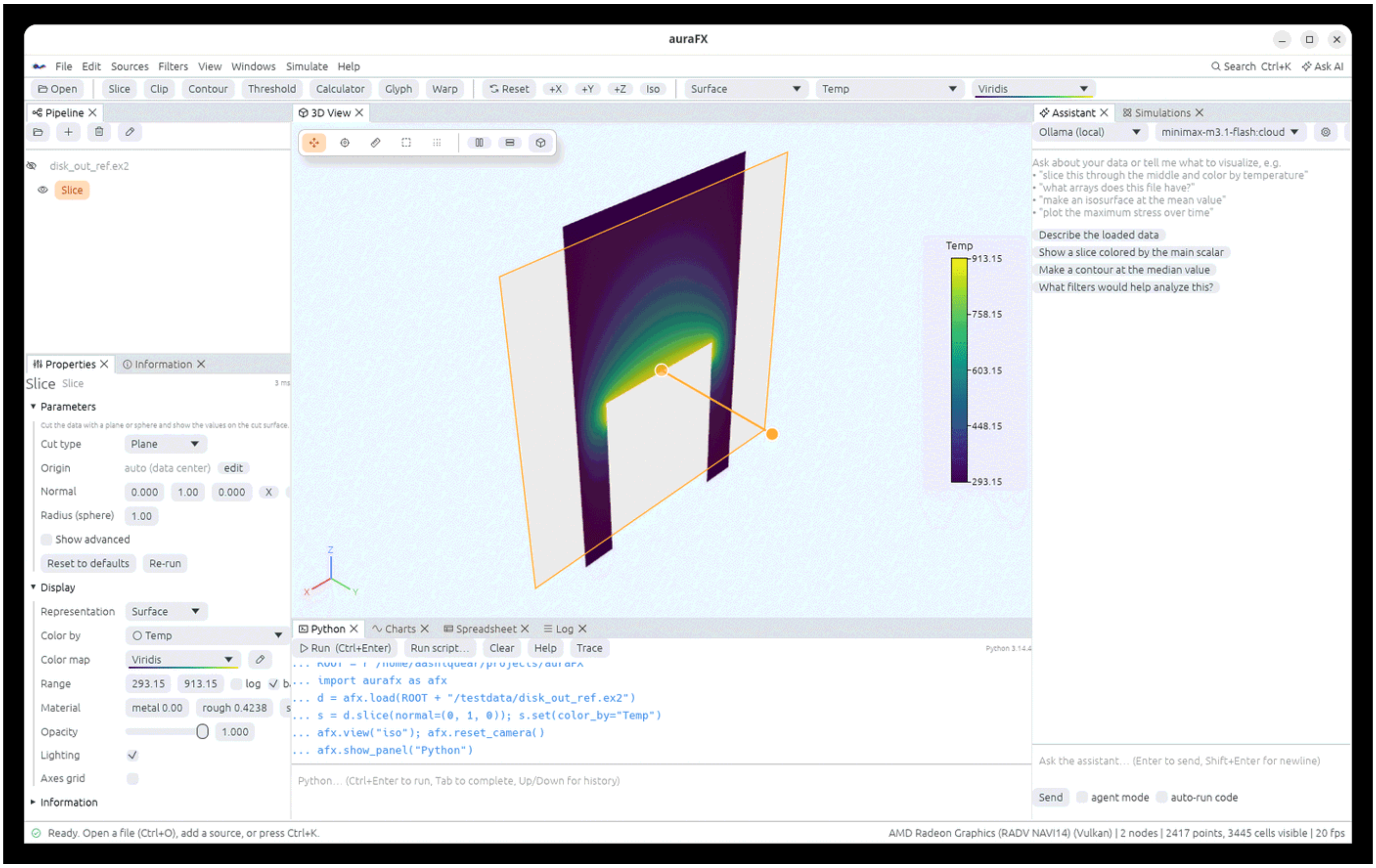
Curved cells



Quadratic cells and Lagrange or Bezier cells of any order (VTK types 68–80, with rational weights and per-cell degrees) are drawn curved. **Curved cells › subdivision** in Properties sets how finely: 0 draws straight edges between corners, 1 passes through the nodes, each further level halves the pieces. Slice, Clip, Contour, Probe and Integrate Variables follow the curved shape; the **Tessellate** filter turns curved cells into straight ones for filters that need them. The Information panel lists the cell types.

```
a.set(nonlinear_subdivision=3)
t = a.tessellate(level=2)
```

Python


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Temp
Viridis
Pipeline
3D View
Assistant
Simulations
disk_out_ref.ex2
Slice
Temp
913.15
758.15
603.15
448.15
293.15
Z
X
Y
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

```
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... s = d.slice(normal=(0, 1, 0)); s.set(color_by="Temp")
... afx.view("iso"); afx.reset_camera()
... afx.show_panel("Python")
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Slice Slice

Parameters

Cut the data with a plane or sphere and show the values on the cut surface.

Cut type	Plane
Origin	auto (data center)
Normal	0.000 1.00 0.000
Radius (sphere)	1.00
Show advanced	☐

Display

Representation	Surface
Color by	Temp
Color map	Viridis
Range	293.15 913.15 ☐ log
Material	metal 0.00 rough 0.4238
Opacity	1.000
Lighting	☑
Axes grid	☐

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 2417 points, 3445 cells visible | 20 fps

auraFX embeds Python with NumPy. The **Python** panel is a console: type a line and press **Ctrl+Enter** (or **Run**), use Tab to complete, Up/Down for history, **Run script…** to run a file, **Clear** to empty the log, **Help** for the cheat sheet. **Trace** turns what you did with the mouse into the equivalent code, which you can paste into a script. Everything the UI does is reachable from the `afx` module, and `aurafx --script file.py` runs a script at startup (the app stays open afterwards).

Packages installed into the embedded interpreter with `pip` (SciPy, pandas, PyTorch…) are available in the console.

Loading and the pipeline

```
d = afx.load("run.e")                     # a Node; afx.load("slices/img_*.png") stacks images
d = afx.load("/scratch/run.e", host="cluster")   # read on another machine (see Data on another machine)
afx.nodes(); afx.active(); afx.get("Slice")      # find nodes
s = d.slice(normal=(0, 1, 0))             # any filter: node.<filter>(**params)
s.set(origin=(0, 1, 0), opacity=0.5)      # parameters and display settings
s.inputs(); s.activate(); s.delete()
s.update(); s.output()                    # wait; the data (an afx.Data: points, arrays as NumPy)
s.info()                                  # arrays, ranges, bounds, counts
d.array("Temp")                           # a NumPy array; d.add_array("T_C", T - 273.15)
afx.apply_multi("append_datasets", [a, b])
afx.points_from(xyz, temperature=t)       # a node from NumPy
afx.filters(); afx.help_filter("contour")
afx.run()                                 # run whatever is pending
afx.undo(); afx.redo()
afx.save_state("work.afx"); afx.load_state("work.afx")
```

Display

```
d.set(color_by="Temp", cmap="Inferno", range=(300, 900), log_scale=True, representation="Surface With Edges")
d.set(opacity=0.6, point_size=6, points_as_spheres=True, line_width=2, axes_grid=True)
d.set(metallic=1, roughness=0.2, specular=0.5)           # material
d.set(representation="Volume", volume_opacity=2, volume_threshold=0.3)
d.set(representation="Surface LIC", lic_vectors="V")
d.set(nonlinear_subdivision=2)                           # curved cells
d.show(); d.hide(); d.set_name("inlet")
```

Views and camera

```
afx.view("iso"); afx.view("+x"); afx.view([1, 1, 0.5]); afx.reset_camera()
afx.split_view("right"); afx.active_view(1); d.send_to_view(1, keep=True)
afx.link_views(True); afx.match_views()
afx.layout("analyze"); afx.show_panel("Charts"); afx.hide_panel("Log");
afx.set_theme("light")
afx.annotate("Run 42", corner="upper left", size=18); afx.time_label(True);
afx.clear_annotations()
afx.lighting(shadows=True, ssao=True, edl=False, fxaa=True, tone_mapping="aces",
             environment="sky", ray_tracing=False)      # see Lighting
```

Time, charts, selection

```
afx.times(); afx.time(10)
afx.over_time(d, "EQPS", stat="max")      # returns (times, values) and plots them
afx.plot(x, y, label="T", title="Temperature"); afx.clear_plots()
afx.temporal_statistics(d)
afx.select(d, ids, "cells"); afx.selection(); afx.extract_selection();
afx.clear_selection()
afx.probe(d, (0, 0, 1))                   # values at a point
```

Pictures, animations, files

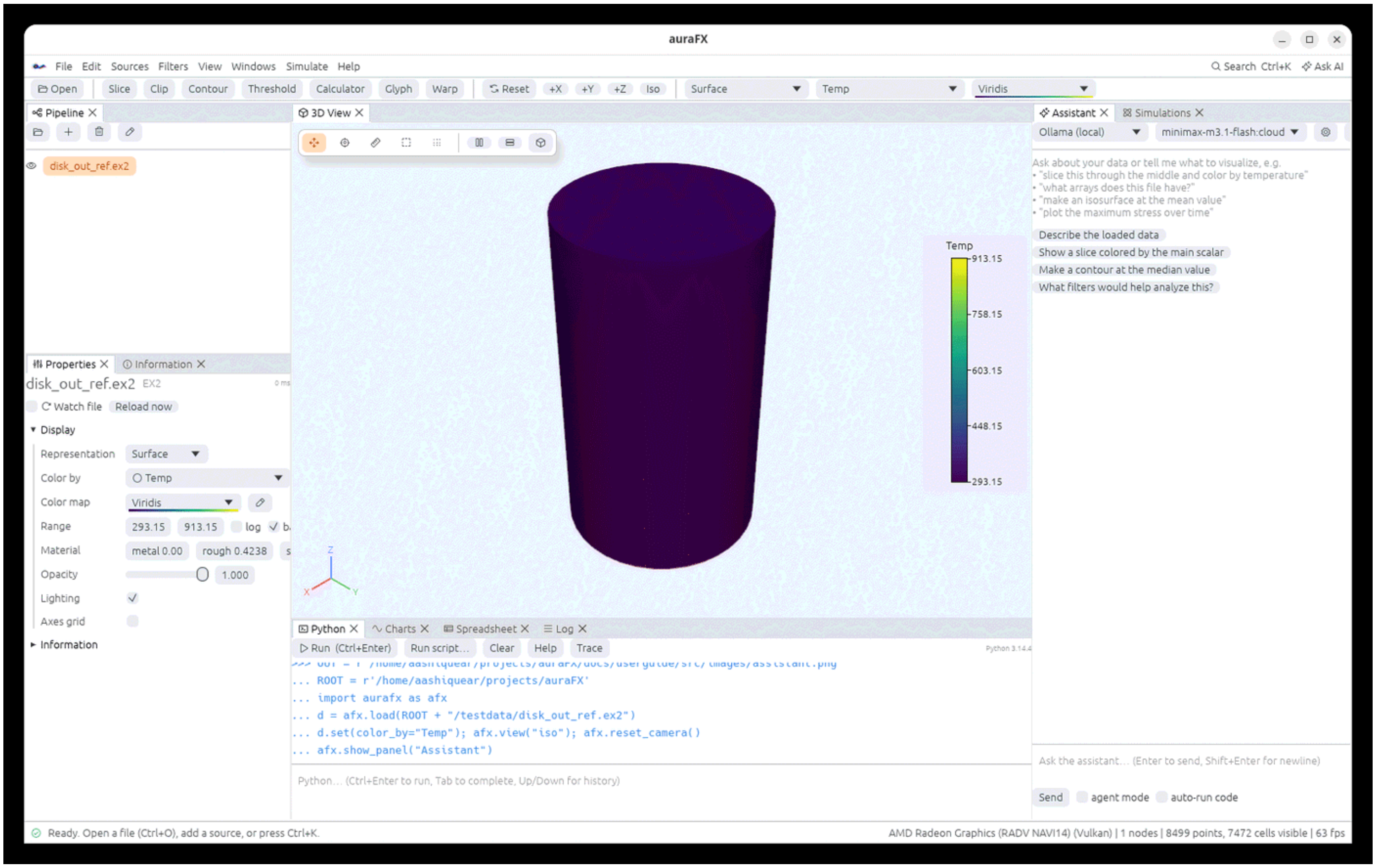
```
afx.screenshot("fig.png", width=3840, transparent=True, annotations=True)   #
waits until written
afx.screenshot("fig.svg")                 # vector picture
afx.export_scene("scene.glb")             # also .obj, .x3d, .wrl
afx.save_animation("run.mp4", fps=24); afx.save_animation("turn.gif", orbit=72)
afx.key_camera(0); afx.key_camera(1, interp="ease in-out"); d.key("opacity", 1,
0.2)
afx.animation(frames=120, time=True); afx.animation_time(0.5);
afx.clear_keyframes()
afx.save_animation("sweep.gif", cue=lambda t: c.set(values=[300 + 600 * t]),
frames=60)
d.save("out.vtu"); d.save("all.exo", all_times=True); d.save("all.vtkhdf",
all_times=True)
afx.colormaps(); afx.define_colormap(name, points);
afx.import_colormaps("maps.json")
```

Assistant, plugins, simulations, servers

```
afx.ask("plot the maximum stress over time", agent=True)
afx.plugins(); afx.load_plugin("tools.py")
afx.submit("lmp -in in.melt", host="rocm-server", workdir="~/runs", gpu="amd:0",
outputs="dump.*", auto_load=True)
afx.sweep("lmp -var T {T} -in in.melt", T=[300, 600, 900], host="cluster")
afx.jobs(); afx.job(id); afx.cancel(id); afx.gpus("rocm-server"); afx.hosts()
afx.servers(); afx.load_remote("cluster", "/scratch/run.e")
```

The **Help › Python API cheat sheet** window lists the same calls inside the app, and every function has a docstring (`help(afx.lighting)`).

AI assistant


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Temp
Viridis
Pipeline
disk_out_ref.ex2
3D View
Temp
913.15
758.15
603.15
448.15
293.15
Z
X
Y
Properties
Information
Watch file
Reload now
Display
Information
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

disk_out_ref.ex2 EX2

- Representation: Surface
- Color by: Temp
- Color map: Viridis
- Range: 293.15 913.15 ☐ log
- Material: metal 0.00 rough 0.4238
- Opacity: 1.000
- Lighting: ☑
- Axes grid: ☐

Ollama (local) · minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

```
>>> OUT = r'/home/aashiquear/projects/aurafx/docs/userguide/src/images/assistant.png'
... ROOT = r'/home/aashiquear/projects/auraFX'
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/disk_out_ref.ex2")
... d.set(color_by="Temp"); afx.view("iso"); afx.reset_camera()
... afx.show_panel("Assistant")
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 8499 points, 7472 cells visible | 63 fps

The **Assistant** panel (right side; **Windows › Assistant**, or the **Ask AI** button) is a chat with a language model that knows the app and your data. Ask in plain words: “slice this through the middle and color by temperature”, “what arrays does this file have?”, “plot the maximum stress over time”, “why does my contour look cracked?”. The assistant answers and, when something should happen in the app, writes Python using the `afx` API.

What the assistant sees

A text description of your pipeline: the objects, their filters and parameters, array names with ranges, bounds, cell types and time steps, plus the last error. It never receives the data itself. The description is rebuilt for every message.

Running the code it writes

Code appears in a block with a **Run code** button; nothing runs until you press it, unless **auto-run code** is ticked. With **agent mode** on, the assistant runs its code itself, looks at the result (values, errors, the pipeline) and keeps going until the task is done, so “find where the plate yields and plot it” becomes several steps without you in between.

```
afx.ask("make an isosurface at the mean temperature")
afx.ask("find where the plate yields and plot the max stress over time",
agent=True)
```

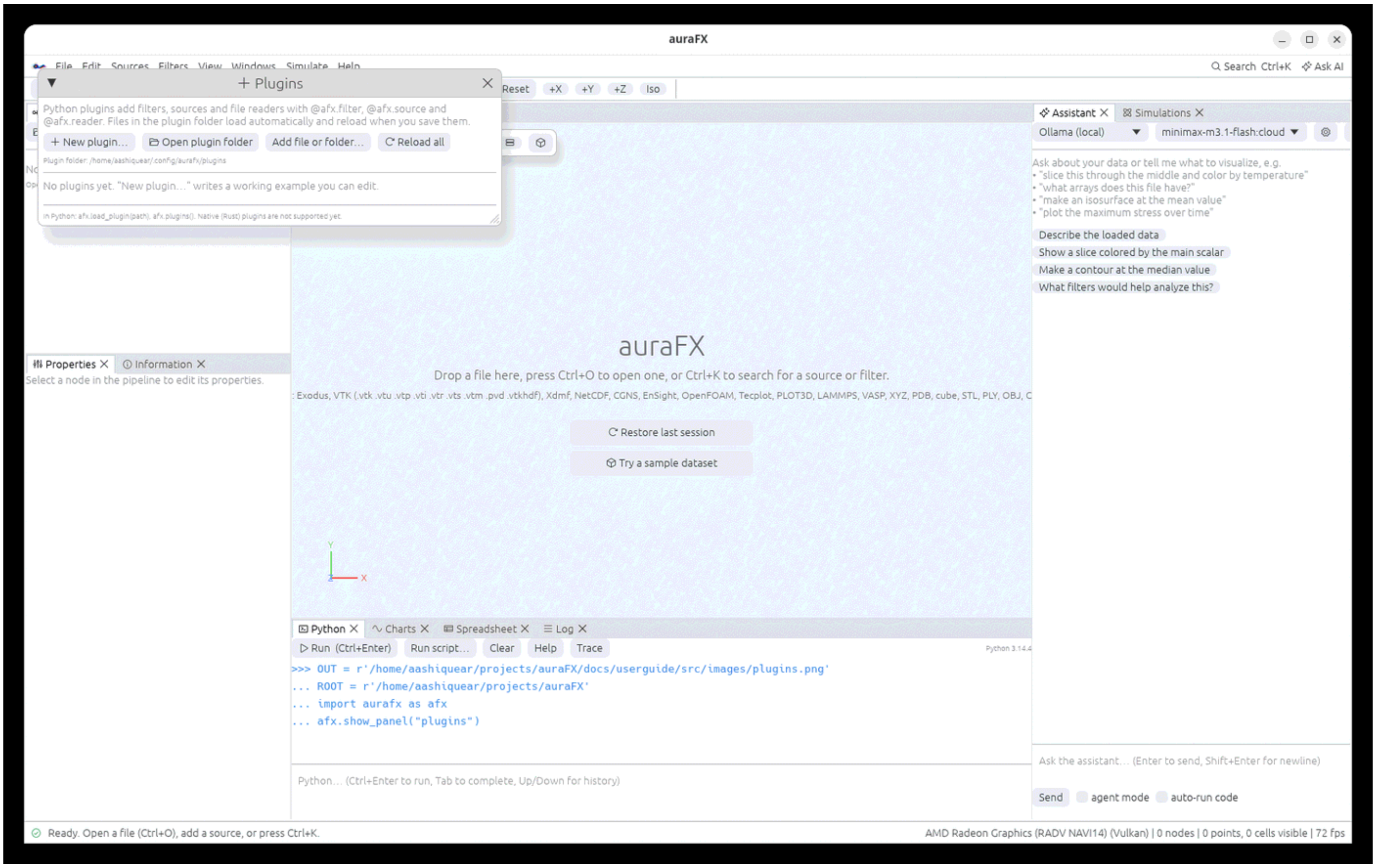
Providers

Choose the provider and model in the panel’s drop-downs or in **Settings**:

Provider	Setup
Ollama (local or cloud models)	the server URL from `OLLAMA_HOST` or Settings; the model list comes from the server
Anthropic	set `ANTHROPIC_API_KEY` or paste a key in Settings
OpenAI-compatible (OpenAI, OpenRouter, Groq, vLLM, LM Studio…)	the base URL and key in Settings

Answers stream as they are produced. The suggestion buttons under the welcome text are good first questions for a new file.

Plugins


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Plugins
+ New plugin…
Open plugin folder
Add file or folder…
Reload all
Reset
+X
+Y
+Z
Iso
Properties
Information
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Restore last session
Try a sample dataset
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Python plugins add filters, sources and file readers with @afx.filter, @afx.source and @afx.reader. Files in the plugin folder load automatically and reload when you save them.

Plugin folder: /home/aashiquear/.config/aurafx/plugins

No plugins yet. "New plugin…" writes a working example you can edit.

In Python: afx.load_plugin(path), afx.plugins(). Native (Rust) plugins are not supported yet.

Select a node in the pipeline to edit its properties.

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Drop a file here, press Ctrl+O to open one, or Ctrl+K to search for a source or filter.

: Exodus, VTK (.vtk .vtu .vtp .vti .vtr .vts .vtm .pvd .vtkhdf), Xdmf, NetCDF, CGNS, EnSight, OpenFOAM, Tecplot, PLOT3D, LAMMPS, VASP, XYZ, PDB, cube, STL, PLY, OBJ, C

```
>>> OUT = r'/home/aashiquear/projects/auraFX/docs/userguide/src/images/plugins.png'
... ROOT = r'/home/aashiquear/projects/auraFX'
... import aurafx as afx
... afx.show_panel("plugins")
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 0 nodes | 0 points, 0 cells visible | 72 fps

A plugin is a Python file that adds filters, sources and file readers. They appear in the Filters and Sources menus, in the command palette, in Python (`node.my_filter()`, `afx.my_source()`), in the Open dialog (readers) and to the AI assistant, like the built-in ones.

Where plugins live

- The plugin folder: **Edit › Plugins… › Open plugin folder** (`~/.config/aurafx/plugins` on Linux, the auraFX config folder on macOS and Windows). Every `.py` file in it is loaded.
- Folders listed in the `AURAFX_PLUGIN_PATH` environment variable.
- Files or folders added with **Add file or folder…** in the Plugins window.
- From a script: `afx.load_plugin("path/to/tools.py")`.

Plugins reload when you save them, and the objects that use their filters re-run. Files whose name starts with `_` are skipped (use them for shared helpers). A plugin that fails to load shows its error in the Plugins window and the Log; switch a plugin off with its checkbox. **New plugin…** writes a working example and opens it in your editor.

A filter

```
import aurafx as afx
import numpy as np

@afx.filter(category="Flow")
def speed(data, vectors=afx.ArrayParam(vector=True), name="speed"):
    """Magnitude of a vector array."""
    v = data.point_data[vectors]
    data.point_data[name] = np.linalg.norm(v, axis=1)
    return data
```

Keyword arguments become parameters in Properties; their type comes from the default: `float`, `int`, `bool`, `str`, a 3-tuple (a vector), `afx.Choice([...])`, `afx.ArrayParam(vector=False, cells=False)` (an array of the input) and `afx.Code("...")` (a text box). The docstring's first line is the description. Other decorator options: `label`, `description`, `keywords`, `id`, and `inputs=2` or `inputs=(2, None)` for filters with several inputs (the function receives a list).

`data` is an `afx.Data`, a copy of the input: `data.points` (N×3), `data.point_data["T"]`, `data.cell_data[...]`, `data.field_data[...]` (assign to add, `del` to remove), `data.n_points`, `data.n_cells`, `data.bounds`, `data.type`, `data.cells` (offsets, connectivity, VTK types), `data.blocks()`, `data.copy()`. The spellings `data.PointData.append(a, "name")` and `data.CellData` work too. Build new data with `afx.make_points`, `afx.make_polydata`, `afx.make_image`, `afx.make_grid` and `afx.make_multiblock`.

Sources and readers

```
@afx.source(category="Shapes")
def helix(turns=4.0, points=400):
    t = np.linspace(0, turns * 2 * np.pi, int(points))
    return afx.make_polydata(np.c_[np.cos(t), np.sin(t), t / 10], lines=
[list(range(len(t)))])

@afx.reader(extensions=["xyzt"], label="XYZ + temperature")
def read_xyzt(path):
    a = np.loadtxt(path, ndmin=2)
    d = afx.make_points(a[:, :3])
    d.point_data["T"] = a[:, 3]
    return d
```

Readers take precedence over the built-in readers for their extensions.

Rules

- Plugin functions run on a worker thread while the pipeline runs: do not call pipeline functions (`afx.load`, `node.slice()`, `show`...) inside them.
- Ids must be unique and cannot replace built-in filters.
- For one-off code, the **Programmable Filter**, **Programmable Source** and **Python Calculator** run Python stored in the pipeline (and in state files).
- Native (Rust) plugins are not supported yet.

File formats

Every reader was checked against a reference implementation: the same points, cells, bounds and value ranges on the same files. The comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Readers

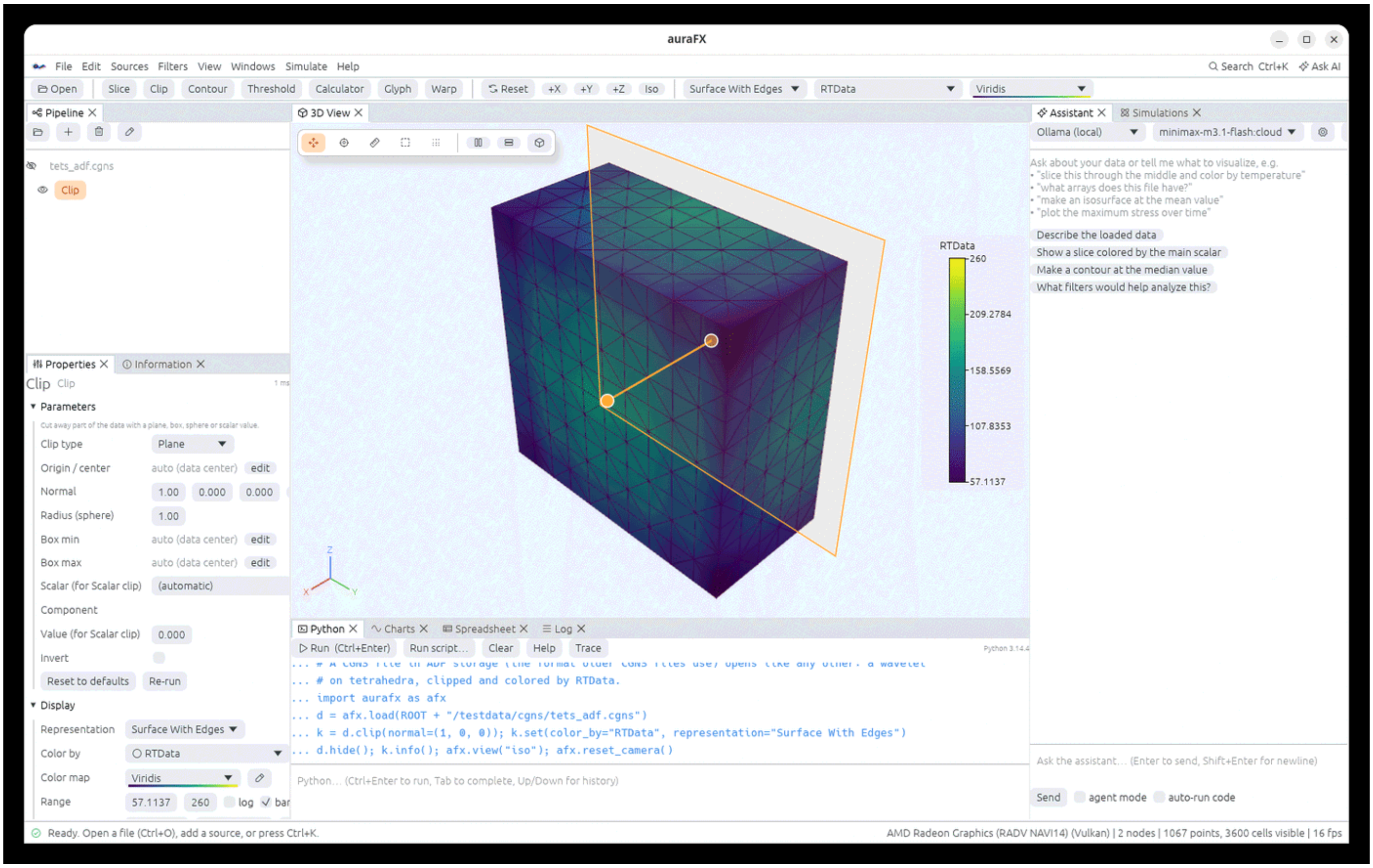
Format	Extensions	Notes
Exodus II	`.e .exo .ex2 .g .gen`	NetCDF-3 and NetCDF-4; blocks, side and node sets, nodal/element/global variables, time steps, displacements
VTK legacy	`.vtk`	ASCII and binary, all dataset types
VTK XML	`.vtu .vtp .vti .vtr .vts .vtt`, parallel `.pvtu …`, `.vtm`, `.pvd`	inline, appended, base64; zlib and LZ4; curved (Lagrange/Bezier) cells
AMR (adaptive mesh refinement)	VTK `.vthb .vth`, AMReX/BoxLib plotfiles (the folder, or its `Header`), FLASH and Chombo HDF5 files, Enzo (`.hierarchy`)	levels of boxes with cell data; coarse cells under finer boxes are left out; see below
Hyper tree grids	VTK `.htg` (XML version 2)	trees of refined cells; drawn as their leaves; see below
Cell grids (discontinuous Galerkin fields)	VTK `.dg`	cells whose fields are polynomials; drawn with the fields at the corners; see below
VTKHDF	`.vtkhdf .hdf`	image data, unstructured grids, poly data, partitioned collections with their assembly, time steps; chunked and gzip storage; no HDF5 library needed
OpenFOAM	`.foam`	ASCII/binary/gzip, decomposed and reconstructed cases, time directories, boundary patches
EnSight Gold	`.case .encas`	ASCII and C binary, all element types, structured blocks, time sets, wildcard

Format	Extensions	Notes
		file names
CGNS	`.cgns`	HDF5 and ADF storage; structured and unstructured zones, MIXED and NGON/NFACE sections, solutions, time steps
Xdmf 2/3	`.xmf .xdmf`	inline, HDF5 and raw binary data; collections; `xi:include` and XPath references (FEniCS/Dolfinx); HyperSlab and JOIN items
NetCDF (CF)	`.nc .nc4 .cdf`	classic and NetCDF-4; lon/lat grids on a sphere with cell bounds, curvilinear grids with 2-D longitude/latitude (POP, CICE, ROMS), rectilinear otherwise; time steps; packing; fill values become NaN (grey); see below
CAM-SE	`.nc` + connectivity file (found from the contents)	CESM atmosphere on its cubed sphere: lon/lat map or sphere, surface, midpoint or interface variables, one level or all; see below
MPAS	`.nc` (found from the file's contents)	MPAS ocean, atmosphere and land-ice output: cell centers joined into triangles, any vertical level or all of them, sphere or lon/lat map, time steps; see below
Tecplot	`.dat .plt .tec .tp`	ASCII (ordered and FE zones, BLOCK/POINT, cell-centered variables) and binary `.plt`; `.szplt` is not read
PLOT3D	`.xyz .x .p3d .bin + .q`	layout detected from the file (Fortran/C binary, byte order, precision, multi-grid, 2-D/3-D, IBLANK), ASCII
LAMMPS	`.dump .lammpstrj` `.atom` (+ `.gz`), `.data` `.lmp`	multi-frame dumps with any columns, triclinic boxes; data files with bonds
XYZ / extended XYZ	`.xyz .extxyz`	multi-frame; properties and lattice from the comment line

Format	Extensions	Notes
PDB	`.pdb` `.ent`	all atoms (hydrogens kept), MODELs as time steps, CONECT and distance bonds, residues and chains
Gaussian cube	`.cube`	the density as a volume in Å, plus the atoms with bonds
Chemical Markup Language	`.cml`	the atoms and the file's bonds with their orders (`bond_order`)
VASP	`POSCAR` , `CONTCAR` , `CHGCAR` , `LOCPOT` , `ELFCAR` , `PARCHG` (by name), `.vasp`	atoms with bonds and the lattice; volumes on the cell's axes (charge per Å³)
Images	`.png` `.jpg` `.bmp` `.tif`	as image data; multi-page TIFFs and `img_*.png` wildcards as volumes
GIS rasters	GeoTIFF `.tif` , Esri ASCII grid `.asc`	at their map coordinates, a cell per pixel, every band; no-data values become NaN (grey); see below
GIS vectors	Shapefile `.shp` , GeoJSON `.geojson` `.json` , KML `.kml` , KMZ `.kmz` , GeoPackage `.gpkg` , FlatGeobuf `.fgb` , GML `.gml`	points, lines and polygon outlines (or filled areas) with their attributes as cell data, a block per layer; see below
EBSD maps and microstructures	TSL/EDAX `.ang` , Oxford `.ctf` , `.h5ebsd` stacks, EDAX and Bruker `.h5` , `.dream3d`	crystal orientations on a square grid with phases and quality columns; voxel volumes and meshes with feature and phase arrays; see Grains and crystals
Surfaces	`.stl` `.ply` `.obj`	
CSV / TSV	`.csv` `.tsv` `.txt`	delimiter and header detection; tables with x, y, z columns become points

Numbered files (`run_0001.vtu` ...) open as a series when one of them is opened.

CGNS files come in two storage formats with the same `.cgns` extension: HDF5, the default since CGNS 3.4, and ADF, which older solvers and older CGNS libraries write. auraFX looks at the file and reads either one the same way, with **File › Open** or `afx.load("wing.cgns")` . Both the 64-bit ADF layout and the older 32-bit one are read, in either byte order. Links inside the file and links to other ADF files are followed. A link to an HDF5 file from an ADF file is not.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Pipeline
tets_adf.cgns
Clip
3D View
RTData
260
209.2784
158.5569
107.8353
57.1137
Z
X
Y
Assistant
Simulations
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Surface With Edges · RTData · Viridis

Ollama (local) · minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Clip — Clip

Parameters

Cut away part of the data with a plane, box, sphere or scalar value.

Clip type: Plane
Origin / center: auto (data center) edit
Normal: 1.00 0.000 0.000
Radius (sphere): 1.00
Box min: auto (data center) edit
Box max: auto (data center) edit
Scalar (for Scalar clip): (automatic)
Component:
Value (for Scalar clip): 0.000
Invert: ☐

Reset to defaults · Re-run

Display

Representation: Surface With Edges
Color by: RTData
Color map: Viridis
Range: 57.1137 260 ☐ log ☑ bar

```
... # A CGNS file in ADF storage (the format older CGNS files use) opens like any other: a wavelet
... # on tetrahedra, clipped and colored by RTData.
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/cgns/tets_adf.cgns")
... k = d.clip(normal=(1, 0, 0)); k.set(color_by="RTData", representation="Surface With Edges")
... d.hide(); k.info(); afx.view("iso"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

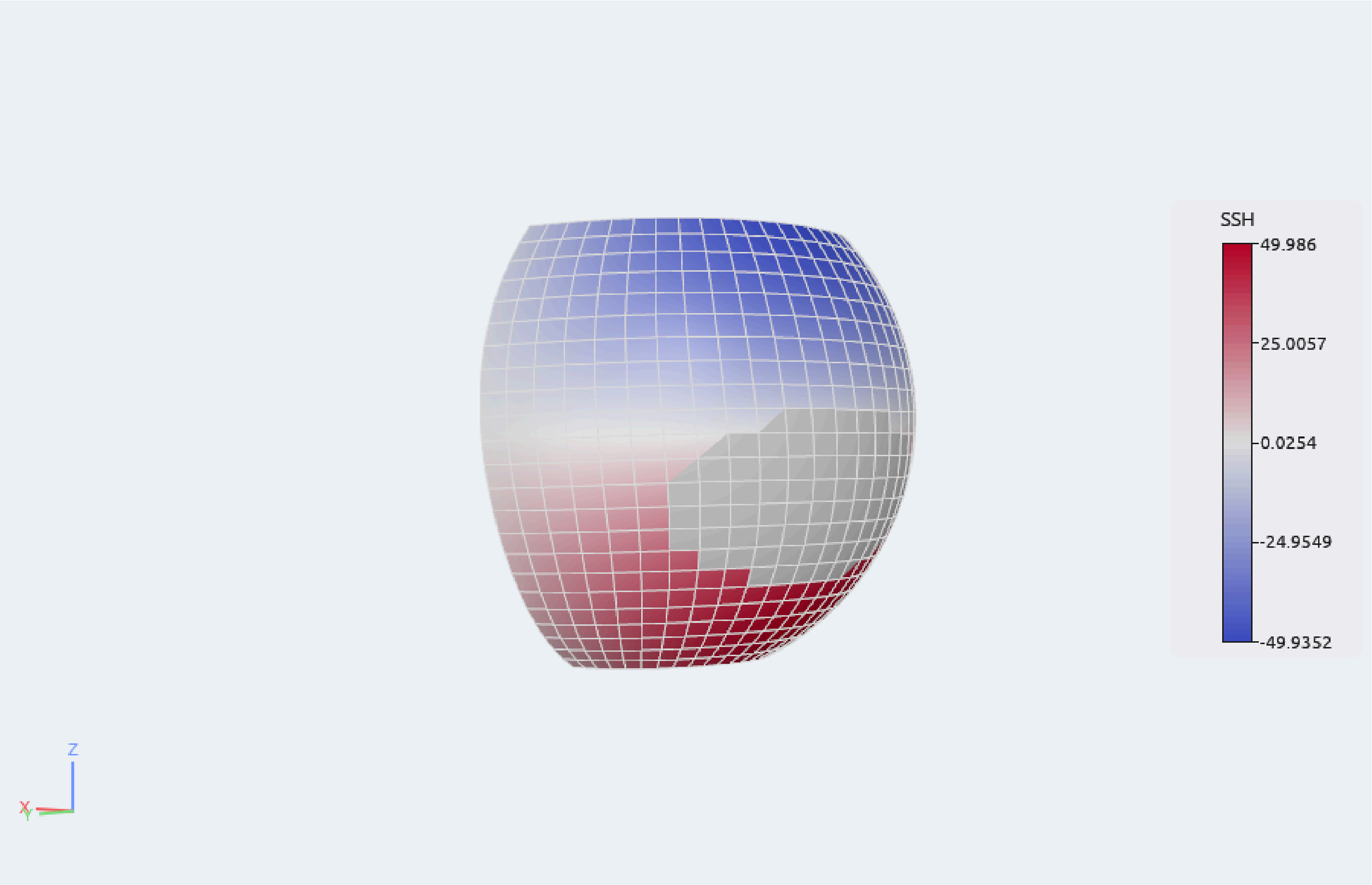
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 1067 points, 3600 cells visible | 16 fps

NetCDF climate and ocean files

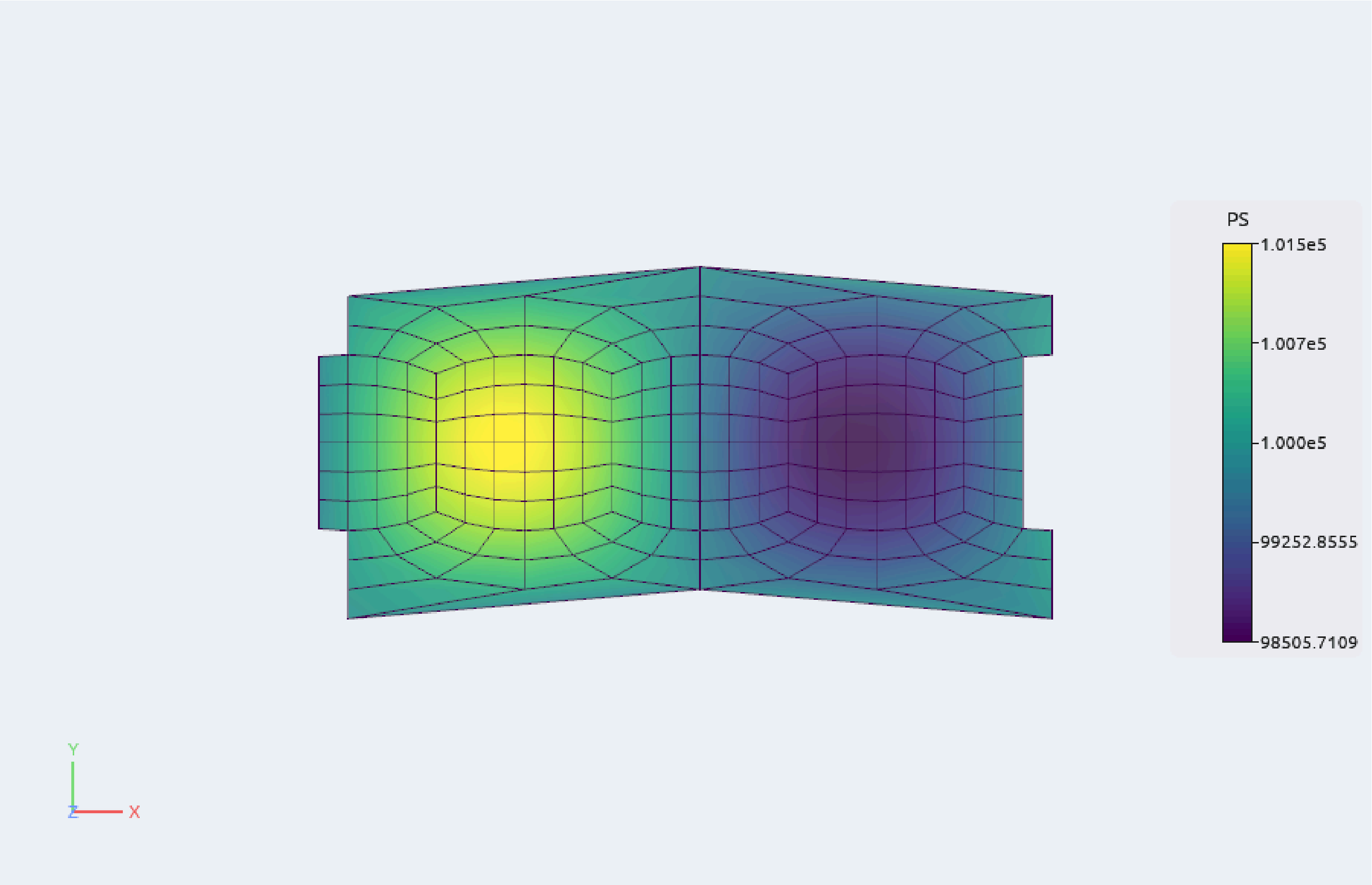
Files that follow the CF conventions are placed by their coordinates:

- Longitude and latitude axes put the grid on a sphere of radius 1. A third (vertical) axis becomes the radius. Depths marked `positive="down"` are turned around, and all levels are shifted so that none is below zero, so deeper levels lie further in (the deepest at the center).
- Ocean and sea-ice models such as POP (CESM's ocean), CICE and ROMS use index dimensions (`nlat`, `nlon`) and give each point's longitude and latitude in 2-D variables (`TLONG`, `TLAT`) named in the `coordinates` attribute. These become curvilinear grids on the sphere. The coordinate variables themselves are not shown as data.
- When the variables live on different grids (say `TEMP` on depth, latitude and longitude, and `SSH` on the surface only), **Reader options › Variables on** picks the grid. Python: `ds.set(variables="(nlat, nlon)")`. The grid with the most dimensions is read first.



CAM-SE output (the spectral-element core of CESM’s atmosphere model) comes as two files: the data, with columns `ncol` at `lat` / `lon`, and a connectivity file whose `element_corners` joins the columns into quads. Open the data file. auraFX finds the connectivity file in the same folder, or you can name it under **Reader options › Connectivity file** (Python: `ds.set(connectivity="ne30np4_connectivity.nc")`). It draws the grid on a flat lon/lat map:

- Quads that cross the 0/360° seam are moved to the side where they stick out least, on copies of their points, so the map runs a little past 0 and 360.
- **Variables** picks the surface variables `[time, ncol]`, or those on midpoint (`lev`) or interface (`ilev`) levels. With **All levels** on, the default, each quad becomes a column of hexahedra, the top level highest, and the `lev` (or `ilev`) array holds each point’s level. With it off, **Level** chooses one level.
- **Layout › Sphere** puts the columns on a sphere instead. There is no seam on a sphere.
- A single level lies at height 0.



MPAS output (from the ocean, atmosphere or land-ice models of the Model for Prediction Across Scales) is NetCDF too. When a `.nc` file has MPAS's mesh variables (`xCell` , `cellsOnVertex`), auraFX reads it as MPAS instead of as a CF grid:

- Each MPAS cell center becomes a point, and the cells around each MPAS vertex are joined into a triangle (or a quad, on quad meshes). On a sphere, the points lie at the planet's radius in meters.
- Variables on cells (`temperature` , `ssh` …) become point data, and variables on vertices (`relativeVorticity` …) become cell data. Time steps are numbered 0, 1, 2…, and the date of each step (`xtime`) is kept with the data.
- For other dimensions than time and position, the first index is used, except the vertical levels, which you choose with the reader options below.

Reader options

Some readers have settings for the file they read. They appear under **Properties › Reader options** when the file is selected, and from Python they are set like any other property, with `node.set(...)` . Changing one reads the file again; the choice is saved with the state file. **Reset to defaults** goes back to the file as first opened.

An MPAS file offers:

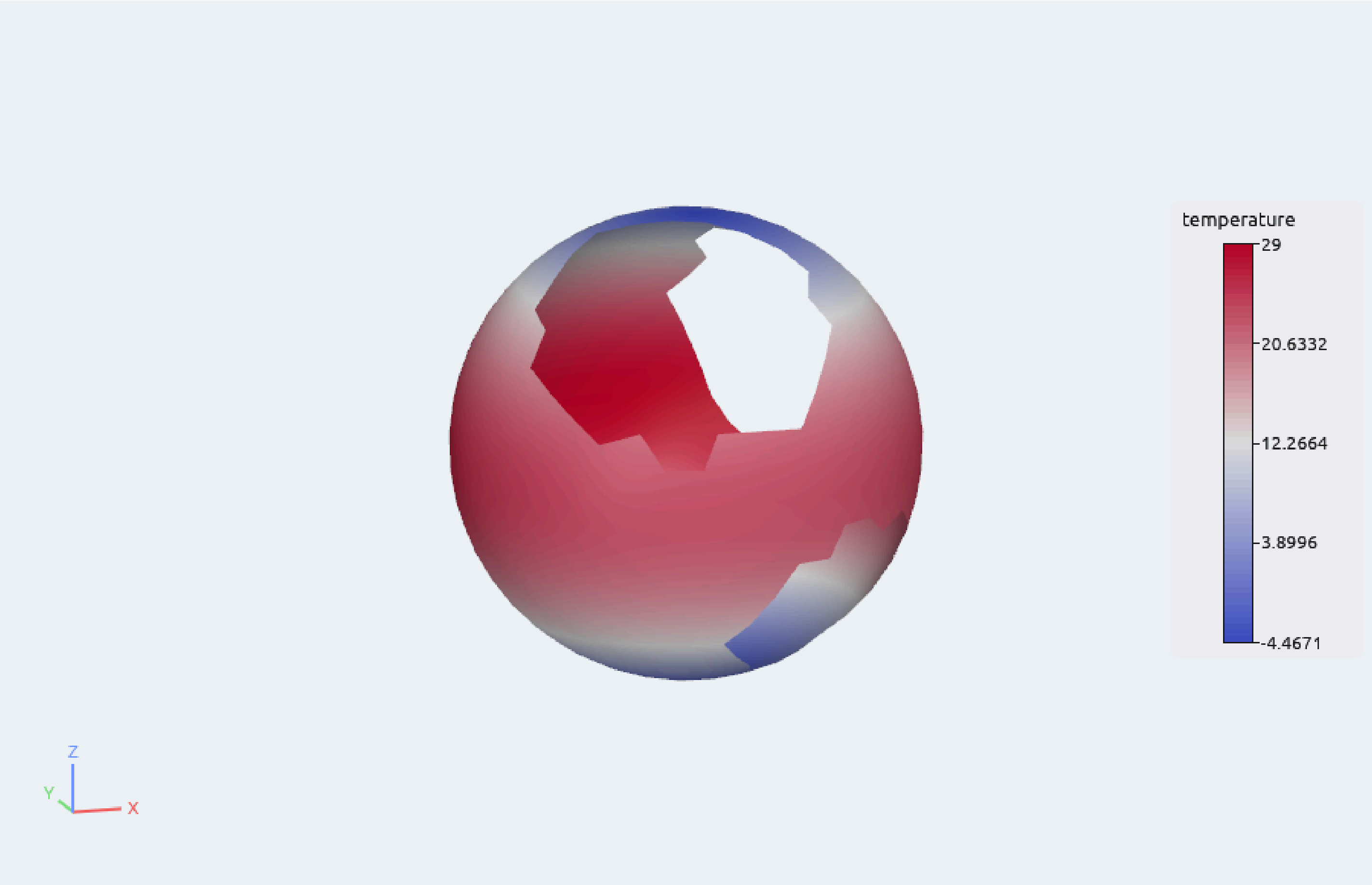
Option	Python	What it does
Vertical level	`vertical_level=2`	which level of 3-D variables is shown (0 is the top of the ocean, the bottom of the atmosphere)
All levels (multilayer)	`multilayer=True`	stacks every level: each triangle becomes a column of wedges, and points carry each level’s values
Layer thickness	`layer_thickness=10000`	the distance between levels in the multilayer view, in meters on the sphere and degrees on the map; 0 picks one so that the whole stack is easy to see
Layers go up (atmosphere)	`atmosphere=True`	stacks levels outwards (or upwards on the map) instead of downwards
Layout	`layout="Lat/Lon map"`	`"Sphere"` (default) or a flat map with longitude and latitude in degrees
Map center longitude	`center_lon=0`	the longitude in the middle of the map (180 by default)

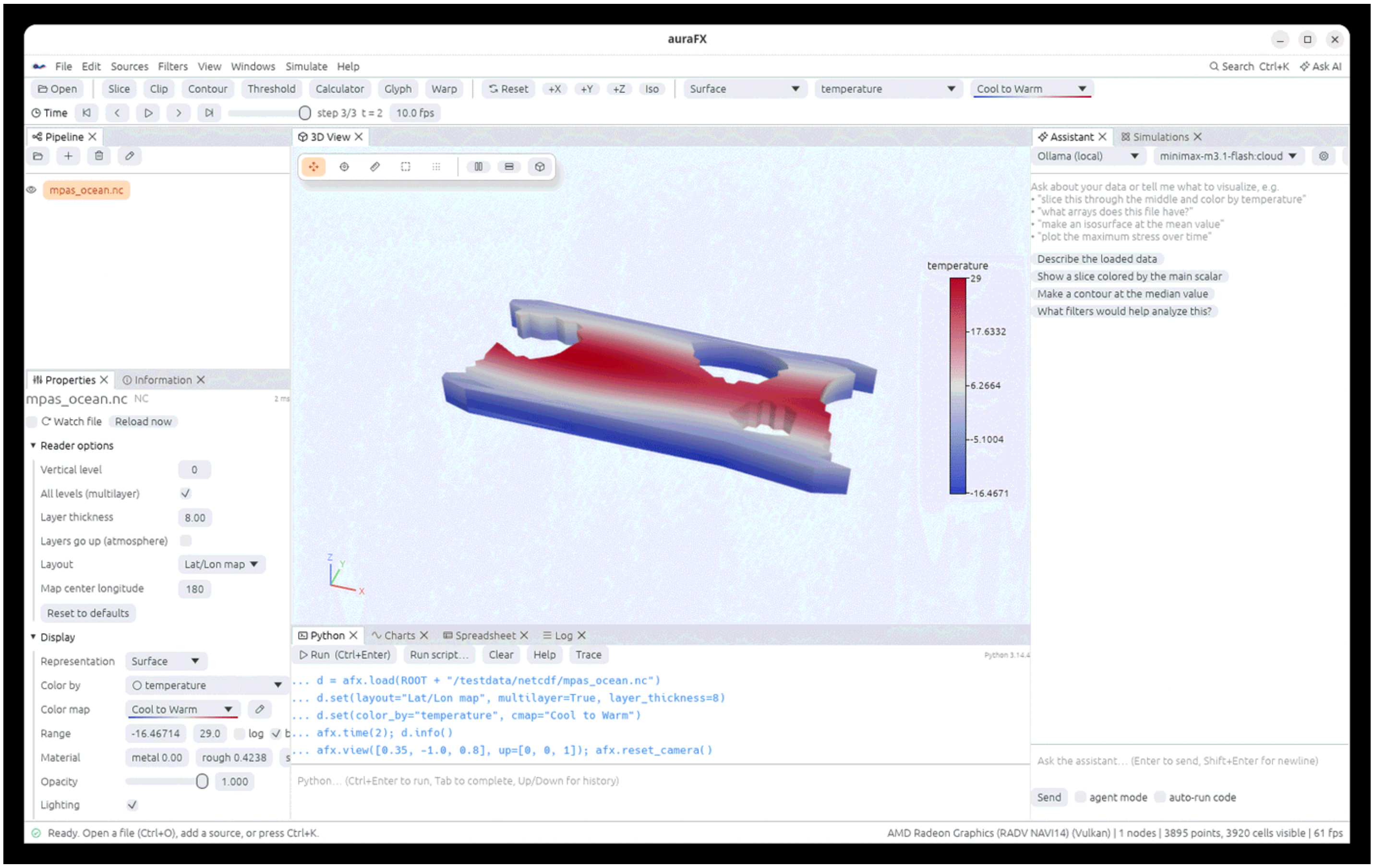
```
ocean = afx.load("output.nc")
ocean.set(vertical_level=10)                              # the 11th level down
ocean.set(layout="Lat/Lon map", multilayer=True)      # all levels, as a flat 3-D map
```

On the map, cells that cross its left or right edge are drawn on both sides, so the map reaches a little beyond 360°.

- The triangles at a mesh boundary and, when the file has `maxLevelCell`, the triangles that touch land are left out, so land shows as holes. In the multilayer view, the levels below the sea floor are left out too, and each point carries its level’s value.
- Reader options do not reach files opened on another machine through a server yet.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).


temperature
29
20.6332
12.2664
3.8996
-4.4671
Z
Y
X


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp Reset +X +Y +Z Iso
Surface
temperature
Cool to Warm
Time
step 3/3 t = 2
10.0 fps
Pipeline
mpas_ocean.nc
3D View
temperature
29
17.6332
6.2664
-5.1004
-16.4671
Z
Y
X
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Properties
Information
NC
2 ms
Watch file
Reload now
Reader options
Display
Reset to defaults
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

mpas_ocean.nc

Reader options

- Vertical level: 0
- All levels (multilayer): ☑
- Layer thickness: 8.00
- Layers go up (atmosphere): ☐
- Layout: Lat/Lon map
- Map center longitude: 180

Display

- Representation: Surface
- Color by: temperature
- Color map: Cool to Warm
- Range: -16.46714 29.0 ☐ log ☑ b
- Material: metal 0.00 rough 0.4238 s
- Opacity: 1.000
- Lighting: ☑

```
... d = afx.load(ROOT + "/testdata/netcdf/mpas_ocean.nc")
... d.set(layout="Lat/Lon map", multilayer=True, layer_thickness=8)
... d.set(color_by="temperature", cmap="Cool to Warm")
... afx.time(2); d.info()
... afx.view([0.35, -1.0, 0.8], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 3895 points, 3920 cells visible | 61 fps

GIS rasters

A GeoTIFF is a TIFF that also records where it lies on the map. auraFX notices these tags and reads the file as a map raster instead of as a picture. Esri ASCII grids (`.asc`), a common text format for elevation models, and ERDAS Imagine files (`.img`, common in remote sensing) are read the same way. Open them with **File › Open** or `afx.load("dem.tif")`.

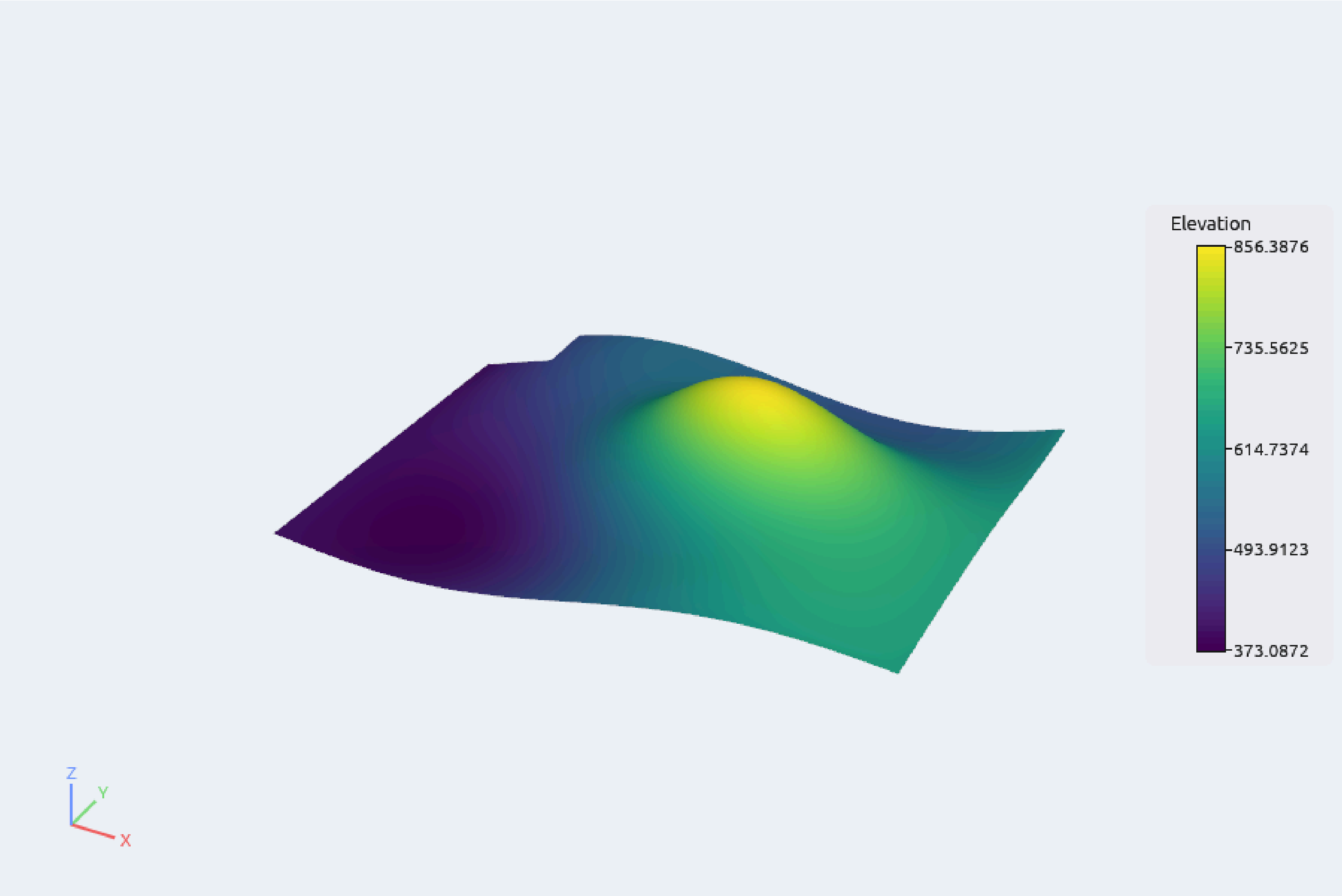
- The raster becomes image data with one cell per pixel, placed at its map coordinates (meters for UTM, degrees for longitude/latitude), with north up.
- Values are cell data. A single band, such as an elevation model, is named `Elevation`. In a file with several bands, the first is `Elevation` and the others are `Band 2`, `Band 3`… An RGB(A) image becomes one `Elevation` array with 3 or 4 components.
- Pixels equal to the file's no-data value become NaN and are drawn grey (after Warp By Scalar they leave a hole, since a NaN height has no place).
- The field data holds `NO_DATA_VALUE` per band and `MAP_PROJECTION`. For a GeoTIFF that is the EPSG code (`EPSG:32633`). For an ASCII grid it is the text of the `.prj` file beside it.
- 8- to 64-bit integer and floating-point bands are read, compressed or not, in strips or tiles, with the bands stored together or one after another. A pixel-is-point GeoTIFF is moved half a pixel to its corner, as GDAL does.
- ERDAS Imagine: every layer of the same size is a band (1-, 2- and 4-bit, 8- to 32-bit integer and 32- and 64-bit floating-point layers, in tiles, plain or run-length compressed), placed by its map information as GDAL does. The no-data value is the layer's "non-initialized" value. `MAP_PROJECTION` is the EPSG code for UTM and longitude/latitude on WGS 84, and the projection's name otherwise. The layers are bands as above: an RGB `.img` gives `Elevation`, `Band 2` and `Band 3` (it does not say which layer is which color). Data kept in a separate `.ige` file (very large images) is not read.

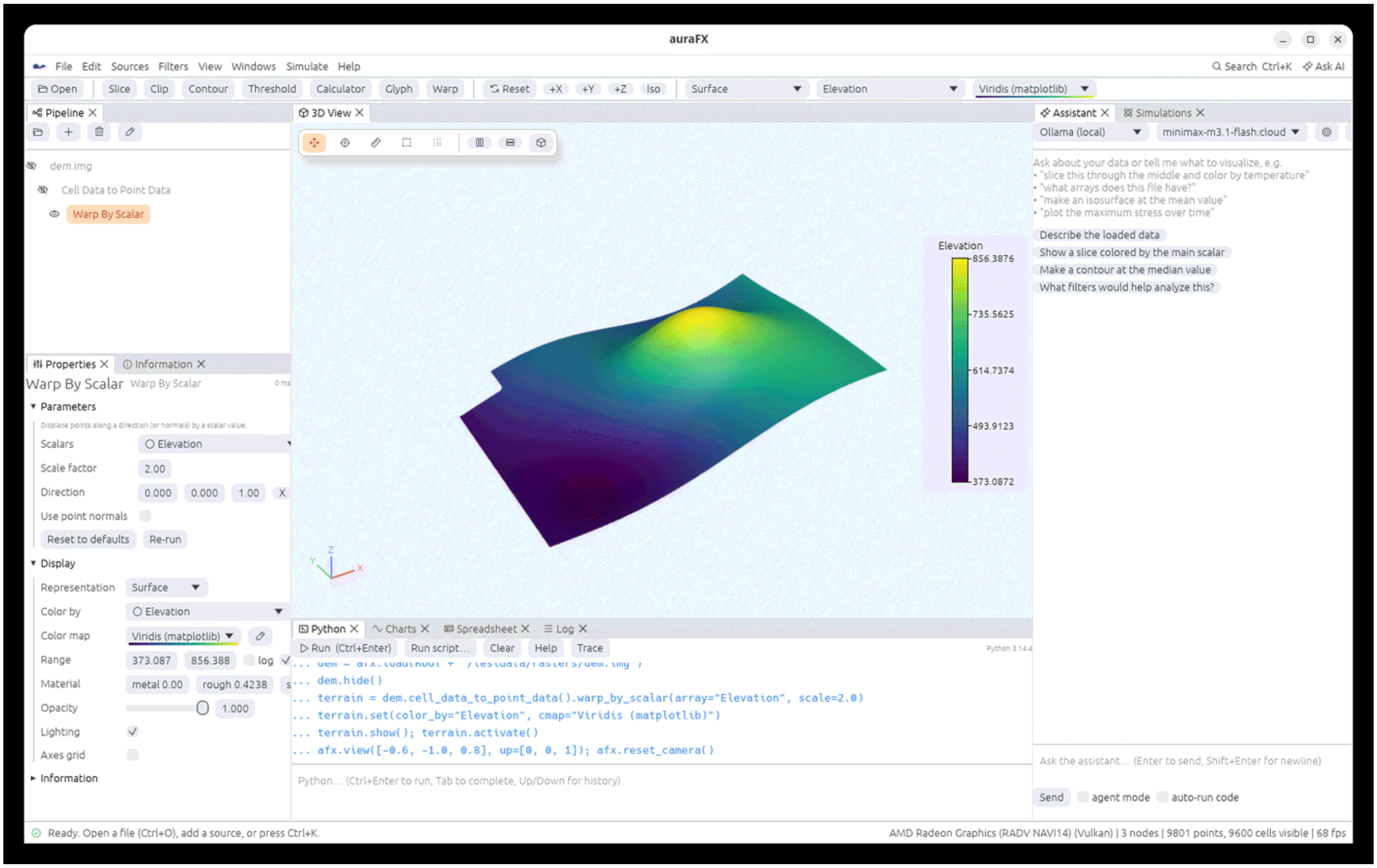
To see an elevation model as terrain, move the values to the points with **Cell Data to Point Data**, then raise them with **Warp By Scalar**:

```
dem = afx.load("dem.tif")                          # or dem.img (ERDAS Imagine)
terrain = dem.cell_data_to_point_data().warp_by_scalar(array="Elevation",
scale=1.0)
terrain.set(color_by="Elevation")
```

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Other GDAL raster formats (NetCDF through GDAL, JPEG 2000…) are not read yet.


Elevation
856.3876
735.5625
614.7374
493.9123
373.0872
Z
Y
X


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp
Reset +X +Y +Z Iso
Surface
Elevation
Viridis (matplotlib)
Pipeline
3D View
Assistant
Simulations
dem.img
Cell Data to Point Data
Warp By Scalar
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Warp By Scalar Warp By Scalar

Parameters

Displace points along a direction (or normals) by a scalar value.

Scalars: Elevation

Scale factor: 2.00

Direction: 0.000 0.000 1.00

Use point normals: ☐

Display

Representation: Surface

Color by: Elevation

Color map: Viridis (matplotlib)

Range: 373.087 856.388 ☐ log

Material: metal 0.00 rough 0.4238

Opacity: 1.000

Lighting: ☑

Axes grid: ☐

```
... dem.hide()
... terrain = dem.cell_data_to_point_data().warp_by_scalar(array="Elevation", scale=2.0)
... terrain.set(color_by="Elevation", cmap="Viridis (matplotlib)")
... terrain.show(); terrain.activate()
... afx.view([-0.6, -1.0, 0.8], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 9801 points, 9600 cells visible | 68 fps

GIS vectors

Maps of points, lines and areas, such as countries, roads, rivers, wells or survey sites, open like any other file:

Format	Files
ESRI Shapefile	`name.shp` with `name.dbf` (attributes) and `name.prj` (coordinate system) beside it; open the `.shp`
GeoJSON	`.geojson`, or a `.json` file that holds GeoJSON
KML (Google Earth)	`.kml`; each folder is a layer
KMZ	`.kmz`: a zipped KML (Google Earth's usual download); the KML document inside is read
GeoPackage	`.gpkg`; each feature table is a layer
FlatGeobuf	`.fgb`: a fast binary format for large vector data; one layer
GML	`.gml` (GML 2, 3 and 3.2, as GDAL and most web feature services write it); each feature type is a layer, typed by the `.xsd` schema beside the file when there is one
MapInfo table	`name.tab` with `name.map` (geometry), `name.id` (its index) and `name.dat` (attributes) beside it; open the `.tab`
MapInfo interchange	`name.mif` (geometry, in text) with `name.mid` (attributes) beside it; open the `.mif`
Esri file geodatabase	a `name.gdb` folder; open the folder (or any file in it). Each feature class and table is a layer

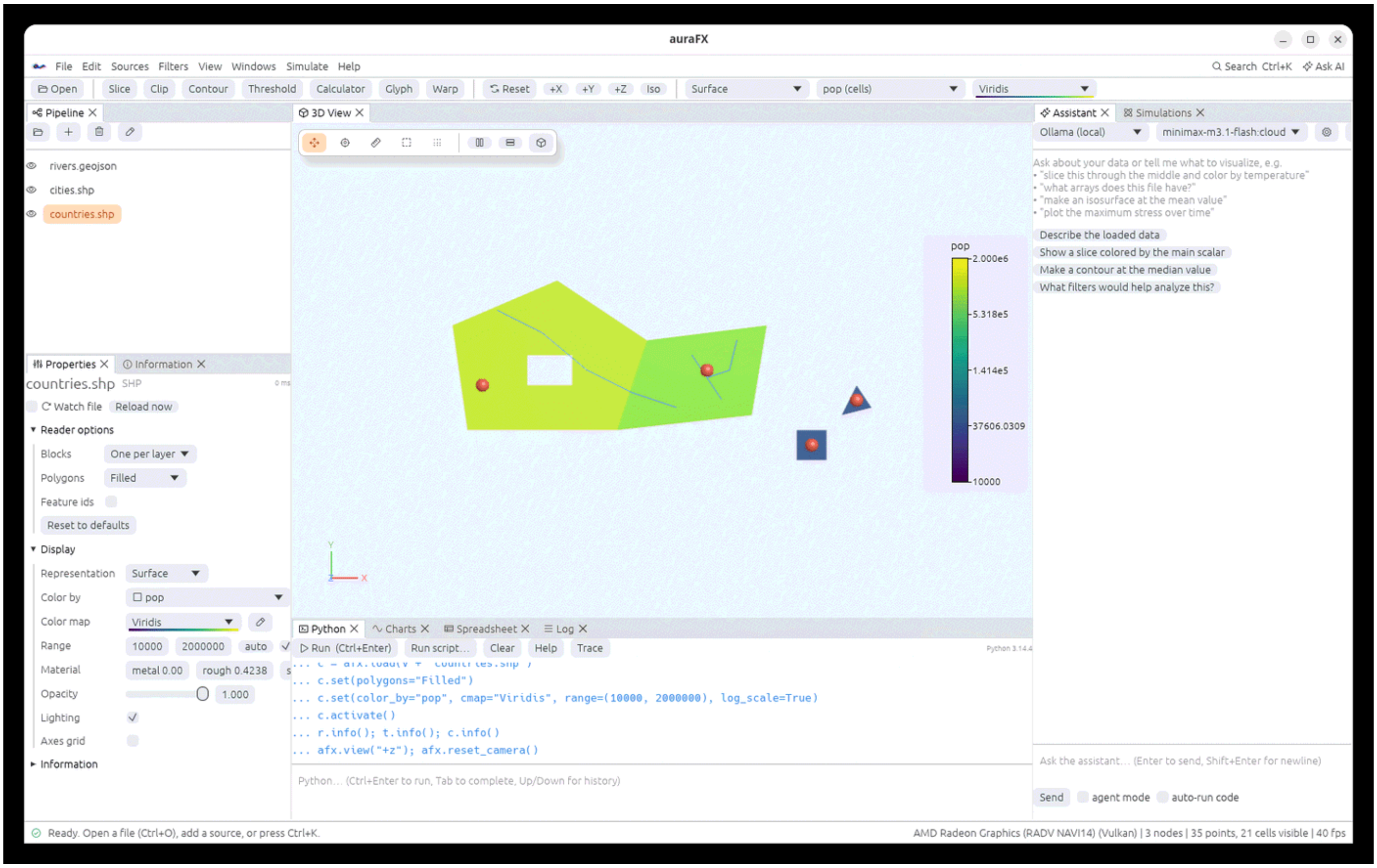
What you get:

- The coordinates stay as stored: longitude and latitude in degrees for most GeoJSON and KML files, or meters for projected shapefiles. Heights (the third coordinate) are kept, so a file with heights in meters on a map in degrees looks very tall. Flatten it with **Transform** (scale z by 0) if that gets in the way.
- Points become vertices, lines become polylines, and polygon rings become closed lines.
- Each feature's attributes become cell data on its cells: whole numbers as integers, decimals as doubles, and anything else (names, dates) as text. 64-bit integers stay numbers.
- `MAP_PROJECTION` in the field data holds the coordinate system: the `.prj` text, the EPSG code of a GeoPackage, FlatGeobuf or GML layer, the coordinate system text of a file geodatabase layer, or `EPSG:4326` (longitude/latitude) for GeoJSON and KML.
- File geodatabases store coordinates on a fine grid and add `Shape_Length` and `Shape_Area` attributes to lines and areas; both come through as stored.

- MapInfo tables store coordinates as whole numbers on a grid set by the file (a millionth of a degree for longitude and latitude). auraFX converts them as GDAL does, rounding to that grid. A MapInfo region's rings are grouped into areas with holes: a `.tab` file says which rings are holes. In a `.mif` file, a ring inside a larger one is a hole, as GDAL decides it.
- GML in a geographic coordinate system named the long way (`urn:ogc:def:crs:EPSG::4326`) stores latitude first. auraFX swaps it back to longitude, latitude, as GDAL does, so GML lines up with the other formats.

Reader options (Properties › Reader options, or `node.set(...)`):

Option	Choices
Layer (files with several layers)	**All** or one layer by name
Blocks	**One per layer** (default: one block per layer, a row of attributes per feature), or **One per feature**: one block per feature, slow for thousands of features
Polygons	**Outlines**, or **Filled**: triangles covering each area, with holes left open, to color areas by an attribute
Feature ids	adds the feature number as `_vtkPedigreeIds`


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
pop (cells)
Viridis
Pipeline
rivers.geojson
cities.shp
countries.shp
3D View
pop
2.000e6
5.318e5
1.414e5
37606.0309
10000
Y
X
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

countries.shp SHP

☐ Watch file Reload now

Reader options

- Blocks: One per layer
- Polygons: Filled
- Feature ids: ☐
- Reset to defaults

Display

- Representation: Surface
- Color by: pop
- Color map: Viridis
- Range: 10000 2000000 auto ✓
- Material: metal 0.00 rough 0.4238
- Opacity: 1.000
- Lighting: ✓
- Axes grid: ☐

Information

```
... c.set(polygons="Filled")
... c.set(color_by="pop", cmap="Viridis", range=(10000, 2000000), log_scale=True)
... c.activate()
... r.info(); t.info(); c.info()
... afx.view("+z"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"

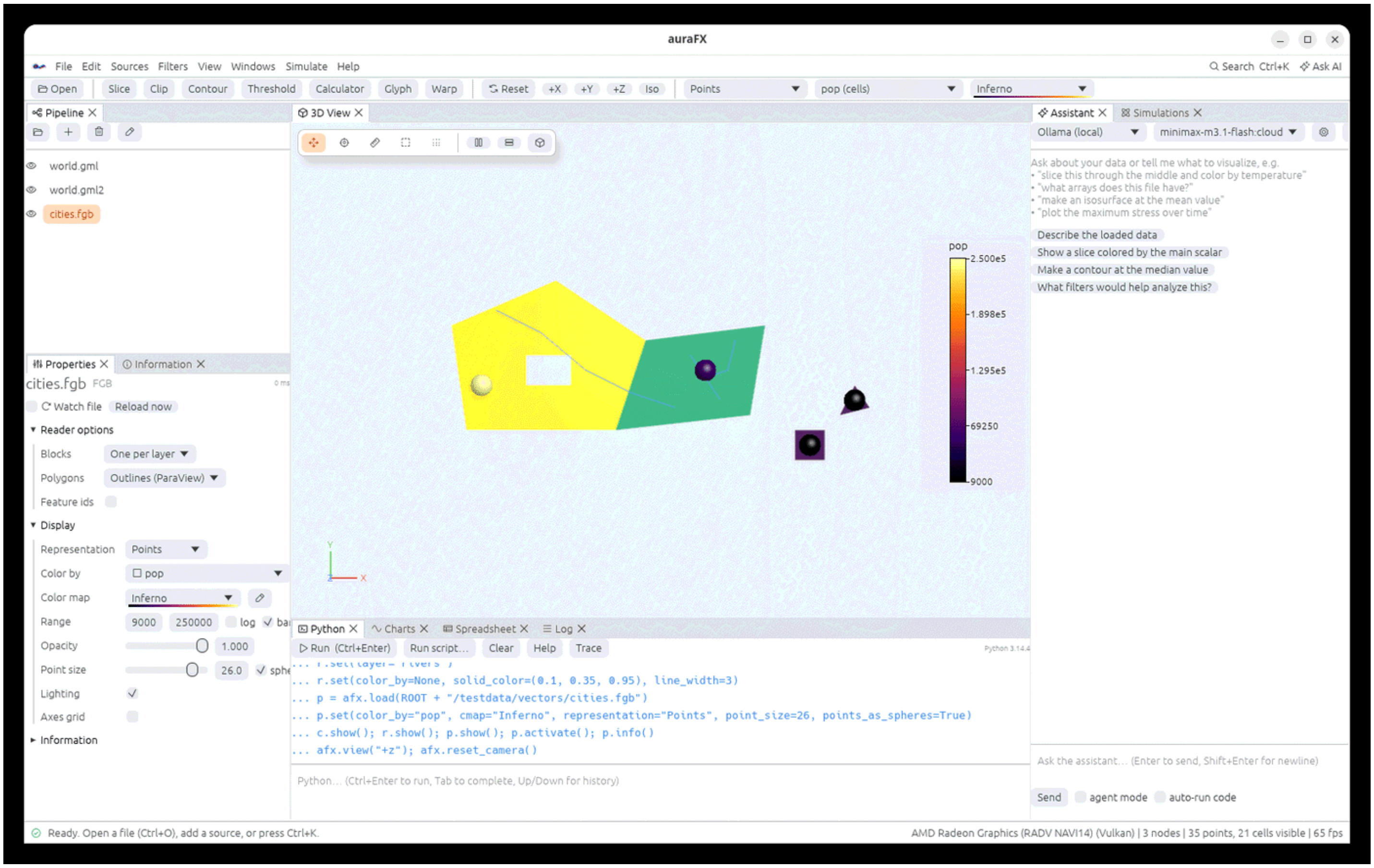
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 35 points, 21 cells visible | 40 fps

```
countries = afx.load("countries.shp")
countries.set(polygons="Filled")
countries.set(color_by="pop", log_scale=True)
roads = afx.load("world.gpkg")
roads.set(layer="roads")                    # one layer of a GeoPackage
sites = afx.load("sites.fgb")               # FlatGeobuf
rivers = afx.load("hydro.gml")              # GML, a block per feature type
places = afx.load("trip.kmz")               # Google Earth KMZ
parcels = afx.load("parcels.tab")           # MapInfo (or parcels.mif)
lots = afx.load("city.gdb")                 # an Esri file geodatabase folder
lots.set(layer="lots", polygons="Filled")
```


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Points
pop (cells)
Inferno
Pipeline
world.gml
world.gml2
cities.fgb
3D View
pop
2.500e5
1.898e5
1.295e5
69250
9000
Y
X
Assistant
Simulations
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

cities.fgb FGB

☐ Watch file Reload now

Reader options

Blocks: One per layer

Polygons: Outlines (ParaView)

Feature ids: ☐

Display

Representation: Points

Color by: pop

Color map: Inferno

Range: 9000 250000 ☐ log ☑ ba

Opacity: 1.000

Point size: 26.0 ☑ sphe

Lighting: ☑

Axes grid: ☐

Information

```
... r.set(color_by=None, solid_color=(0.1, 0.35, 0.95), line_width=3)
... p = afx.load(ROOT + "/testdata/vectors/cities.fgb")
... p.set(color_by="pop", cmap="Inferno", representation="Points", point_size=26, points_as_spheres=True)
... c.show(); r.show(); p.show(); p.activate(); p.info()
... afx.view("+z"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

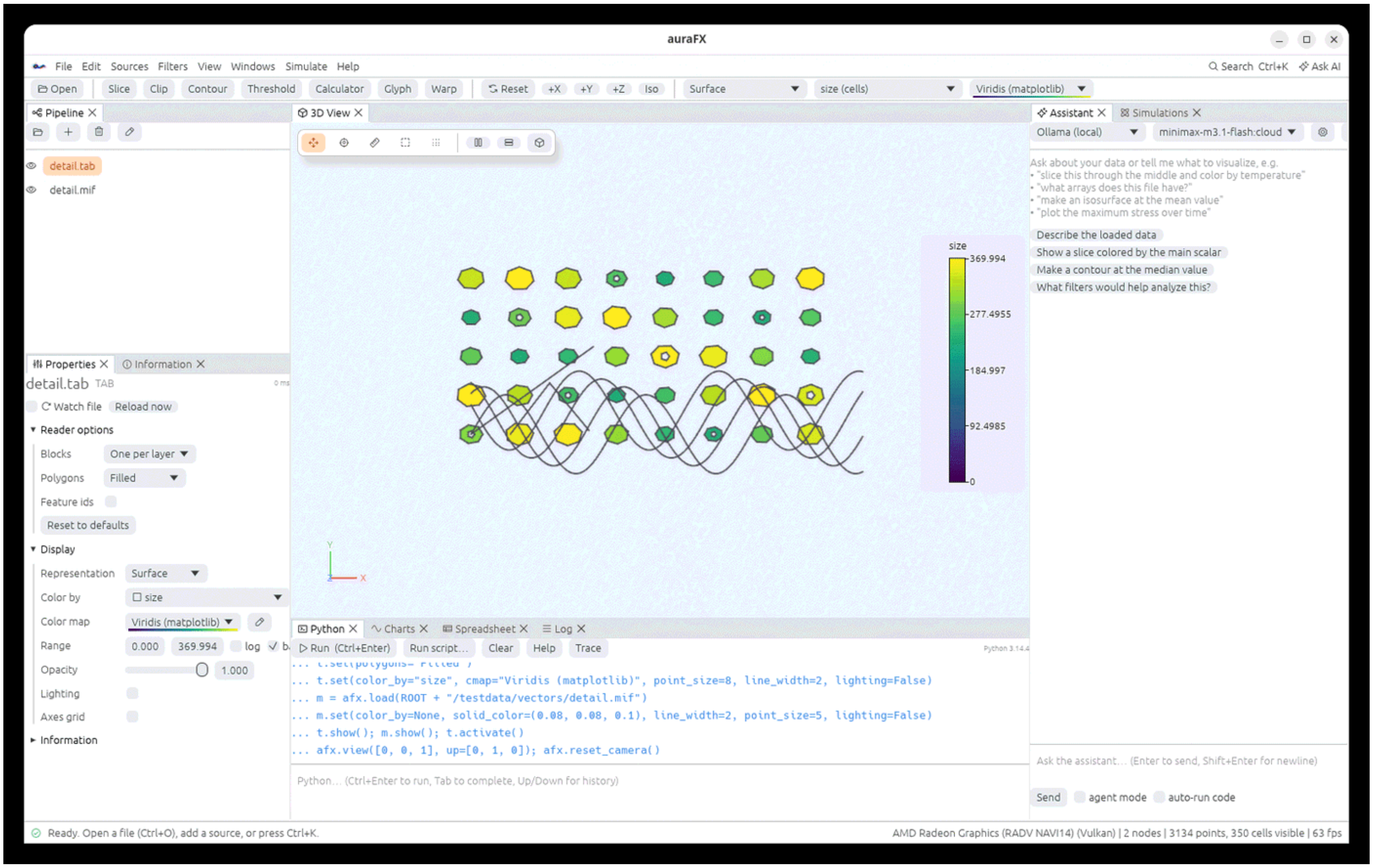
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 35 points, 21 cells visible | 65 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
size (cells)
Viridis (matplotlib)
Pipeline
3D View
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
detail.tab
detail.mif
size
369.994
277.4955
184.997
92.4985
0
Properties
Information
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

detail.tab TAB

Watch file · Reload now

Reader options

Blocks: One per layer
Polygons: Filled
Feature ids: ☐
Reset to defaults

Display

Representation: Surface
Color by: size
Color map: Viridis (matplotlib)
Range: 0.000 369.994 ☐ log ☑
Opacity: 1.000
Lighting: ☐
Axes grid: ☐

Information

Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"

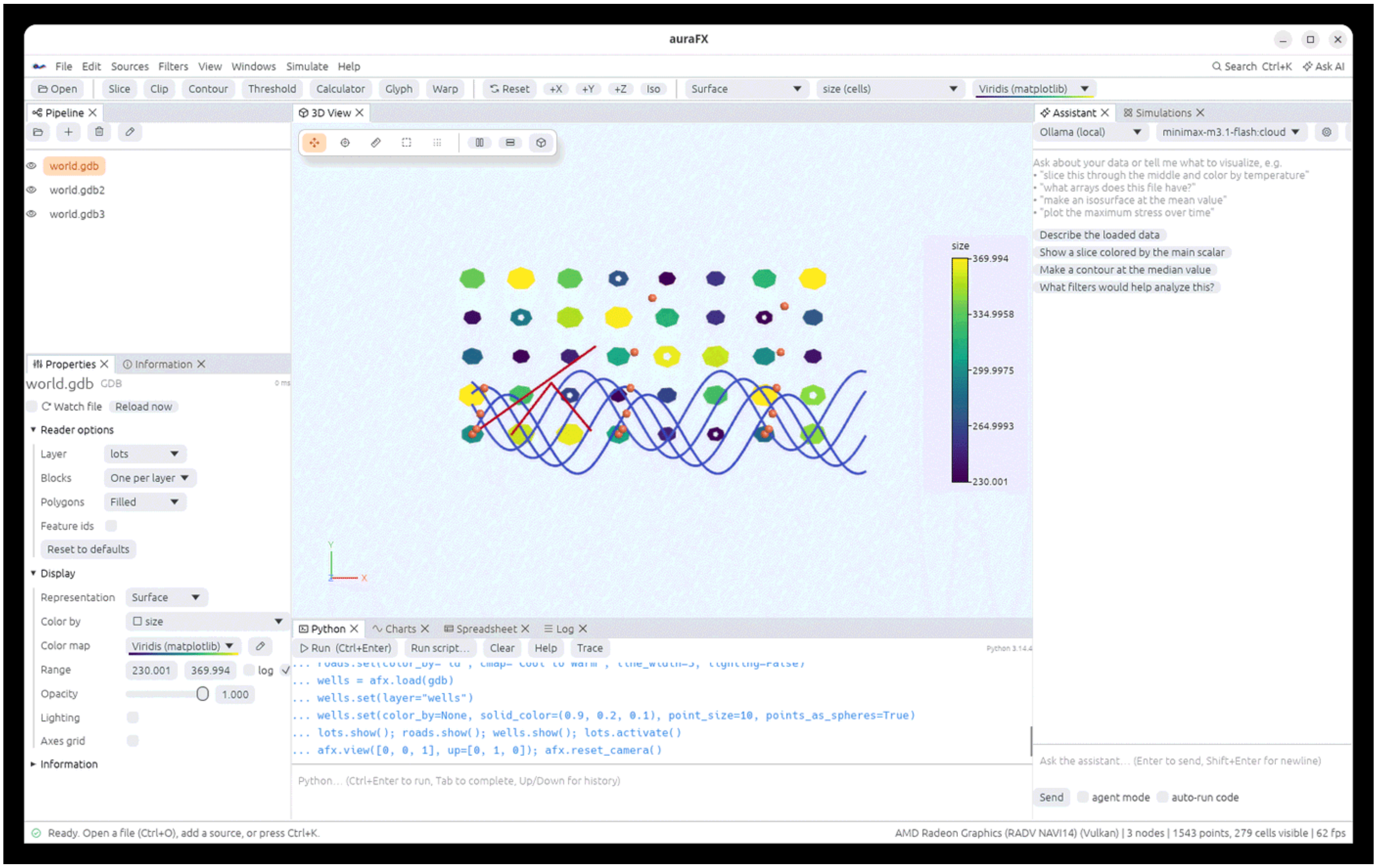
```
... t.set(polygons="Filled")
... t.set(color_by="size", cmap="Viridis (matplotlib)", point_size=8, line_width=2, lighting=False)
... m = afx.load(ROOT + "/testdata/vectors/detail.mif")
... m.set(color_by=None, solid_color=(0.08, 0.08, 0.1), line_width=2, point_size=5, lighting=False)
... t.show(); m.show(); t.activate()
... afx.view([0, 0, 1], up=[0, 1, 0]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 3134 points, 350 cells visible | 63 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
size (cells)
Viridis (matplotlib)
Pipeline
3D View
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
world.gdb
world.gdb2
world.gdb3
size
369.994
334.9958
299.9975
264.9993
230.001
Properties
Information
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

world.gdb GDB

Watch file · Reload now

Reader options

Layer: lots
Blocks: One per layer
Polygons: Filled
Feature ids: ☐
Reset to defaults

Display

Representation: Surface
Color by: size
Color map: Viridis (matplotlib)
Range: 230.001 369.994 ☐ log ☑
Opacity: 1.000
Lighting: ☐
Axes grid: ☐

Information

Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"

```
... roads.set(color_by="id", cmap="Cool to Warm", line_width=3, lighting=False)
... wells = afx.load(gdb)
... wells.set(layer="wells")
... wells.set(color_by=None, solid_color=(0.9, 0.2, 0.1), point_size=10, points_as_spheres=True)
... lots.show(); roads.show(); wells.show(); lots.activate()
... afx.view([0, 0, 1], up=[0, 1, 0]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

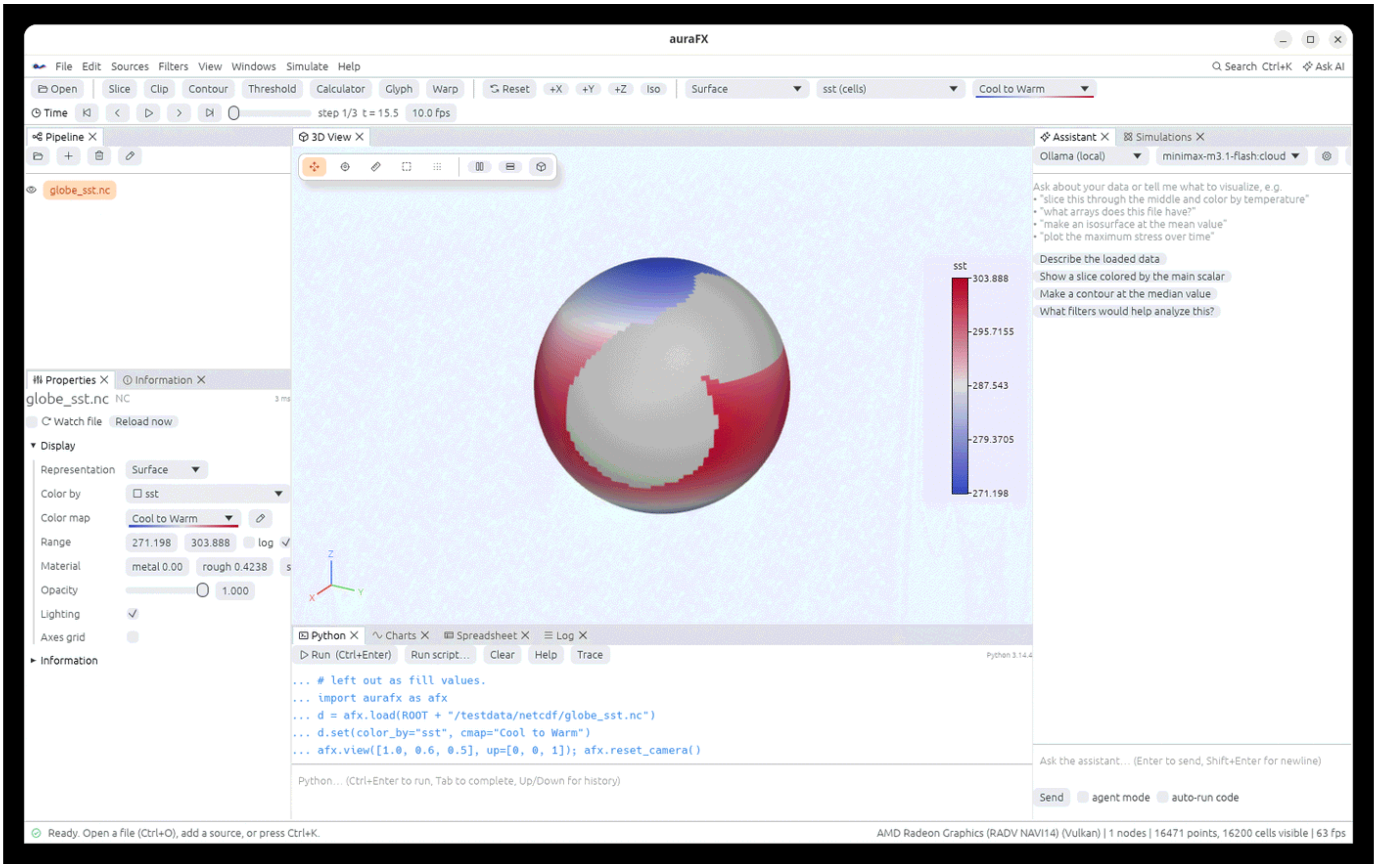
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 1543 points, 279 cells visible | 62 fps

A feature without geometry gives an empty block. Any layer of a file can be picked on its own, and by default all layers are read.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Not read yet: curved geometry (arcs; a file geodatabase's curves are read as straight lines between their vertices) and file geodatabase rasters and multipatches. In MapInfo files, text, arcs, rectangles, ellipses and collections are read without their geometry (their attributes are kept). Shapefile multipatches (3-D surfaces) are skipped.


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
sst (cells)
Cool to Warm
Time
step 1/3 t = 15.5
10.0 fps
Pipeline
globe_sst.nc
3D View
sst
303.888
295.7155
287.543
279.3705
271.198
Z
Y
X
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
Watch file
Reload now
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

globe_sst.nc NC 3 ms

▾ Display

Representation: Surface

Color by: ☐ sst

Color map: Cool to Warm

Range: 271.198 303.888 ☐ log ✓

Material: metal 0.00 rough 0.4238

Opacity: 1.000

Lighting: ✓

Axes grid: ☐

▸ Information

```
... # left out as fill values.
... import aurafx as afx
... d = afx.load(ROOT + "/testdata/netcdf/globe_sst.nc")
... d.set(color_by="sst", cmap="Cool to Warm")
... afx.view([1.0, 0.6, 0.5], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 16471 points, 16200 cells visible | 63 fps

AMR data (adaptive mesh refinement)

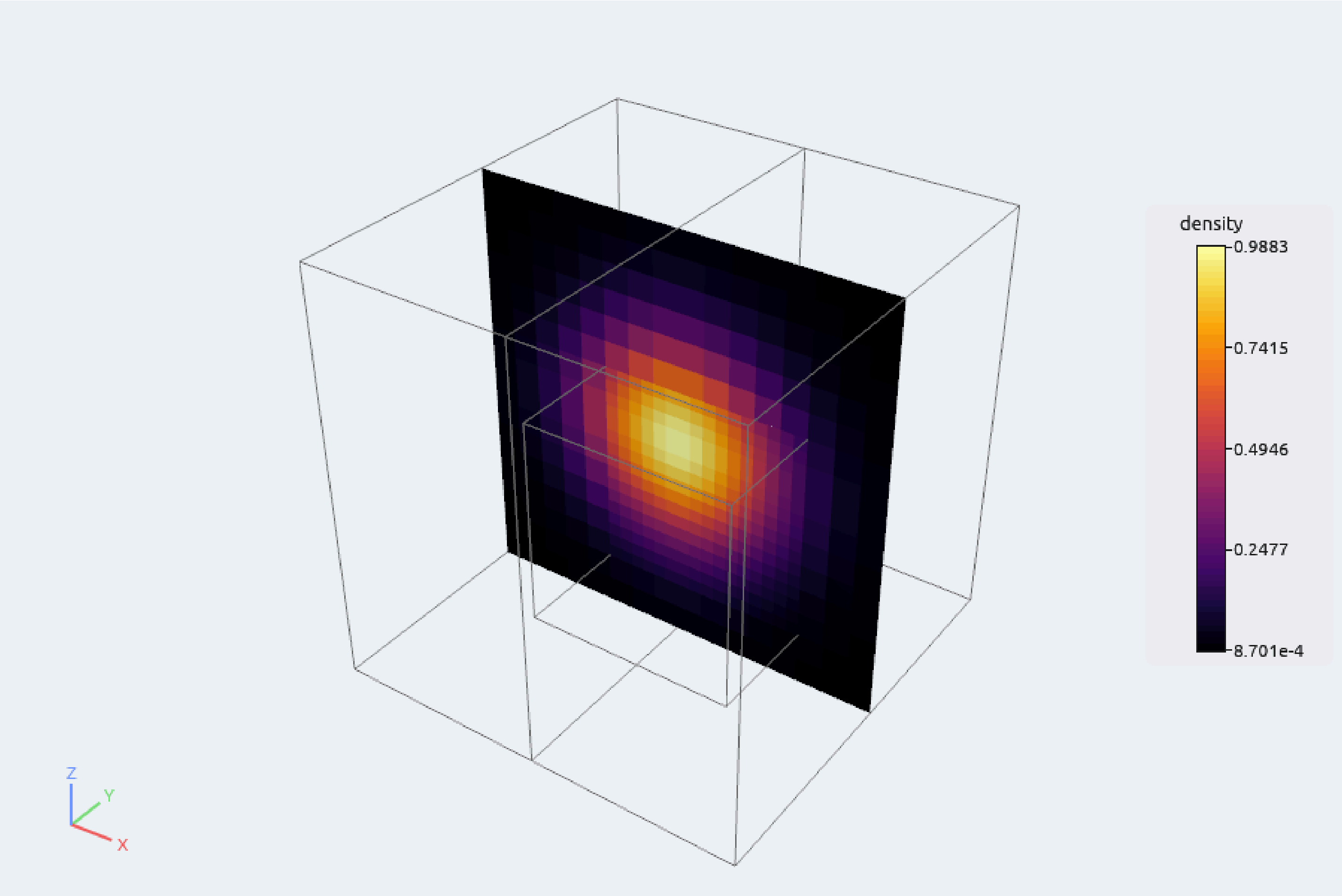
AMR simulations (AMReX, BoxLib, Castro, MAESTRO, Nyx, WarpX, FLASH, Enzo…) store their domain as levels of boxes: a coarse level everywhere and finer boxes where more detail was needed. auraFX reads:

- **AMReX / BoxLib plotfiles**: a folder like `plt00100` with a `Header` file. Open the `Header` file in **File › Open** (choose "All files"), drop the folder on the window, or `afx.load("plt00100")`.
- **VTK AMR files** (`.vthb`, `.vth`).

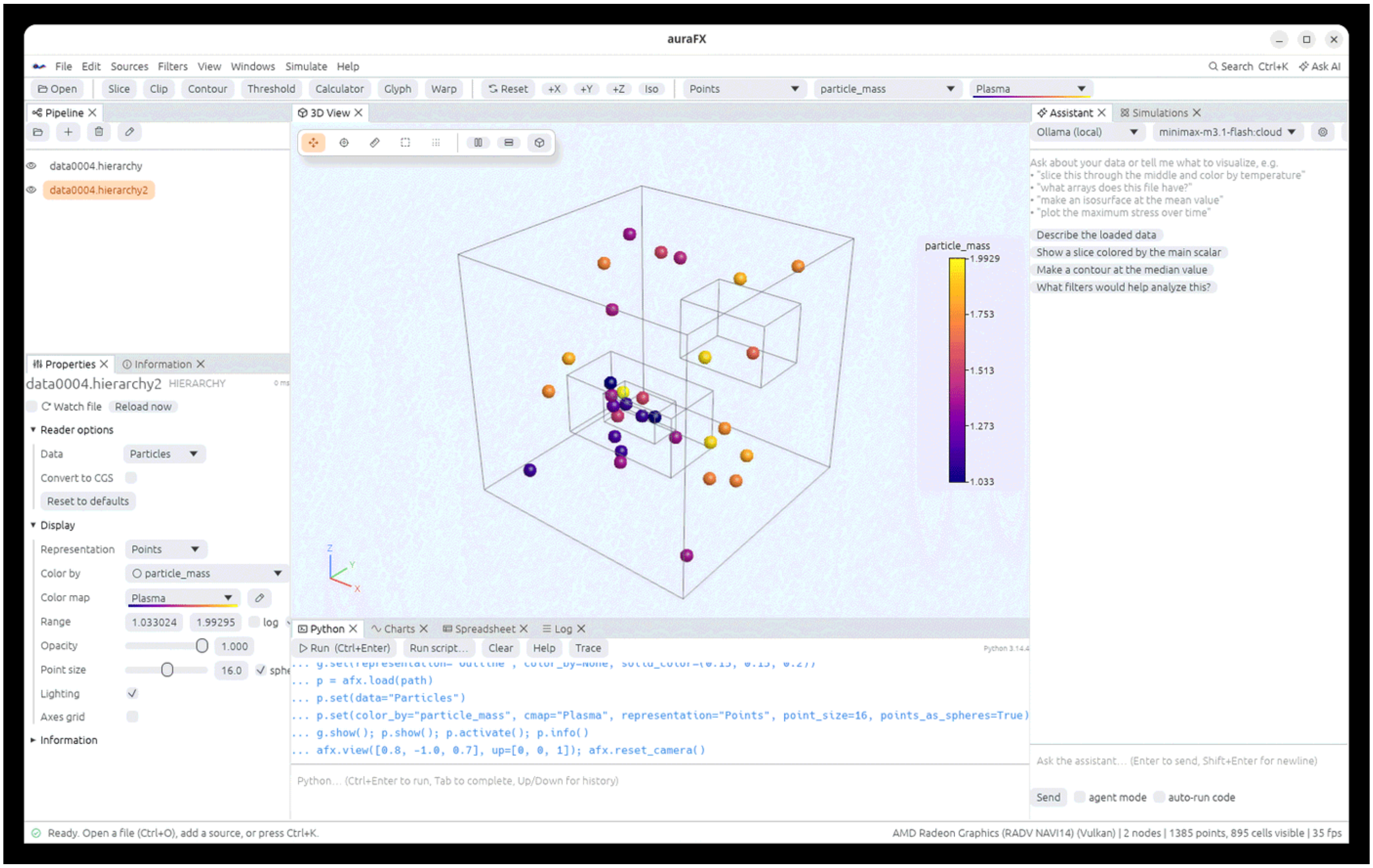
- **FLASH** plot and checkpoint files (`name_hdf5_plt_cnt_0007` , `name_hdf5_chk_0003` ; HDF5, FLASH 2 to 4). They usually have no extension: open them in **File › Open** with “All files”; auraFX recognizes them by their contents. Every block of the PARAMESH tree becomes a box on its refinement level (FLASH’s level 1 is auraFX’s Level 0), with every variable in `unknown names` as cell data in double precision. 1-D, 2-D and 3-D files are read.
- **Enzo** output: open the `.hierarchy` file of a data dump (`DD0010/data0010.hierarchy`). The parameter file (`data0010`) and the grid files (`data0010.cpu0000` …) must be beside it. Each grid is a box on its level; fields keep their stored type (usually single precision). Reader option **Convert to CGS** multiplies each field by its `#DataCGSConversionFactor` from the parameter file. It is off by default.
- **Particles** in FLASH and Enzo files: when a file has them, the reader option **Data** switches between **Grid** (the AMR data) and **Particles**, a point for every particle with its properties as point data (FLASH’s as doubles, Enzo’s in their stored types). Open the file twice to see both, as in the picture below. Chombo files with a `particles` group get the same option (properties as doubles).
- **Chombo** files (`.hdf5` , `.h5` ; 1-D to 3-D), as Chombo-based codes write them (`level_0` , `level_1` … groups with boxes). The component names become cell arrays, or point arrays in node-centered files. Ghost cells stored around each box are dropped, and the boxes are placed from `prob_lo` and the level’s `dx` (or `vec_dx`).
- **4-D Chombo** files (two space axes and two velocity axes, as gyrokinetic codes such as COGENT write them, single level): a 2-D grid over the space axes, whose every cell holds each component as an array with one value per velocity cell (parallel velocity major, magnetic moment minor). `_vpar` and `_mu` hold those cells’ velocity coordinates, and `<component>_sum` the sum over the velocity cells (leaving out velocity cells no box covers, which are NaN in the arrays). Color by a component of the array to see one velocity cell, or by the sum.

What you get:

- One block per level (**Level 0**, **Level 1**, …), each holding that level’s boxes as image data with the variables as cell data. The **Information** panel lists them.
- Coarse cells that a finer box covers are marked as hidden in `vtkGhostType` . Surfaces, slices, clips, contours, thresholds, Integrate Variables, Resample To Image and probing skip them, so every place is shown and counted once, at its finest level.
- AMReX vector variables written as `amrexvec_<name>_x` , `_y` , `_z` become one vector `<name>` . In 2-D files the missing z is 0.
- The time of the plotfile or dump is in the field data (`TimeValue`). Single- and double-precision files are read.
- **Outline** draws the outline of every box, which shows where the refinement is.
- Contours of cell data are stitched across levels (see below).



```
amr = afx.load("plt00100")                 # the plotfile folder
amr.set(representation="Outline")          # where the refinement is
s = amr.slice(normal=(0, 0, 1))            # finest data on the cut
s.set(color_by="density")
lev = amr.overlapping_amr_level_ids()      # LevelIdScalars: each cell's level
flash = afx.load("sedov_hdf5_plt_cnt_0010")  # FLASH
enzo = afx.load("DD0010/data0010.hierarchy")  # Enzo
enzo.set(cgs=True)                         # fields in CGS units
chombo = afx.load("plot.0010.2d.hdf5")      # Chombo
stars = afx.load("DD0010/data0010.hierarchy")
stars.set(data="Particles")                # the particles instead of the grid
```


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Points
particle_mass
Plasma
Pipeline
data0004.hierarchy
data0004.hierarchy2
3D View
particle_mass
1.9929
1.753
1.513
1.273
1.033
Z
Y
X
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

data0004.hierarchy2 HIERARCHY

☐ Watch file Reload now

Reader options
- Data: Particles
- Convert to CGS: ☐
- Reset to defaults

Display
- Representation: Points
- Color by: particle_mass
- Color map: Plasma
- Range: 1.033024 1.99295 ☐ log
- Opacity: 1.000
- Point size: 16.0 ✓ sphe
- Lighting: ✓
- Axes grid: ☐

Information

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

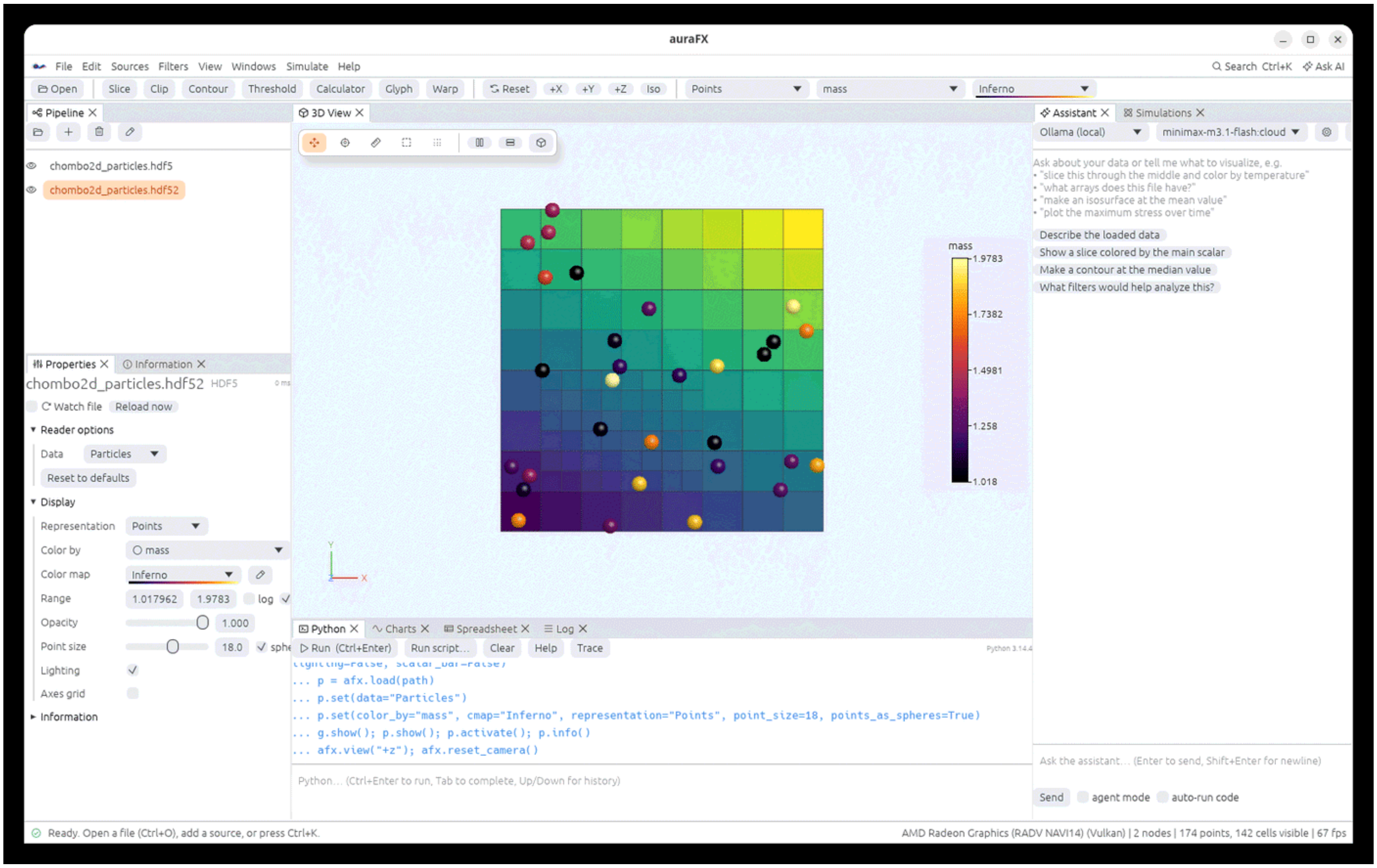
```
... p = afx.load(path)
... p.set(data="Particles")
... p.set(color_by="particle_mass", cmap="Plasma", representation="Points", point_size=16, points_as_spheres=True)
... g.show(); p.show(); p.activate(); p.info()
... afx.view([0.8, -1.0, 0.7], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 1385 points, 895 cells visible | 35 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Points
mass
Inferno
Pipeline
chombo2d_particles.hdf5
chombo2d_particles.hdf52
3D View
mass
1.9783
1.7382
1.4981
1.258
1.018
Y
X
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
agent mode
auto-run code

chombo2d_particles.hdf52 HDF5

☐ Watch file Reload now

Reader options
- Data: Particles
- Reset to defaults

Display
- Representation: Points
- Color by: mass
- Color map: Inferno
- Range: 1.017962 1.9783 ☐ log ✓
- Opacity: 1.000
- Point size: 18.0 ✓ sphe
- Lighting: ✓
- Axes grid: ☐

Information

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

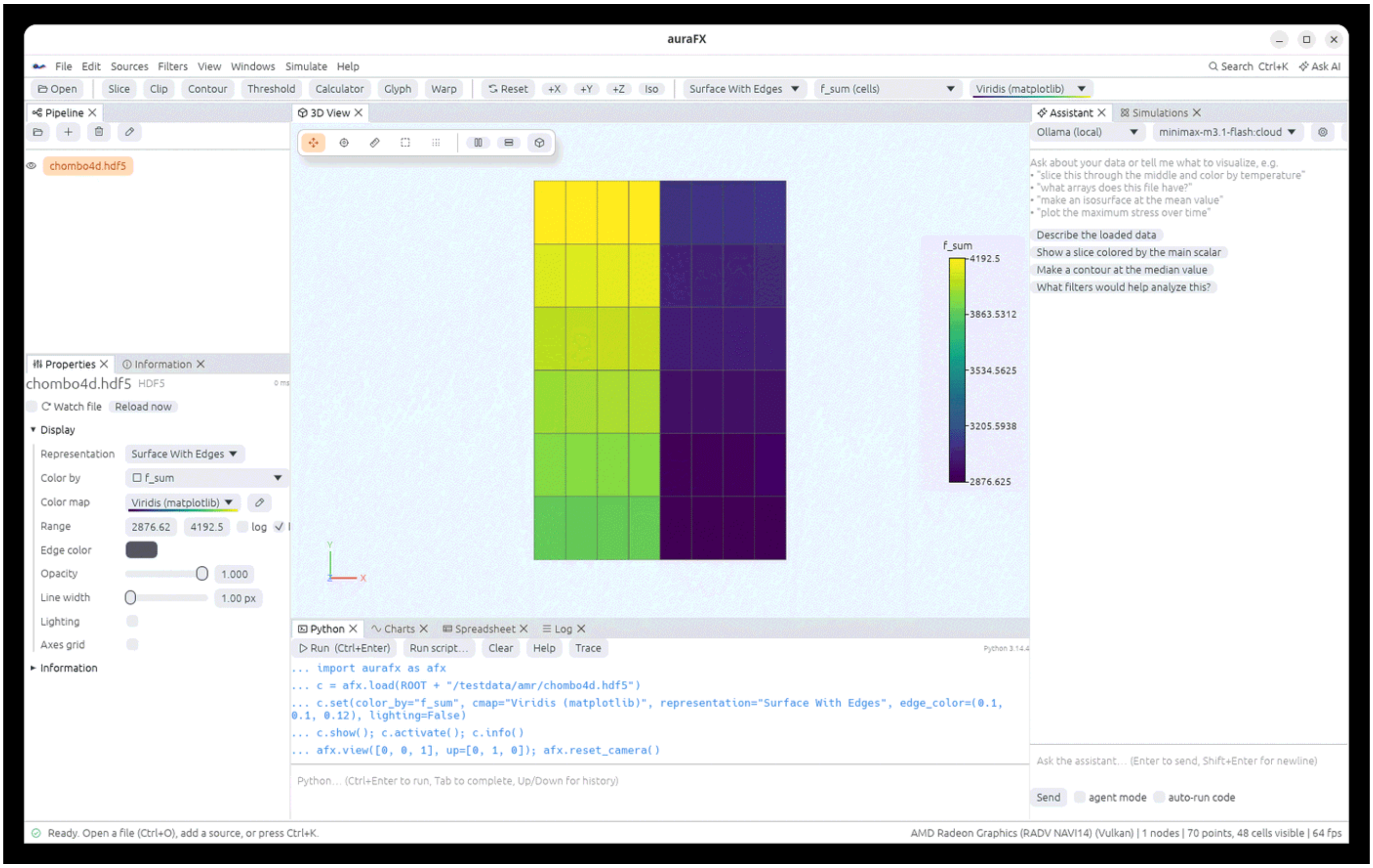
```
... p = afx.load(path)
... p.set(data="Particles")
... p.set(color_by="mass", cmap="Inferno", representation="Points", point_size=18, points_as_spheres=True)
... g.show(); p.show(); p.activate(); p.info()
... afx.view("+z"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 174 points, 142 cells visible | 67 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Pipeline
3D View
Assistant
Simulations
chombo4d.hdf5
Properties
Information
f_sum
4192.5
3863.5312
3534.5625
3205.5938
2876.625
Y
X
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Surface With Edges | f_sum (cells) | Viridis (matplotlib)

Ollama (local) | minimax-m3.1-flash:cloud

chombo4d.hdf5 HDF5

☐ Watch file Reload now

Display

- Representation: Surface With Edges
- Color by: f_sum
- Color map: Viridis (matplotlib)
- Range: 2876.62 4192.5 ☐ log ☑
- Edge color
- Opacity: 1.000
- Line width: 1.00 px
- Lighting ☐
- Axes grid ☐

Information

```
... import aurafx as afx
... c = afx.load(ROOT + "/testdata/amr/chombo4d.hdf5")
... c.set(color_by="f_sum", cmap="Viridis (matplotlib)", representation="Surface With Edges", edge_color=(0.1,
0.1, 0.12), lighting=False)
... c.show(); c.activate(); c.info()
... afx.view([0, 0, 1], up=[0, 1, 0]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

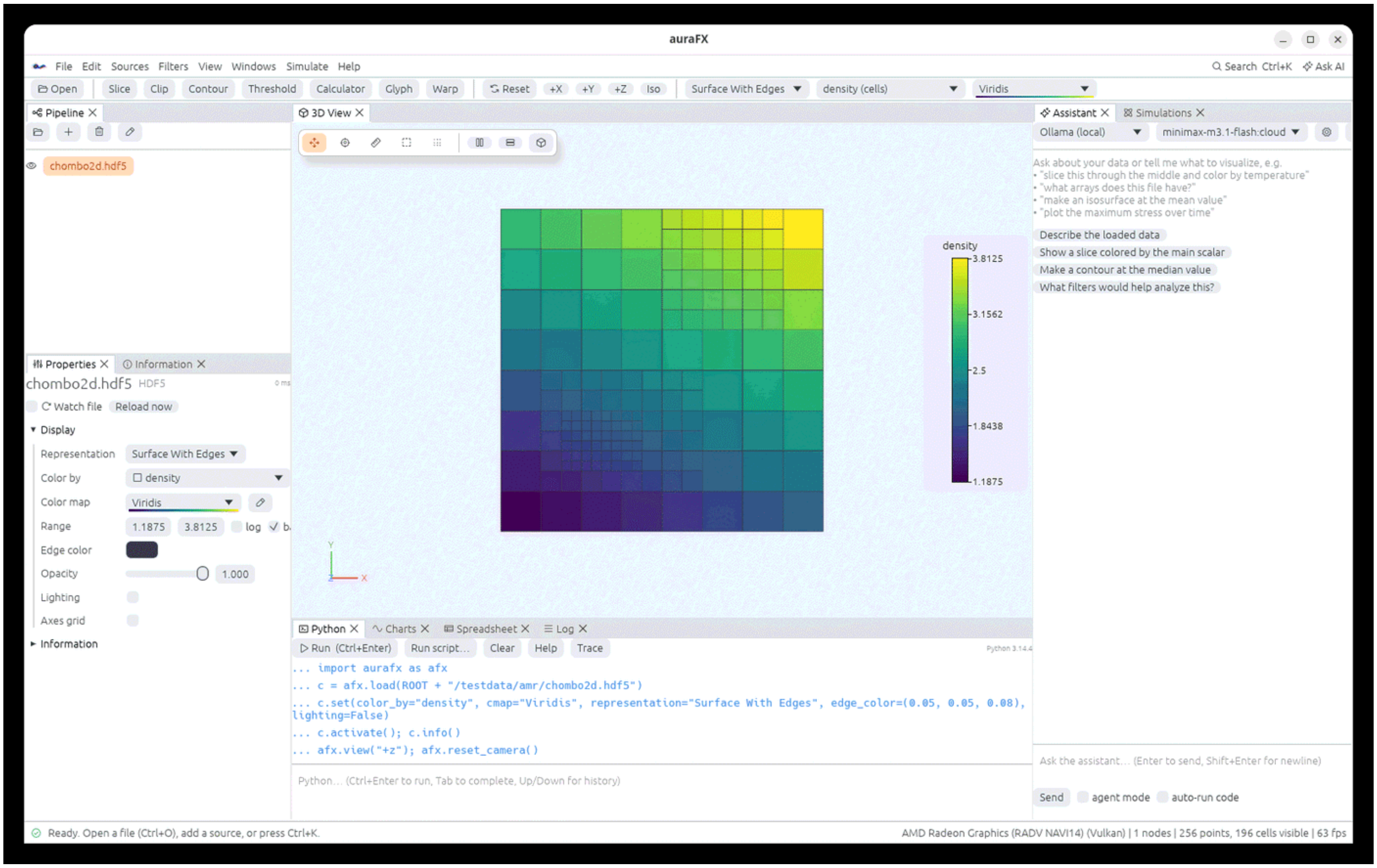
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 70 points, 48 cells visible | 64 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Pipeline
3D View
Assistant
Simulations
chombo2d.hdf5
Properties
Information
density
3.8125
3.1562
2.5
1.8438
1.1875
Y
X
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Surface With Edges | density (cells) | Viridis

Ollama (local) | minimax-m3.1-flash:cloud

chombo2d.hdf5 HDF5

☐ Watch file Reload now

Display

- Representation: Surface With Edges
- Color by: density
- Color map: Viridis
- Range: 1.1875 3.8125 ☐ log ☑ b
- Edge color
- Opacity: 1.000
- Lighting ☐
- Axes grid ☐

Information

```
... import aurafx as afx
... c = afx.load(ROOT + "/testdata/amr/chombo2d.hdf5")
... c.set(color_by="density", cmap="Viridis", representation="Surface With Edges", edge_color=(0.05, 0.05, 0.08),
lighting=False)
... c.activate(); c.info()
... afx.view("+z"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

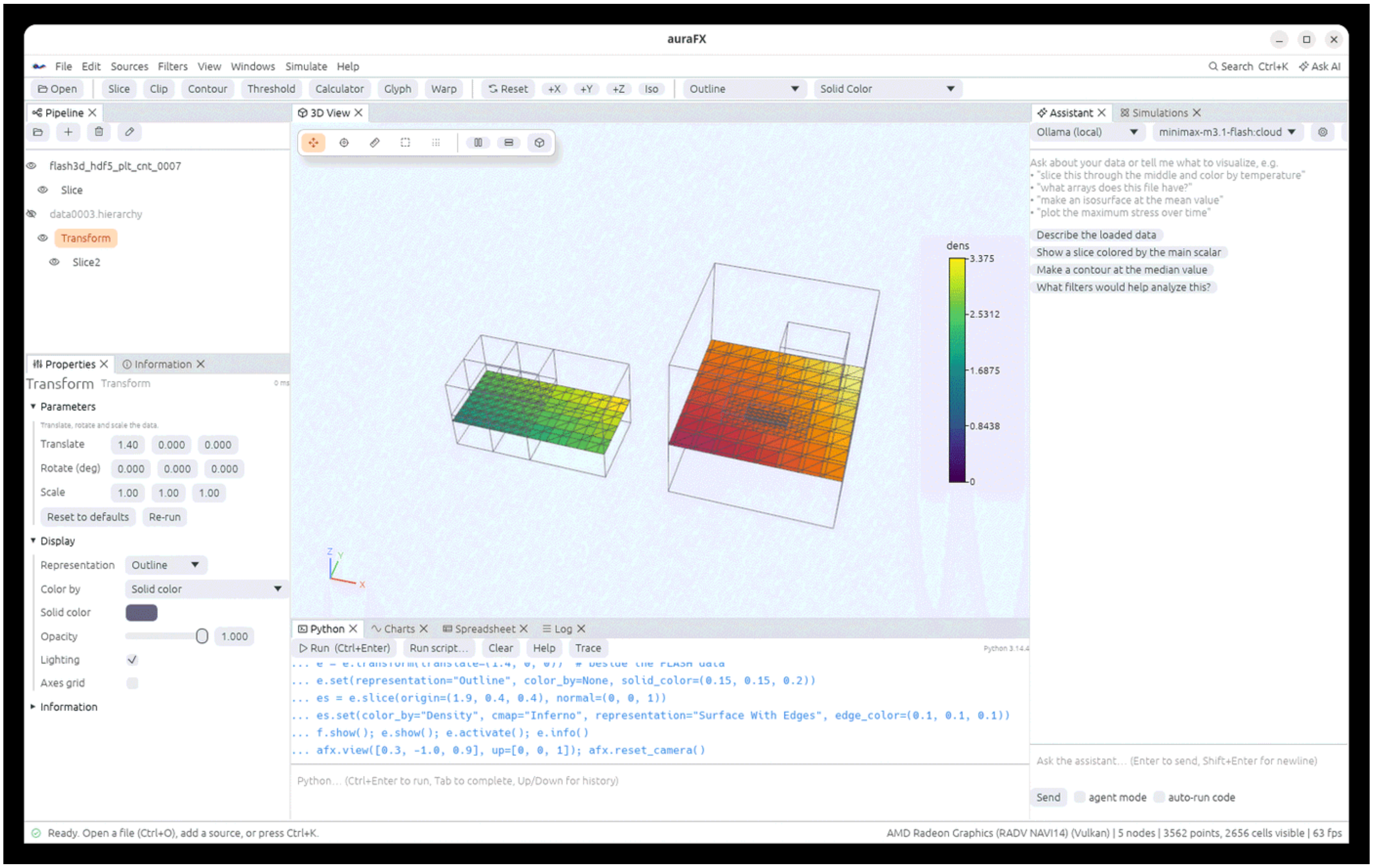
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 256 points, 196 cells visible | 63 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Outline
Solid Color
Pipeline
flash3d_hdf5_plt_cnt_0007
Slice
data0003.hierarchy
Transform
Slice2
3D View
dens
3.375
2.5312
1.6875
0.8438
0
Z
Y
X
Properties
Information
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Transform Transform

Parameters

Translate, rotate and scale the data.

Translate 1.40 0.000 0.000

Rotate (deg) 0.000 0.000 0.000

Scale 1.00 1.00 1.00

Reset to defaults Re-run

Display

Representation: Outline

Color by: Solid color

Opacity: 1.000

Lighting ☑

Axes grid ☐

Information

```
... e = e.transform(translate=(1.4, 0, 0))  # beside the FLASH data
... e.set(representation="Outline", color_by=None, solid_color=(0.15, 0.15, 0.2))
... es = e.slice(origin=(1.9, 0.4, 0.4), normal=(0, 0, 1))
... es.set(color_by="Density", cmap="Inferno", representation="Surface With Edges", edge_color=(0.1, 0.1, 0.1))
... f.show(); e.show(); e.activate(); e.info()
... afx.view([0.3, -1.0, 0.9], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.

- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

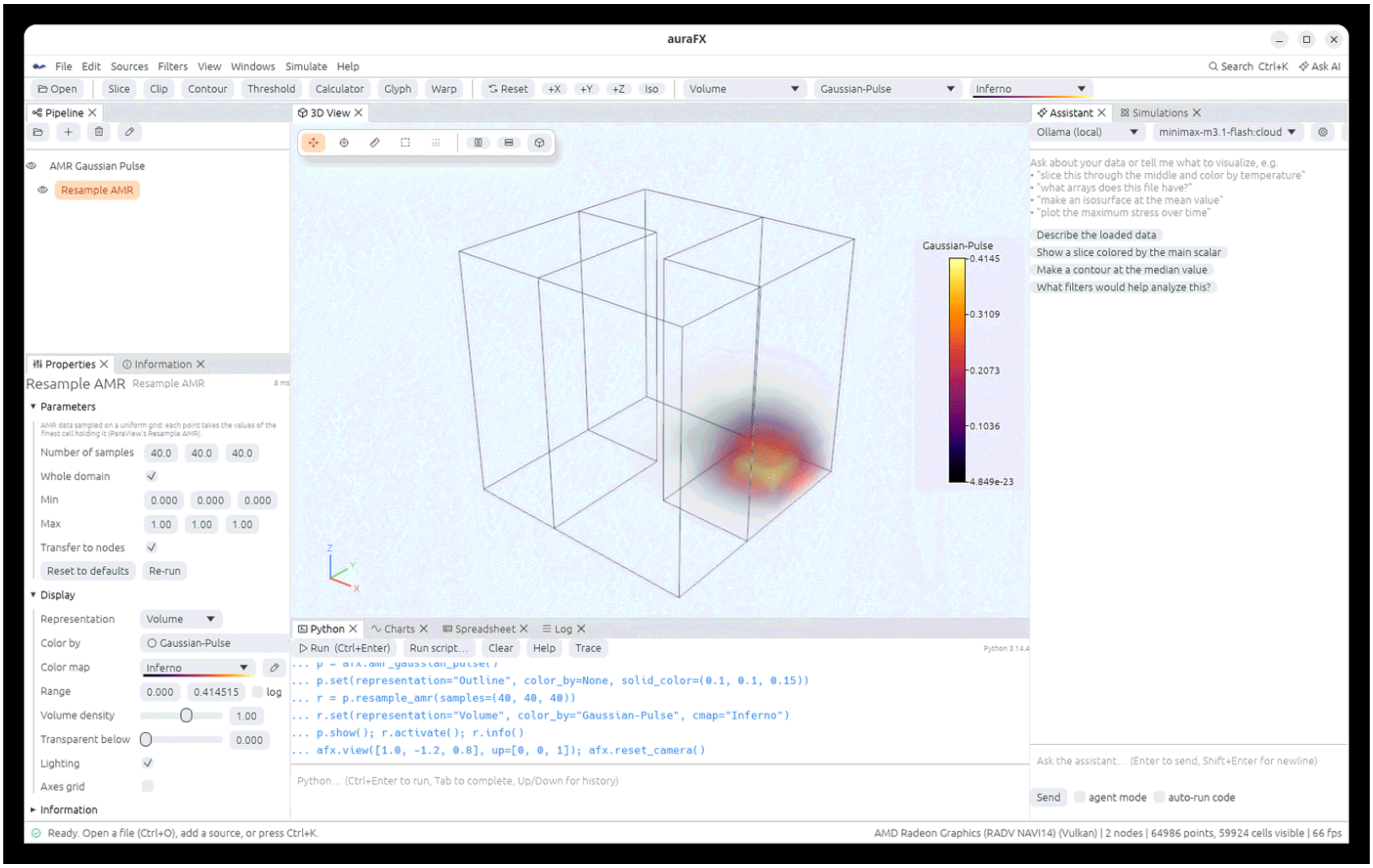
☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 5 nodes | 3562 points, 2656 cells visible | 63 fps

Sources › AMR Gaussian Pulse makes a small AMR test dataset: a coarse box and two finer boxes holding a Gaussian pulse, in 2-D or 3-D. **Filters › Overlapping AMR Level Ids** adds the cell array `LevelIdScalars` with each cell's level.

Filters › Resample AMR samples AMR data on one uniform grid, so tools that want a single image can use it (volume rendering, image filters, export). **Number of samples** sets the grid points per axis; with **Whole domain** off, **Min** and **Max** pick a box (clipped to the data, dropping the samples that fall outside). Each point takes the values of the finest cell holding it. With **Transfer to nodes** off, the values are taken at the cell centers instead and become cell data. Points no box holds are marked hidden.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Volume
Gaussian-Pulse
Inferno
Pipeline
AMR Gaussian Pulse
Resample AMR
3D View
Gaussian-Pulse
0.4145
0.3109
0.2073
0.1036
4.849e-23
Z
Y
X
Assistant
Simulations
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Resample AMR Resample AMR 8 ms

▾ Parameters

AMR data sampled on a uniform grid: each point takes the values of the finest cell holding it (ParaView's Resample AMR).

Number of samples 40.0 40.0 40.0
Whole domain ☑
Min 0.000 0.000 0.000
Max 1.00 1.00 1.00
Transfer to nodes ☑
Reset to defaults Re-run

▾ Display

Representation Volume
Color by Gaussian-Pulse
Color map Inferno
Range 0.000 0.414515 ☐ log
Volume density 1.00
Transparent below 0.000
Lighting ☑
Axes grid ☐

▸ Information

```
... p = afx.amr_gaussian_pulse()
... p.set(representation="Outline", color_by=None, solid_color=(0.1, 0.1, 0.15))
... r = p.resample_amr(samples=(40, 40, 40))
... r.set(representation="Volume", color_by="Gaussian-Pulse", cmap="Inferno")
... p.show(); r.activate(); r.info()
... afx.view([1.0, -1.2, 0.8], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local)
minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 64986 points, 59924 cells visible | 66 fps

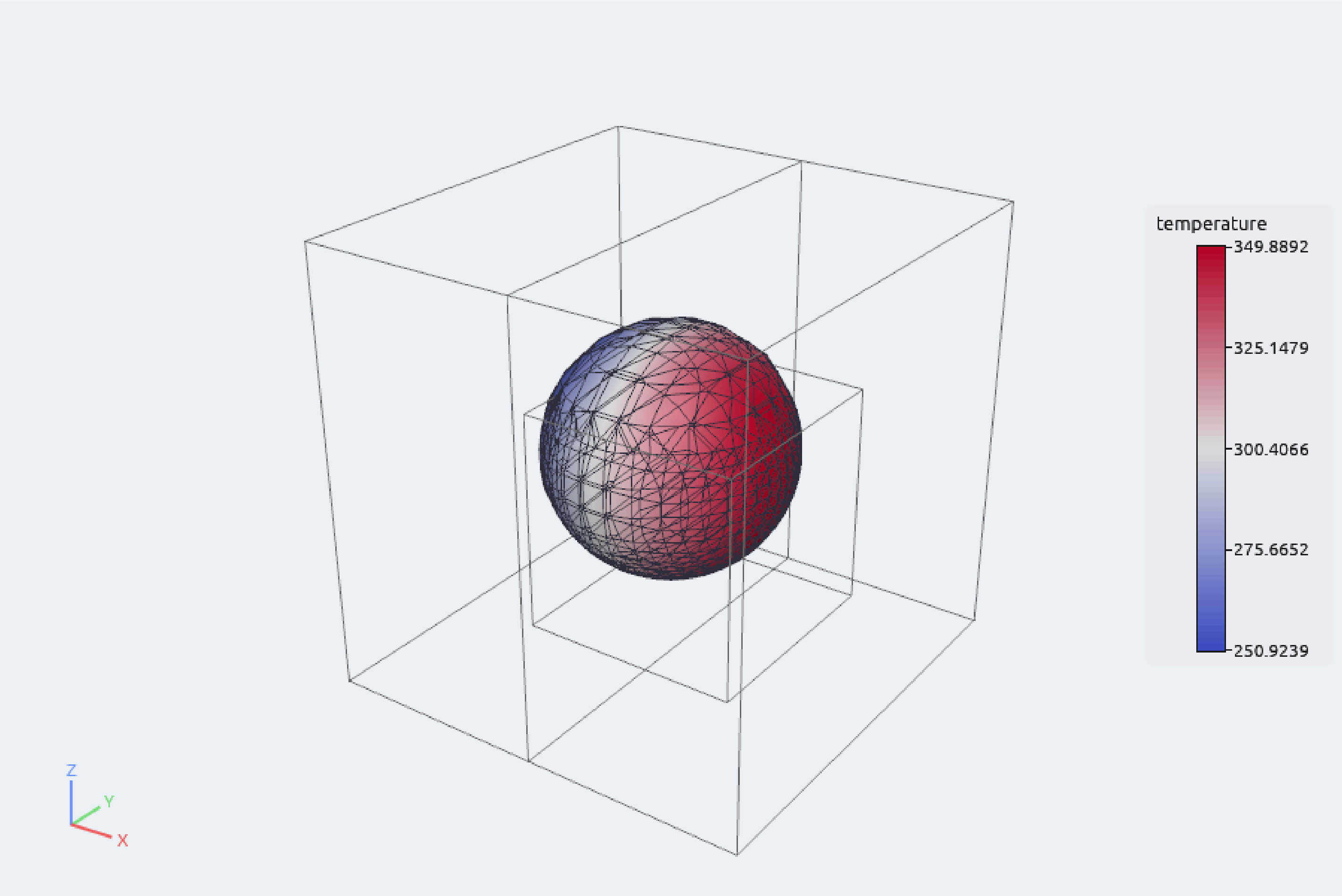
```
grid = amr.resample_amr(samples=(64, 64, 64))
part = amr.resample_amr(samples=(32, 32, 16), whole_domain=False, min=(0, 0, 0),
max=(1, 1, 0.5))
```

Resample AMR always uses every level of the data. 2-D FLASH files are read like 3-D ones. Ghost cells stored in Chombo files are dropped on every level. FLASH particles are given once each, with their own properties.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Contours of AMR data

Values of AMR data sit at cell centers, and cells change size where levels meet. Averaging them to points box by box leaves gaps and overlaps along every level boundary. **Filters › AMR Contour** joins the cell centers of all levels into one mesh, the *dual grid*. Where a finer box meets coarser cells, its border cells reach out to the coarse cell centers, so the isosurface continues across the boundary without a crack. The other cell arrays are interpolated onto the surface. **Contour** on a cell array of AMR data does the same automatically. In 2-D data the result is closed isolines.



```
c = amr.amr_contour(array="density", values=[0.3])
c = amr.contour(array="density", values=[0.3])      # the same, for AMR cell data
dual = amr.amr_dual_grid()                           # cell values on one stitched mesh
cut = dual.clip(type="Scalar", array="density", value=0.3)    # a crack-free "dual clip"
```

Filters › AMR Dual Grid gives that mesh itself: one point per cell center, carrying all the cell arrays as point data. Clip, Slice, Stream Tracer and other filters that interpolate point data work on it across levels without seams.

On a test plotfile whose density isosurface is a sphere crossing from the fine box into the coarse ones, the contour is closed (every edge shared by exactly two triangles, the topology of a sphere), its area is within 0.2 % of the sphere's, and its center is within 0.001 of the sphere's center. The box-by-box contour had 346 open edges there. More in the verification notes (`docs/reference/verification.md` in the source tree).

Not read yet: plotfiles without FAB headers (the "NoFabHeader" variants) and AMReX particles. Not available yet: material-interface filters for CTH data (AMR Fragments, Material Interface).

Hyper tree grids

A *hyper tree grid* (HTG) starts from a grid of root cells. Each root cell is the top of a tree: a refined cell splits into 2×2×2 children (3×3×3 with branch factor 3; 2×2 or 3×3 in 2-D), and so on down to the leaves. Every cell of every tree, coarse or leaf, can hold values. A *mask* can hide cells. It is VTK's tree-based format for adaptive meshes. auraFX reads VTK's `.htg` files (XML version 2).

What you get:

- The grid is drawn as its visible leaves. Every filter that works on cells (Slice, Clip, Threshold, Contour after Cell Data to Point Data, Cell Centers, Integrate Variables…) sees those leaves as voxels (pixels in 2-D) with their values.
- **Information** shows the dimension, branch factor, number of visible leaves and depth. The cell count is the number of all cells (coarse ones included).
- A grid with **material interfaces** is drawn cut to its material (see below). Other filters still see whole leaves.
- Saving as `.htg` keeps the grid: its trees, the mask and every cell's values (the version 2 files). Saving as any other format (`.vtu` and the rest) writes the visible leaves as an unstructured grid.

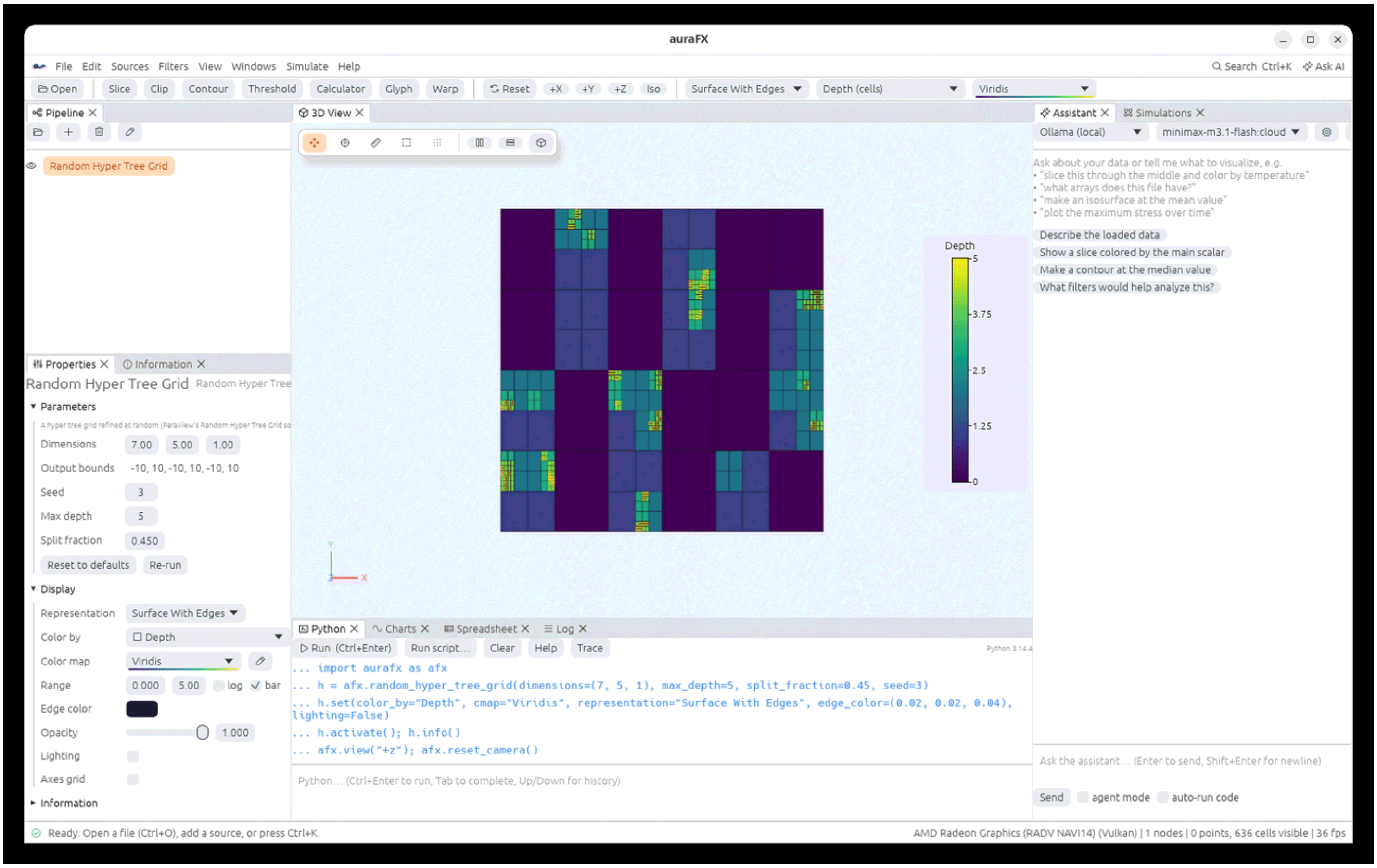
Sources:

- **Hyper Tree Grid** builds a grid from a text descriptor. Levels are separated by `|`. Each level lists `R` (refined) or `.` (leaf) for every cell of that level, all trees in order; spaces are ignored. **Dimensions** (points per axis; 1 makes an axis flat), **Branch factor**, **Max depth**, **Grid scale**, and **Use mask** with a **Mask** string of the same layout (`0` hides a leaf, `1` keeps it). The mask is ignored on refined cells.
- **Random Hyper Tree Grid** refines cells at random. **Split fraction** is each cell's chance of being refined, down to **Max depth**. The same **Seed** gives the same grid. **Masked fraction** hides about that share of the volume, whole subtrees at a time (the same settings hide the same cells). Its masking can hide a cell while leaving some cells below it unmasked; those stay hidden.

Filters (Filters › Data):

Filter	What it does
Hyper Tree Grid To Unstructured Grid	the visible leaves as an unstructured grid, with `OriginalIds` (each leaf's cell index)
Hyper Tree Grid Depth Limiter	cuts every tree at a depth: cells at that depth become leaves (Depth 0 keeps only the root cells)
Hyper Tree Grid Generate Fields	`CellSize` (volume, area or length of every cell), `ValidCell` (1 on visible leaves), `CellCenter`, and `TotalVisibleVolume` in the field

Filter	What it does
	data
Hyper Tree Grid To Dual Grid	a point at the center of every visible leaf, with the leaf's values as point data, and a cell (voxel, pixel or line) joining the leaves around every corner. Use it for smooth contours and for filters that need point data. Points on the grid's boundary sit on the boundary; next to masked leaves they move onto the face shared with the masked cell
Hyper Tree Grid Geometry	the outside of the visible leaves as polygons with their values: in 3-D the faces on the grid's boundary and the walls next to masked cells (a face of a masked cell next to a coarser visible leaf carries that leaf's values); in 2-D a quad per visible leaf, in 1-D a line
	With material interfaces, mixed cells are cut to their material (see below). **Fill material** (on by default): off draws only the interface lines in 2-D and leaves out pure cells in 1-D and 2-D
Hyper Tree Grid Feature Edges	the edges of the visible region as lines with their leaves' values: the outline in 2-D, the outer corners and folds in 3-D
Hyper Tree Grid Evaluate Coarse	fills the values of refined cells from their children: Min, Max, Sum, Average, Unmasked Average, Elder Child (the first child) or Splatting Average


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
Depth (cells)
Viridis
Pipeline
Random Hyper Tree Grid
3D View
Depth
5
3.75
2.5
1.25
0
Y
X
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Properties
Information
Send
agent mode
auto-run code

Ollama (local)

minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

Random Hyper Tree Grid Random Hyper Tree

Parameters

A hyper tree grid refined at random (ParaView's Random Hyper Tree Grid so

Dimensions	7.00 5.00 1.00
Output bounds	-10, 10, -10, 10, -10, 10
Seed	3
Max depth	5
Split fraction	0.450

Reset to defaults Re-run

Display

Representation	Surface With Edges
Color by	Depth
Color map	Viridis
Range	0.000 5.00 ☐ log ☑ bar
Edge color	■
Opacity	1.000
Lighting	☐
Axes grid	☐

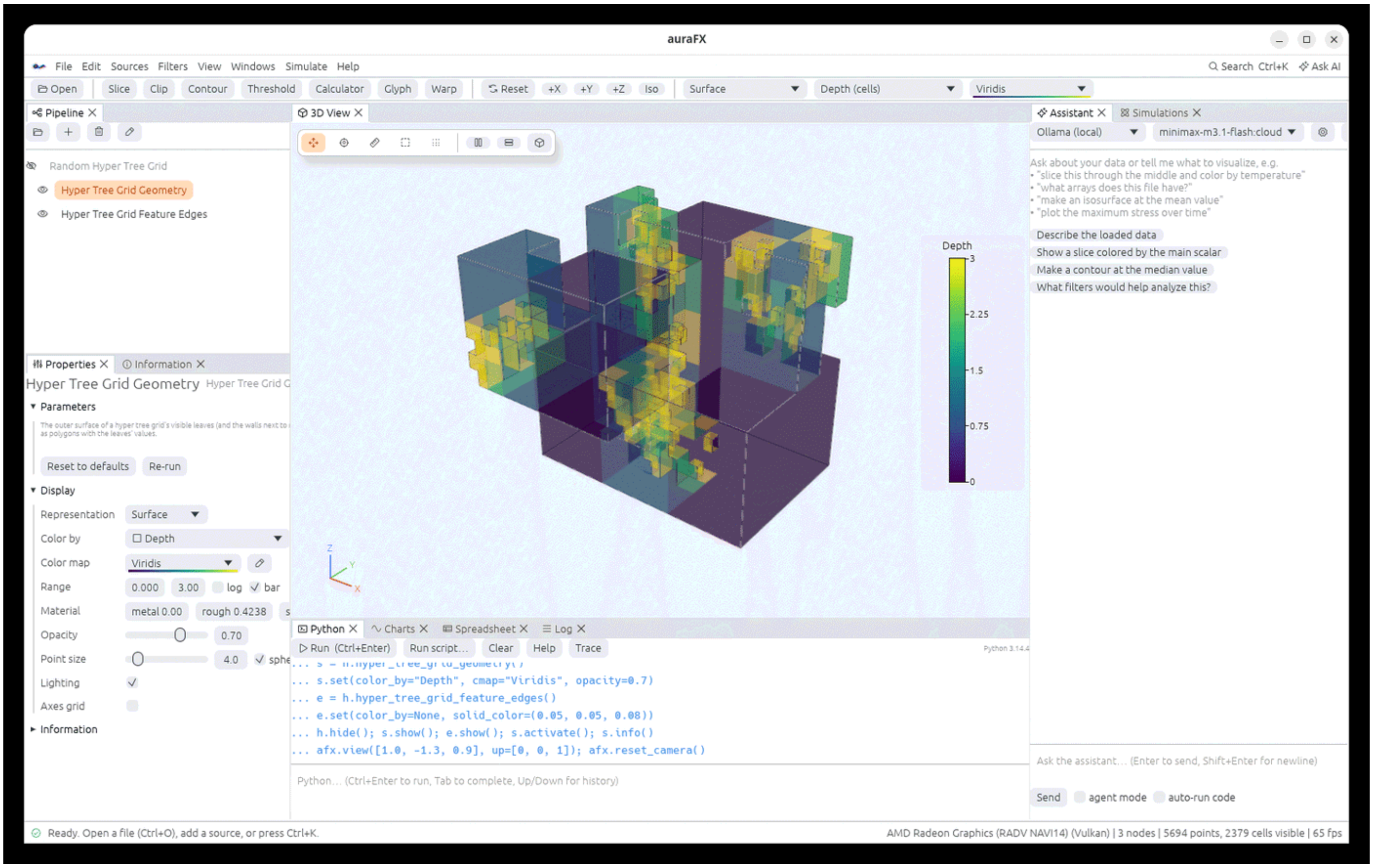
Information

```
... import aurafx as afx
... h = afx.random_hyper_tree_grid(dimensions=(7, 5, 1), max_depth=5, split_fraction=0.45, seed=3)
... h.set(color_by="Depth", cmap="Viridis", representation="Surface With Edges", edge_color=(0.02, 0.02, 0.04),
lighting=False)
... h.activate(); h.info()
... afx.view("+z"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 0 points, 636 cells visible | 36 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Depth (cells)
Viridis
Pipeline
3D View
Assistant
Simulations
Random Hyper Tree Grid
Hyper Tree Grid Geometry
Hyper Tree Grid Feature Edges
Depth
3
2.25
1.5
0.75
0
Z
Y
X
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Properties | Information

Hyper Tree Grid Geometry — Hyper Tree Grid G

Parameters

The outer surface of a hyper tree grid's visible leaves (and the walls next to as polygons with the leaves' values.

Reset to defaults | Re-run

Display

Representation: Surface
Color by: Depth
Color map: Viridis
Range: 0.000 3.00 ☐ log ✓ bar
Material: metal 0.00 rough 0.4238
Opacity: 0.70
Point size: 4.0 ✓ sphe
Lighting: ✓
Axes grid: ☐

Information

```
... s = h.hyper_tree_grid_to_geometry()
... s.set(color_by="Depth", cmap="Viridis", opacity=0.7)
... e = h.hyper_tree_grid_feature_edges()
... e.set(color_by=None, solid_color=(0.05, 0.05, 0.08))
... h.hide(); s.show(); e.show(); s.activate(); s.info()
... afx.view([1.0, -1.3, 0.9], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) | minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

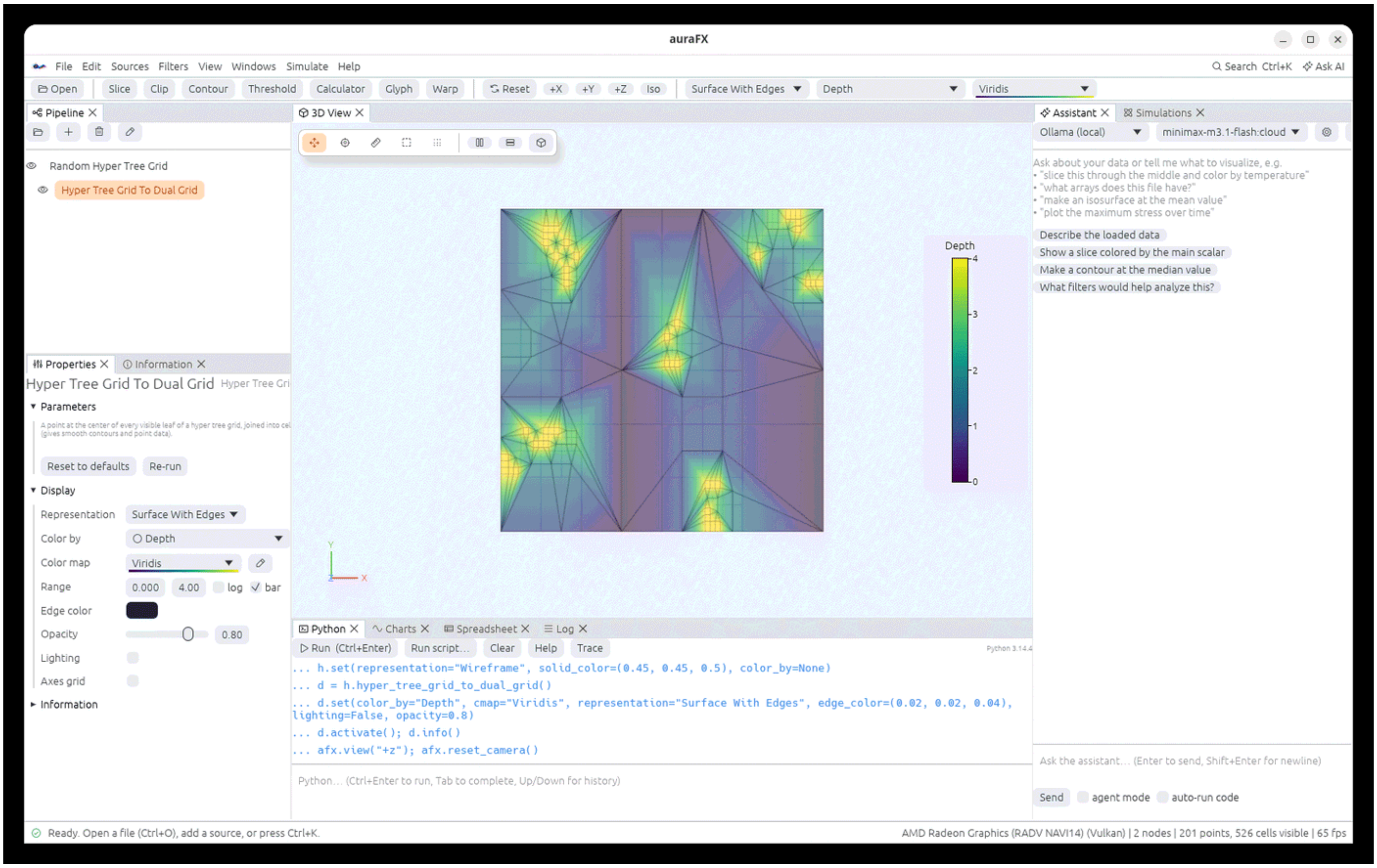
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 5694 points, 2379 cells visible | 65 fps


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
Depth
Viridis
Pipeline
3D View
Assistant
Simulations
Random Hyper Tree Grid
Hyper Tree Grid To Dual Grid
Depth
4
3
2
1
0
Y
X
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Properties | Information

Hyper Tree Grid To Dual Grid — Hyper Tree Gri

Parameters

A point at the center of every visible leaf of a hyper tree grid, joined into cel (gives smooth contours and point data).

Reset to defaults | Re-run

Display

Representation: Surface With Edges
Color by: ○ Depth
Color map: Viridis
Range: 0.000 4.00 ☐ log ✓ bar
Edge color: ■
Opacity: 0.80
Lighting: ☐
Axes grid: ☐

Information

```
... h.set(representation="Wireframe", solid_color=(0.45, 0.45, 0.5), color_by=None)
... d = h.hyper_tree_grid_to_dual_grid()
... d.set(color_by="Depth", cmap="Viridis", representation="Surface With Edges", edge_color=(0.02, 0.02, 0.04),
lighting=False, opacity=0.8)
... d.activate(); d.info()
... afx.view("+z"); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) | minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 201 points, 526 cells visible | 65 fps

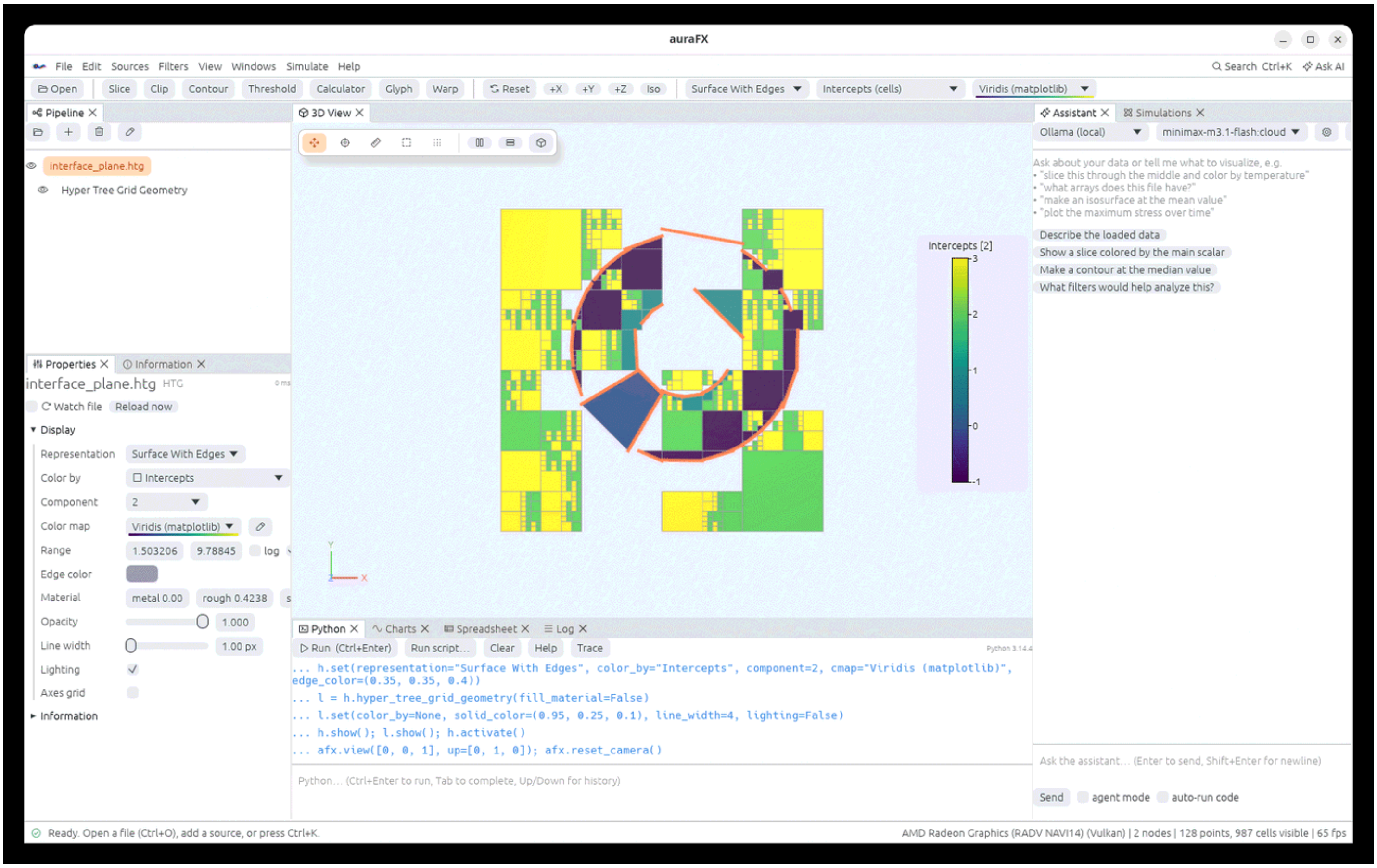
Material interfaces. A grid can describe where a material ends inside some cells, for simulations that track interfaces between materials. The file names two cell arrays (VTK's `InterfaceNormalsName` and `InterfaceInterceptsName`). For each cell they hold a normal *n* (three values) and intercepts (*d1*, *d2*, *type*); each interface is the plane $n \cdot x + d = 0$:

- *type* −1: one plane (*d1*); the material is where $n \cdot x + d1 \geq 0$;
- *type* 1: one plane (*d2*); the material is where $n \cdot x + d2 \leq 0$;
- *type* 0: the material lies between the two planes;
- *type* 2 or more, or a zero normal: the cell is pure (all material).

Hyper Tree Grid Geometry (and the default drawing) cuts each mixed cell by its planes:

- In 1-D and 2-D, each mixed leaf becomes the part of its line or quad that is in the material.
- In 3-D, the faces of mixed cells and of their neighbors are drawn cut to the material, and each mixed cell adds its interface polygons.

With **Fill material** off, 2-D grids give only the interface lines. Saving as `.htg` keeps the interface arrays.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
Intercepts (cells)
Viridis (matplotlib)
Pipeline
interface_plane.htg
Hyper Tree Grid Geometry
3D View
Intercepts [2]
3
2
1
0
-1
Y
X
Properties
Information
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Send

interface_plane.htg HTG

☐ Watch file Reload now

Display

Representation: Surface With Edges
Color by: Intercepts
Component: 2
Color map: Viridis (matplotlib)
Range: 1.503206 9.78845 ☐ log
Material: metal 0.00 rough 0.4238
Opacity: 1.000
Line width: 1.00 px
Lighting: ☑
Axes grid: ☐

Information

```
... h.set(representation="Surface With Edges", color_by="Intercepts", component=2, cmap="Viridis (matplotlib)",
edge_color=(0.35, 0.35, 0.4))
... l = h.hyper_tree_grid_geometry(fill_material=False)
... l.set(color_by=None, solid_color=(0.95, 0.25, 0.1), line_width=4, lighting=False)
... h.show(); l.show(); h.activate()
... afx.view([0, 0, 1], up=[0, 1, 0]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 128 points, 987 cells visible | 65 fps

```
h = afx.random_hyper_tree_grid(dimensions=(7, 5, 1), max_depth=5,
split_fraction=0.45, seed=3)
h.set(color_by="Depth", representation="Surface With Edges")
t = afx.hyper_tree_grid(dimensions=(3, 3, 1), max_depth=3, descriptor="R... | .R..
| ....")
coarse = h.hyper_tree_grid_evaluate_coarse(operator="Max")
top = h.hyper_tree_grid_depth_limiter(depth=2)
leaves = afx.load("tree.htg").hyper_tree_grid_to_unstructured_grid()
dual = h.hyper_tree_grid_to_dual_grid()
holes = afx.random_hyper_tree_grid(seed=3, masked_fraction=0.25)
walls = holes.hyper_tree_grid_geometry()
edges = holes.hyper_tree_grid_feature_edges()
h.save("tree.htg")
m = afx.load("interface_plane.htg")       # drawn cut to its material
lines = m.hyper_tree_grid_geometry(fill_material=False)
```

To Unstructured Grid gives every leaf its index in `OriginalIds`, masked files included. To Dual Grid makes points for visible leaves only. **Cell Centers** works on hyper tree grids directly; there is no separate Axis Reflection filter for them. `.htg` files auraFX writes read back unchanged.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Not available yet: version 1 `.htg` files. Ghost Cells only matters for data split across processes. Interfaces are used by Geometry only: slicing or clipping a grid with interfaces works on whole leaves.

Cell grids

A *cell grid* holds cells whose fields are polynomials rather than one value per point or cell. Simulation codes using finite elements of the discontinuous Galerkin (DG) kind write fields this way. On each cell a field is a sum of basis functions, each times a coefficient stored in the file. A field can jump from one cell to the next. It is VTK's newest data type. auraFX reads VTK's `.dg` files (JSON).

Cells can be hexahedra, tetrahedra, wedges, pyramids, quadrilaterals, triangles, edges and vertices. Fields can be:

- **nodal** (values at the corners, linear or quadratic; quadratic cells also have nodes on their edges, faces and middle), either shared between cells or kept per cell;
- **edge** fields (the H(curl) kind, for electric fields: their tangential part is continuous across faces);
- **face** fields (the H(div) kind, for fluxes: their normal part is continuous across faces);
- **constant** fields, one value per cell.

Edge and face fields are turned from the cell's own coordinates into space with the cell's shape (its Jacobian), as in VTK. The cells' shape can itself be quadratic (curved).

What you get:

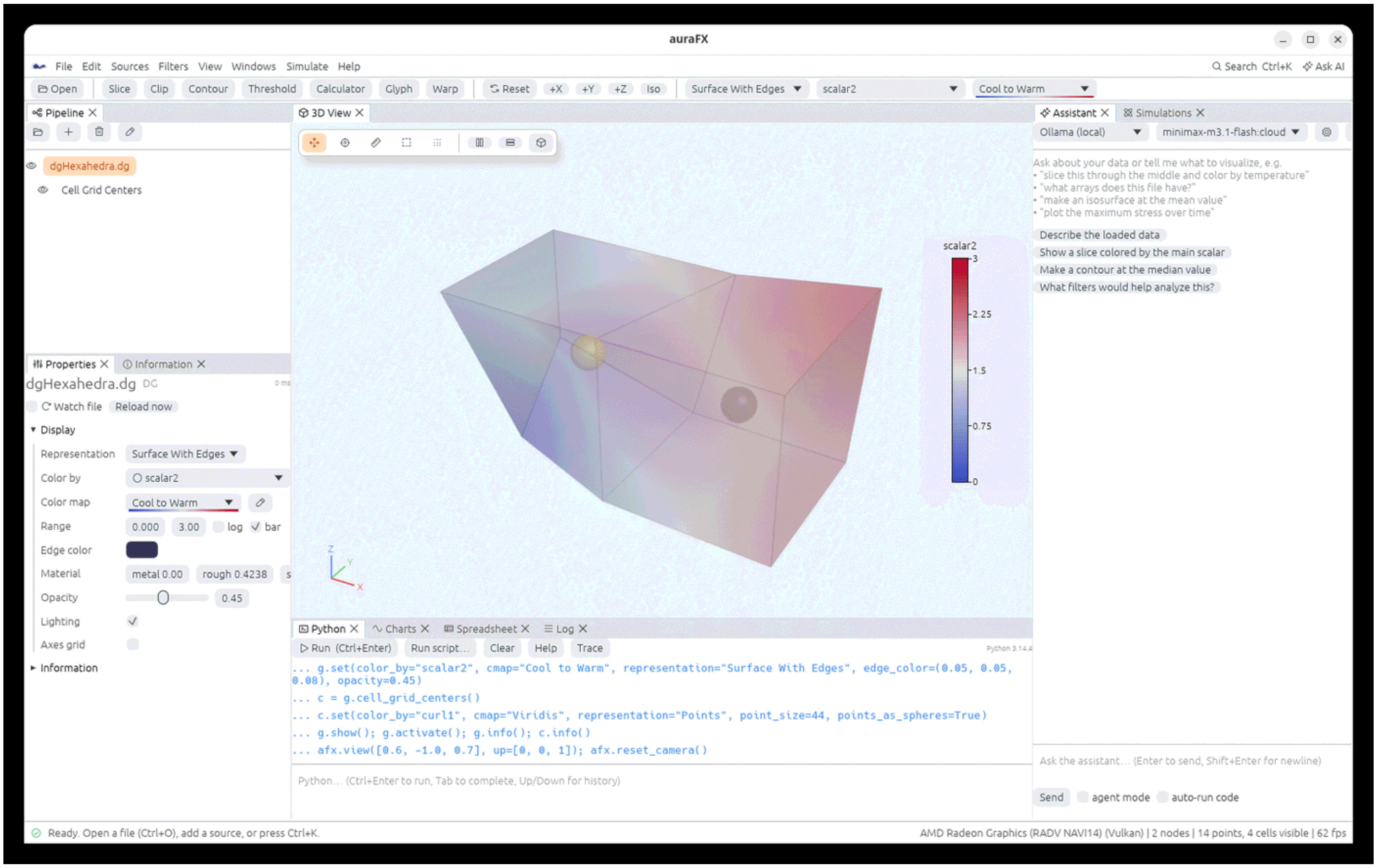
- The grid is drawn as its cells with every field evaluated at the corners. Quadratic cells are drawn as linear cells through their corners. Where cells meet, a field that jumps between them is averaged. Color by, the Spreadsheet, picking and every other filter see that mesh.
- **Information** names the cell types and the number of fields (and, after Compute Sides, the number of sides shown).
- Saving as `.dg` keeps the cell grid (every array, attribute and cell type, in the layout VTK writes). Saving as any other format writes the corner mesh as an unstructured grid.

Source (Sources › Cell Grid): one cell of a chosen **Cell type** with test fields: `hgrad` (1, 0, 0 at the first corner, 0 elsewhere), and on hexahedra, tetrahedra, wedges, quadrilaterals and triangles `hcurl` and `hdiv` (an edge and a face field whose first coefficient is 1). A vertex gets a `constant` field.

Filters (Filters › Data):

Filter	What it does
Cell Grid To Unstructured Grid	the cells with the fields at their corners (what is drawn); corners at the same place become one point
Cell Grid Centers	a point at the center of every cell with every field evaluated there (exact, not averaged)
Cell Grid Elevation	adds a field: the distance from **Origin** along **Axis** (or, with Spherical distance, from Origin)
Cell Grid Transform	moves, rotates (degrees) and scales the cells, as **Transform** does, or another 3-D attribute shared at points or stored per cell. Edge and face fields follow the new shape
Cell Grid Warp	moves the cells' points by **Scale** times a vector attribute shared at the points (the same basis and order as the shape)
Cell Grid Probe	finds the cells holding the points of a second dataset (the **Probe points**): a point for every cell each one is in, with `ContainingCellID` (the cell's number within its type), `CellType` (the VTK cell type number), `ParametricCoordinates` (where in the cell, in the cell's own coordinates) and, when **Attribute** names a field, the field's exact value there. Points in no cell are left out. A point on a face shared by two cells is listed twice
Cell Grid Compute Sides	finds the sides of the cells and draws them instead of the 3-D cells: by default the faces on the outside (those not shared by two cells).

Filter	What it does
	Sides to include picks faces, edges or corners; see below
Convert To Cell Grid	makes a cell grid from a mesh of vertices, lines, triangles, quads, tetrahedra, hexahedra, wedges and pyramids (other cells are skipped): point data becomes fields shared at the points, cell data constant fields (renamed “(cell)” when point data has the same name)


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
scalar2
Cool to Warm
Pipeline
dgHexahedra.dg
Cell Grid Centers
3D View
scalar2
3
2.25
1.5
0.75
0
Z
Y
X
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Send
Properties
Information

Ollama (local) · minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

dgHexahedra.dg DG

☐ Watch file · Reload now

Display

Representation	Surface With Edges
Color by	scalar2
Color map	Cool to Warm
Range	0.000 · 3.00 · ☐ log · ☑ bar
Edge color	
Material	metal 0.00 · rough 0.4238
Opacity	0.45
Lighting	☑
Axes grid	☐

Information

```
... g.set(color_by="scalar2", cmap="Cool to Warm", representation="Surface With Edges", edge_color=(0.05, 0.05,
0.08), opacity=0.45)
... c = g.cell_grid_centers()
... c.set(color_by="curl1", cmap="Viridis", representation="Points", point_size=44, points_as_spheres=True)
... g.show(); g.activate(); g.info(); c.info()
... afx.view([0.6, -1.0, 0.7], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

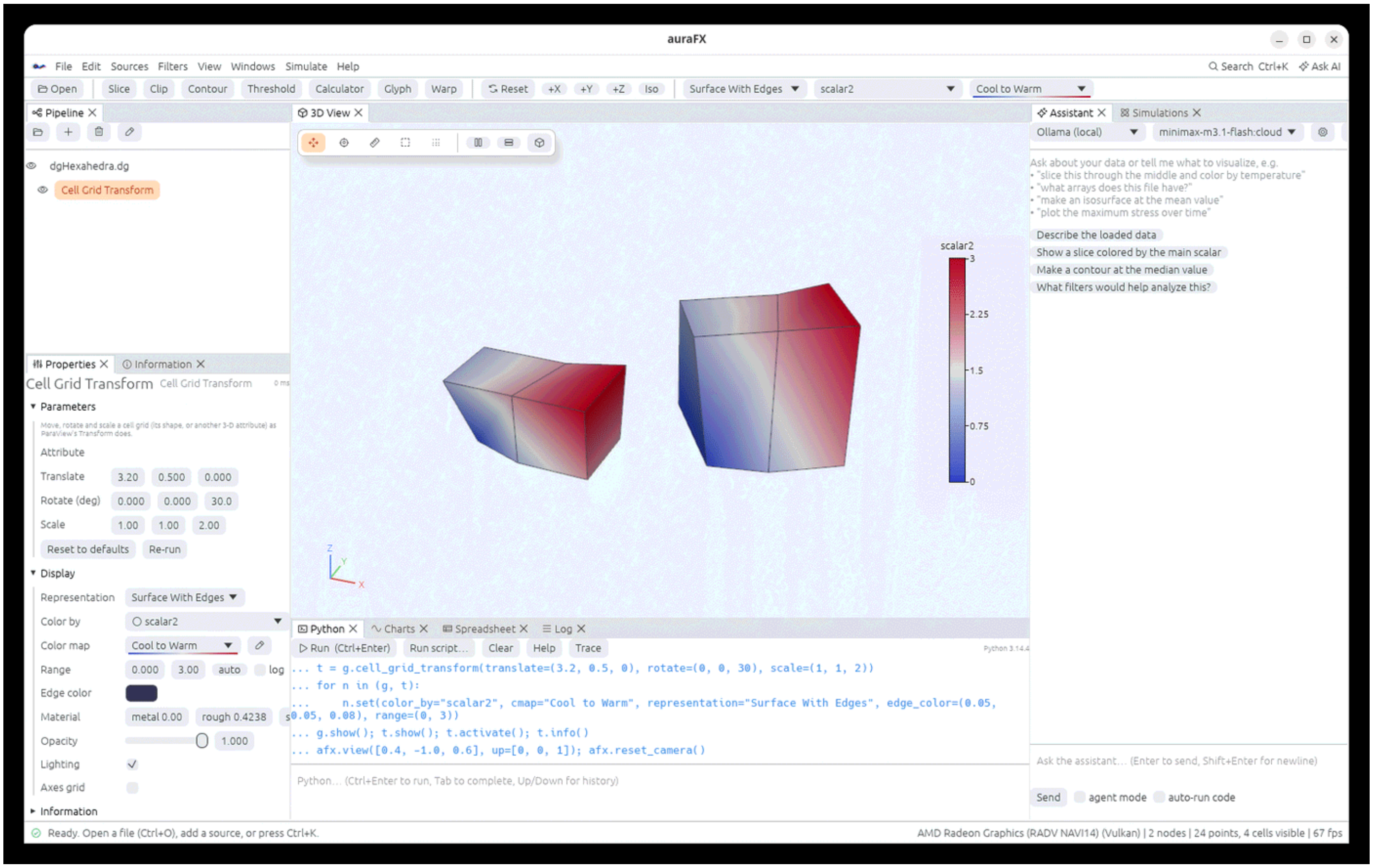
Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 14 points, 4 cells visible | 62 fps

```
g = afx.load("dgHexahedra.dg")
g.set(color_by="scalar2", representation="Surface With Edges")
centers = g.cell_grid_centers()
centers.set(color_by="curl1", points_as_spheres=True)
height = g.cell_grid_elevation(axis=(0, 0, 1))
cell = afx.cell_grid(cell_type="Tetrahedron")
mesh = g.cell_grid_to_unstructured_grid()
turned = g.cell_grid_transform(rotate=(0, 0, 30), scale=(1, 1, 2))
grid = afx.load("mesh.vtu").convert_to_cell_grid()
grid.save("mesh.dg")
cloud = afx.point_source(center=(1, 0.6, 0.5), radius=0.75, count=4000)
found = g.cell_grid_probe(cloud, attribute="quadratic")
outside = g.cell_grid_compute_sides()
edges = g.cell_grid_compute_sides(include="Edges of inputs",
omit_renderable=False)
```


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp
Reset +X +Y +Z Iso
Surface With Edges
scalar2
Cool to Warm
Pipeline
dgHexahedra.dg
Cell Grid Transform
3D View
scalar2
3
2.25
1.5
0.75
0
Z
Y
X
Assistant
Simulations
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Properties | Information

Cell Grid Transform — Cell Grid Transform

Parameters

Move, rotate and scale a cell grid (its shape, or another 3-D attribute) as ParaView's Transform does.

Attribute			
Translate	3.20	0.500	0.000
Rotate (deg)	0.000	0.000	30.0
Scale	1.00	1.00	2.00

Reset to defaults · Re-run

Display

Representation	Surface With Edges
Color by	○ scalar2
Color map	Cool to Warm
Range	0.000 · 3.00 · auto · ☐ log
Edge color	■
Material	metal 0.00 · rough 0.4238
Opacity	1.000
Lighting	☑
Axes grid	☐

Information

```
... t = g.cell_grid_transform(translate=(3.2, 0.5, 0), rotate=(0, 0, 30), scale=(1, 1, 2))
... for n in (g, t):
...     n.set(color_by="scalar2", cmap="Cool to Warm", representation="Surface With Edges", edge_color=(0.05,
0.05, 0.08), range=(0, 3))
... g.show(); t.show(); t.activate(); t.info()
... afx.view([0.4, -1.0, 0.6], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) · minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

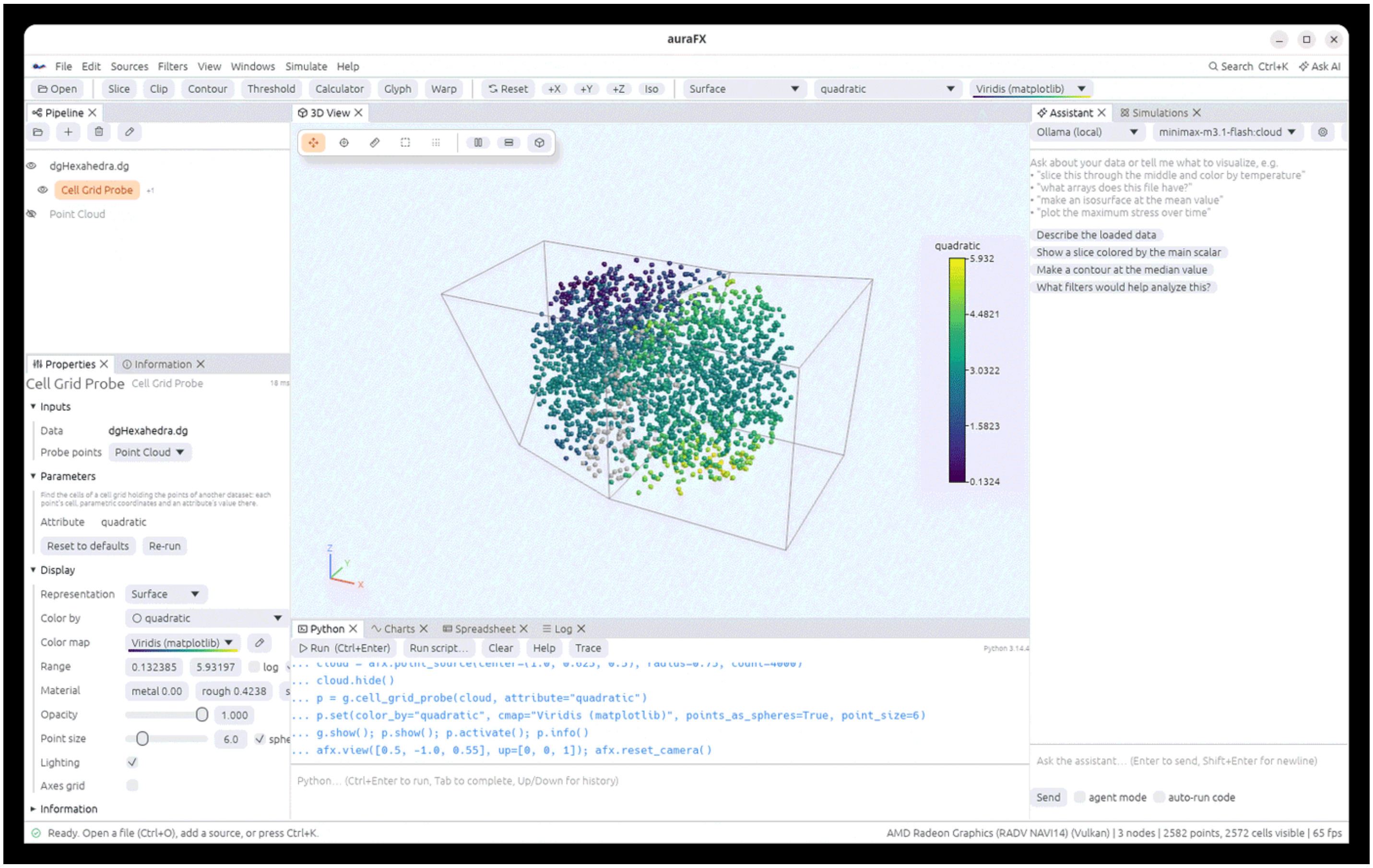
Ask the assistant… (Enter to send, Shift+Enter for newline)

○ agent mode ○ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 24 points, 4 cells visible | 67 fps

Cell Grid Probe looks up points in the cells. Select the cell grid, Ctrl+click the dataset whose points you want to look up (any dataset: its points are used), then choose Filters › Data › Cell Grid Probe. In Properties, **Probe points** picks the second input and **Attribute** the field to sample (empty: only find the cells). A cell is searched with a quick test first: the point must lie behind each of the cell's faces, taken as flat planes through the corners (on a flat cell, the edges' directions stand in for the normals). Then the point's coordinates in the cell are found step by step from the cell's middle. On a cell with curved faces a point just outside can pass the quick test; it is kept, with empty (NaN) coordinates and values. Points on an edge cell are kept when they lie between the two end planes and within the sphere around the edge, and are then kept only if they lie on the edge. The output points are in single precision, and with several cell types they are listed in the file's order. The grey points in the picture are such points, next to the curved face the two cells share.


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
quadratic
Viridis (matplotlib)
Pipeline
dgHexahedra.dg
Cell Grid Probe
Point Cloud
3D View
quadratic
5.932
4.4821
3.0322
1.5823
0.1324
Z
Y
X
Assistant
Simulations
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Cell Grid Probe Cell Grid Probe 18 ms

Inputs

Data dgHexahedra.dg

Probe points Point Cloud

Parameters

Find the cells of a cell grid holding the points of another dataset: each point's cell, parametric coordinates and an attribute's value there.

Attribute quadratic

Display

Representation Surface

Color by ○ quadratic

Color map Viridis (matplotlib)

Range 0.132385 5.93197 ☐ log

Material metal 0.00 rough 0.4238

Opacity 1.000

Point size 6.0 ✓ sphe

Lighting ✓

Axes grid ☐

```
... cloud = afx.point_source(center=(1.0, 0.625, 0.5), radius=0.75, count=4000)
... cloud.hide()
... p = g.cell_grid_probe(cloud, attribute="quadratic")
... p.set(color_by="quadratic", cmap="Viridis (matplotlib)", points_as_spheres=True, point_size=6)
... g.show(); p.show(); p.activate(); p.info()
... afx.view([0.5, -1.0, 0.55], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

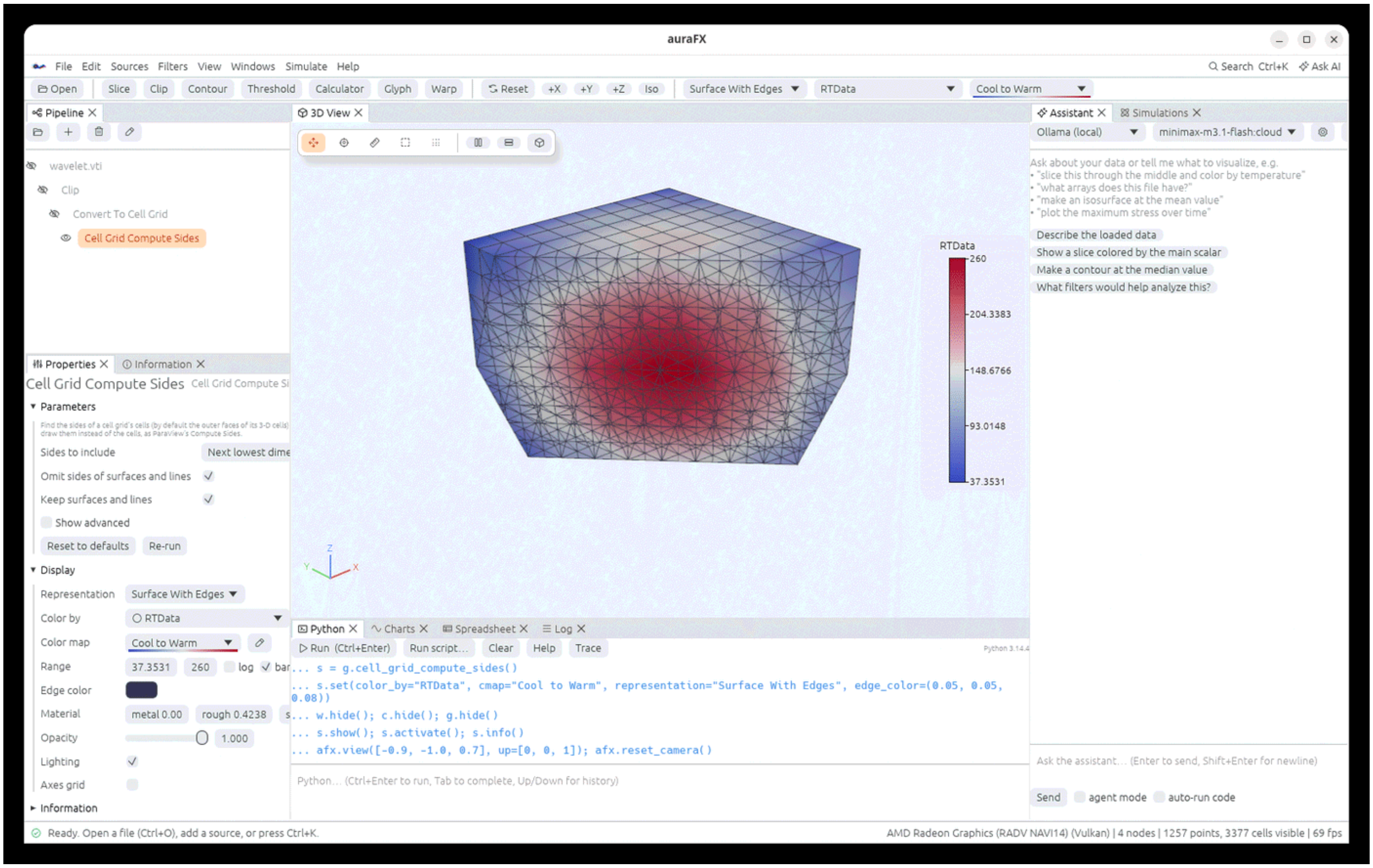
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 2582 points, 2572 cells visible | 65 fps

Cell Grid Compute Sides finds the sides of the cells. Every side of every cell is listed, by cell and then by side, and a side is matched with the same side of a neighbor cell whichever way round its corners go. A side is kept once, for the first cell that has it. Properties:

- **Sides to include**:
    - *Next lowest dimension* (the default): the faces of 3-D cells, the edges of 2-D cells and the end points of edges;
    - *Surfaces of inputs*: the faces of 3-D cells;
    - *Edges of inputs*: the edges of 3-D and 2-D cells;
    - *Vertices of inputs*: the corners of every cell;
    - *All sides*: all of these.
- **Omit sides of surfaces and lines** (on by default): 2-D and 1-D cells give no sides, since they can be drawn as they are.
- **Keep surfaces and lines** (on by default): keep drawing the 2-D and 1-D cells and the sides already listed. 3-D cells are always hidden behind their sides.
- **Keep** (advanced):
    - *Boundary* (the default): faces of 3-D cells (or edges of 2-D cells, end points of edges) found once, so not shared by two cells, and every lower side (edges and corners of 3-D cells) once;
    - *Odd count*: every side found an odd number of times;
    - *Every side*: each side once.

The sides are stored in the grid as lists of (cell, side number) pairs, one per side shape ("quadrilateral sides of vtkDGHex"), as VTK stores them. The fields on a side are the fields of its cell, so a field that jumps between cells shows the value from the cell the side belongs to. Running Compute Sides on its result adds the sides of those sides, for example the edges of the outer faces. Saving as `.dg` keeps the lists. Any filter other than the cell grid ones works on what is drawn: the sides. VTK's markings of ghost and hidden points (which skip sides) are not read.


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
RTData
Cool to Warm
Pipeline
wavelet.vti
Clip
Convert To Cell Grid
Cell Grid Compute Sides
3D View
RTData
260
204.3383
148.6766
93.0148
37.3531
Z
Y
X
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Properties
Information

Cell Grid Compute Sides

Parameters

Find the sides of a cell grid's cells (by default the outer faces of its 3-D cells) draw them instead of the cells, as ParaView's Compute Sides.

Sides to include: Next lowest dime

Omit sides of surfaces and lines ☑

Keep surfaces and lines ☑

☐ Show advanced

Reset to defaults · Re-run

Display

Representation: Surface With Edges

Color by: ○ RTData

Color map: Cool to Warm

Range: 37.3531 · 260 · ☐ log · ☑ bar

Material: metal 0.00 · rough 0.4238

Opacity: 1.000

Lighting ☑

Axes grid ☐

Information

```
... s = g.cell_grid_compute_sides()
... s.set(color_by="RTData", cmap="Cool to Warm", representation="Surface With Edges", edge_color=(0.05, 0.05,
0.08))
... w.hide(); c.hide(); g.hide()
... s.show(); s.activate(); s.info()
... afx.view([-0.9, -1.0, 0.7], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

Send ☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 4 nodes | 1257 points, 3377 cells visible | 69 fps

On quadrilaterals and triangles the cell's shape matrix (its Jacobian) is flat, so it is completed with the cell's unit normal; edge and face fields then lie in the cell's plane, and Cell Grid Probe gives coordinates on flat cells and edges.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Not available yet: cell grid range and copy filters. Slicing or clipping works on the corner mesh, so curved cells come out straight.

Writers

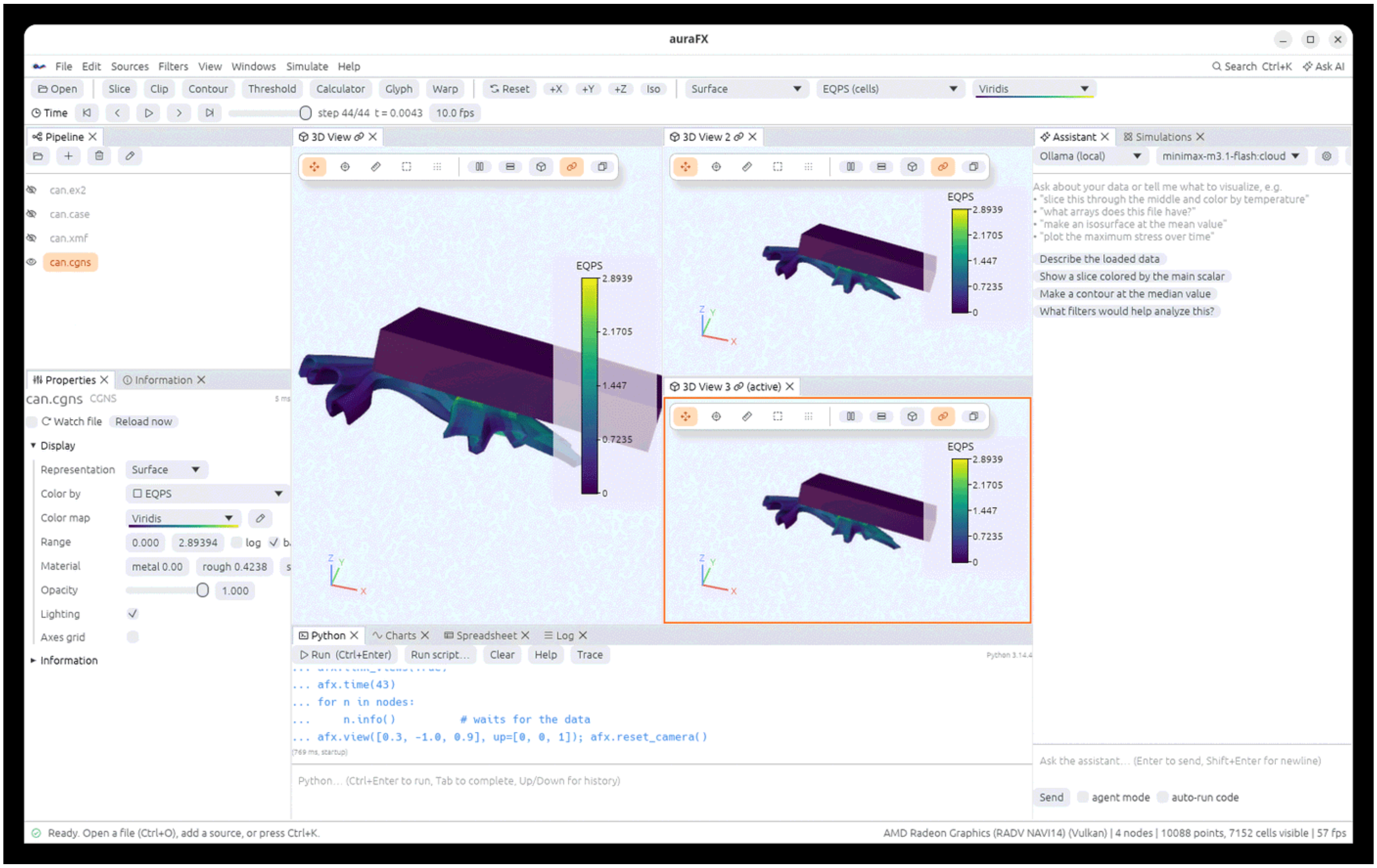
Format	Extensions	Notes

VTK XML	`.vtu` `.vtp`
	`.vti` `.vtr`
	`.vts` `.vtm`
		appended binary; multiblocks as a folder of pieces

VTK hyper tree grid	`.htg`	XML version 2: the trees, mask and cell data
VTK cell grid	`.dg`	the JSON layout VTK writes: arrays, attributes and cell types
VTK legacy	`.vtk`	
VTKHDF	`.vtkhdf`	all dataset kinds; multiblocks as a collection; time series with `all_times=True`; written with auraFX's own HDF5 encoder (read by h5py and VTK)
Exodus II	`.exo`	blocks, nodal and element variables, every time step with `all_times=True`
EnSight Gold	`.case`	C binary; one part per block (image, rectilinear and structured grids as blocks, the rest as elements of every standard type, polygons and polyhedra); every time step with `all_times=True`, the geometry written once when it does not change
CGNS	`.cgns`	HDF5 storage (CGNS 4): a zone per block, image/rectilinear/structured grids as structured zones, the rest as one element section (MIXED when the cell types differ) plus NGON/NFACE for polygons and polyhedra; vectors as `NameX/NameY/NameZ`; with `all_times=True` one solution per step (and coordinates per step when the points move), but the cells must stay the same
Xdmf 3	`.xmf` + `.h5`	the description in the `.xmf`, the arrays in an HDF5 file beside it; block trees as spatial collections, time steps as a temporal collection that shares unchanged points and cells
CSV, STL, PLY		
Scenes	`.glb` `.obj` `.x3d` `.wrl`	see Exporting

Format	Extensions	Notes
Pictures	`.png` `.svg`, GIF/MP4 animations	see Exporting

Files written by the VTK XML, VTKHDF, Exodus, EnSight, Xdmf and CGNS writers were read back with the same points, cells and values (verified against a reference implementation; the comparisons are listed in the verification notes, `docs/reference/verification.md` in the source tree). EnSight groups the cells of each part by type, and CGNS puts polyhedra after the other cells, so cells can come back in a different order (each keeps its values). Cell types a format does not have (Lagrange and Bezier cells, for example) are written by their corner points. A CGNS zone holds cells of one dimension: in a volume mesh, boundary faces and lines are left out.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
EQPS (cells)
Viridis
Time
step 44/44 t = 0.0043
10.0 fps
Pipeline
can.ex2
can.case
can.xmf
can.cgns
3D View
3D View 2
3D View 3 (active)
EQPS
2.8939
2.1705
1.447
0.7235
0
Z
Y
X
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

can.cgns CGNS 5 ms
☐ Watch file Reload now
Display
Representation: Surface
Color by: ☐ EQPS
Color map: Viridis
Range: 0.000 2.89394 ☐ log ☑ b
Material: metal 0.00 rough 0.4238
Opacity: 1.000
Lighting: ☑
Axes grid: ☐
Information

```
... afx.time(43)
... for n in nodes:
...     n.info()          # waits for the data
... afx.view([0.3, -1.0, 0.9], up=[0, 0, 1]); afx.reset_camera()
(769 ms, startup)
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 4 nodes | 10088 points, 7152 cells visible | 57 fps

Curved cells

VTK's Lagrange and Bezier cells of any order, quadratic cells and serendipity cells are read with their node numbering, drawn curved and cut by the filters along their curvature; see Volumes, point clouds and curved cells.

Your own formats

A plugin reader is a Python function with `@afx.reader(extensions=[...])`; it takes precedence over the built-in reader for its extensions and appears in the Open dialog.

Exporting pictures, scenes and animations

Screenshots

File › Save Screenshot… (Ctrl+Shift+S) opens a small dialog:

- **Size**: the view’s size, a multiple of it (2×, 4×), 1920×1080, 3840×2160, or any size; **keep the view’s shape** ties width and height together. Point sizes and line widths are in pixels, so they look thinner in a larger picture.
- **Background › transparent** saves a PNG with an alpha channel, with clean edges.
- **Annotations** includes the legend, axes-grid labels, time label, text annotations and the orientation axes, laid out as on screen and scaled with the picture.
- **PNG** or **SVG**: an SVG holds the surfaces as shaded polygons sorted back to front, the lines and points, and the annotations as real text, for figures that scale without pixels (where surfaces cut through each other the boundary is jagged; convert to PDF with Inkscape if needed).

With ray tracing on, the picture is traced at the requested size first.

```
afx.screenshot("fig.png")                               # the view as shown
afx.screenshot("fig.png", width=3840, transparent=True) # keeps the view's shape
afx.screenshot("fig.png", width=1600, height=900, annotations=False)
afx.screenshot("fig.svg")
```

`afx.screenshot` returns only when the file is written, so a script can change the camera or the data between pictures.

Scenes for other programs

File › Export Scene… (`afx.export_scene("scene.glb")`) writes the visible objects, with their colors, as:

Format	Notes
glTF binary (`.glb`)	for Blender, three.js, PowerPoint, Windows 3D Viewer; vertex colors, normals, translucency, materials (metallic, roughness)
OBJ (`.obj`)	with vertex colors after the coordinates (Blender and MeshLab read them)

Format	Notes
X3D (`.x3d`), **VRML** (`.wrl`)	absolute coordinates, vertex colors

Volumes are not exported (they are not geometry).

Animations

File › Animation… records the time steps, a turntable or the keyframes as **GIF**, **MP4** (`ffmpeg` on the PATH) or a folder of PNG frames; see Time, charts and animation.

Saving data

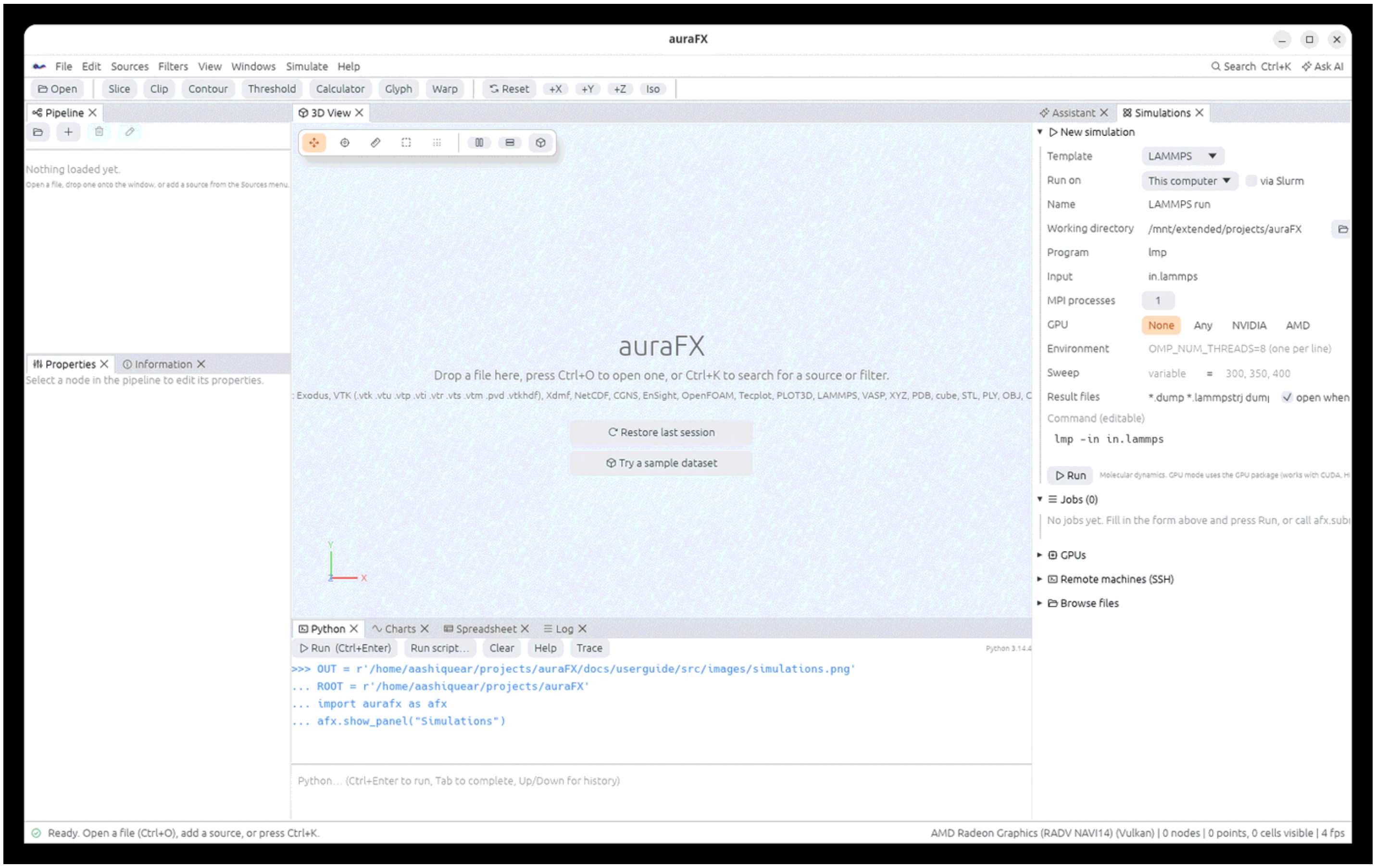
Right-click an object › **Save data…**, or `node.save("out.vtu")` , writes its output; the format follows the extension:

Extension	Format
`.vtu .vtp .vti` `.vtr .vts .vtm`	VTK XML (appended binary); `.vtm` writes a folder of pieces
`.vtk`	legacy VTK
`.vtkhdf`	VTKHDF (HDF5); multiblocks as a collection with the block tree; `all_times=True` writes every time step into one file
`.exo`	Exodus II with blocks and variables; `all_times=True` writes every time step
`.case`	EnSight Gold (C binary): the case file plus a geometry file and one file per variable next to it; `all_times=True` writes every time step
`.xmf`	Xdmf 3: the description plus an `.h5` file with the arrays; `all_times=True` writes every time step
`.cgns`	CGNS (HDF5); `all_times=True` writes every time step when the cells stay the same (moving points are fine)
`.csv`	the point data or the table
`.stl .ply`	surfaces

```
d.save("slice.vtp")
d.save("run_all.exo", all_times=True)
d.save("run_all.vtkhdf", all_times=True)
d.save("run.case", all_times=True)      # run.case, run.geo, run.Temp0000, ...
d.save("run.xmf", all_times=True)       # run.xmf and run.h5
d.save("run.cgns", all_times=True)
```

When the data has several time steps and the chosen format holds them all (Exodus, VTKHDF, EnSight, Xdmf, CGNS), the Save dialog writes every step.

Running simulations


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Pipeline
3D View
Assistant
Simulations
Properties
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Y
X

Nothing loaded yet.

Open a file, drop one onto the window, or add a source from the Sources menu.

Select a node in the pipeline to edit its properties.

auraFX

Drop a file here, press Ctrl+O to open one, or Ctrl+K to search for a source or filter.

: Exodus, VTK (.vtk .vtu .vtp .vti .vtr .vts .vtm .pvd .vtkhdf), Xdmf, NetCDF, CGNS, EnSight, OpenFOAM, Tecplot, PLOT3D, LAMMPS, VASP, XYZ, PDB, cube, STL, PLY, OBJ, C

Restore last session

Try a sample dataset

New simulation

Template	LAMMPS
Run on	This computer ☐ via Slurm
Name	LAMMPS run
Working directory	/mnt/extended/projects/auraFX
Program	lmp
Input	in.lammps
MPI processes	1
GPU	None Any NVIDIA AMD
Environment	OMP_NUM_THREADS=8 (one per line)
Sweep	variable = 300, 350, 400
Result files	*.dump *.lammpstrj dum ☑ open when

Command (editable)

```
lmp -in in.lammps
```

Run — Molecular dynamics. GPU mode uses the GPU package (works with CUDA, H

Jobs (0)

No jobs yet. Fill in the form above and press Run, or call afx.sub

GPUs

Remote machines (SSH)

Browse files

```
>>> OUT = r'/home/aashiquear/projects/auraFX/docs/userguide/src/images/simulations.png'
... ROOT = r'/home/aashiquear/projects/auraFX'
... import aurafx as afx
... afx.show_panel("Simulations")
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 0 nodes | 0 points, 0 cells visible | 4 fps

The **Simulations** panel (right side; **Simulate › Run a simulation…**, or **Windows › Simulations**) launches solvers and opens their results when they finish.

A new simulation

Field	Meaning
Template	LAMMPS, MOOSE app, OpenFOAM, Python pipeline or Custom command; fills in the program, input and result-file patterns
Run on	this computer, or a remote machine from the SSH list; **via Slurm** submits with `sbatch` there (partition, time and extra `#SBATCH` lines appear)
Name, **Working directory**	where the run lives; a remote directory when running over SSH
Program, **Input**, **MPI processes**	the command line is assembled from these and shown below, editable

Field	Meaning
GPU	None, Any, NVIDIA or AMD; the panel finds the GPUs on the host and sets the solver's flags (LAMMPS GPU package, `CUDA_VISIBLE_DEVICES` / `HIP_VISIBLE_DEVICES`)
Environment	extra `VAR=value` lines (one per line)
Sweep	a variable name and a list of values: one job per value, `{var}` in the command replaced, each in its own folder
Result files	patterns of the files to open when the run ends (**open when done**)

Run starts the job. Jobs appear in the **Jobs** list with their state, a live log tail, and buttons to cancel, open the results or the folder, and resubmit. Files a running solver writes can be opened while it runs; auraFX reloads them as they change (live reload).

Remote machines

Remote machines (SSH) holds the hosts: a name, the SSH target (`user@machine`, or an alias from `~/.ssh/config`), port, key file, jump host, and **setup lines** run before every command (module loads, `source` of an environment, `AURAFX_SERVER=...`). SSH keys or an agent must be set up so that `ssh host` works without a password prompt. The same hosts serve **File › Open on Server…** (see Data on another machine).

GPUs lists the GPUs of each host; **Browse files** opens a remote file browser to pick inputs or results.

From Python

```
jid = afx.submit("lmp -sf gpu -pk gpu 1 -in in.melt", host="rocm-server",
                 workdir="~/runs/melt", gpu="amd:0", outputs="dump.melt", auto_load=True)
afx.sweep("lmp -var T {T} -in in.melt", T=[300, 600, 900], host="cluster", workdir="~/sweep")
afx.jobs(); afx.job(jid); afx.cancel(jid)
afx.gpus("rocm-server"); afx.hosts()
afx.load_remote("rocm-server", "~/runs/melt/dump.melt")    # copy a file here and open it
```

`gpu` is `None`, `"any"`, `"nvidia"`, `"amd"` or e.g. `"nvidia:0,1"`. `afx.job(id)` returns the state, exit code, log tail and output files.

Data on another machine

When the data lives on a cluster or a GPU box, auraFX can read and filter it **there** and bring back only what you look at, with nothing to set up beyond SSH.

How it works

`aurafx-server` is a small program without a window. The app starts it on the host over SSH (`ssh host aurafx-server --stdio`) and talks to it through that connection, so keys, jump hosts and `~/.ssh/config` work as usual and no port is opened. Files you open on the server are read there; filters you add below them (Slice, Clip, Contour, Threshold, Stream Tracer, temporal filters and so on) run there too. What comes back:

Situation	What is transferred
Visible, output smaller than 64 MB	the whole output
Visible, larger	its outer surface (all arrays), which is what the 3D view draws
Hidden (for example a reader with a slice below it)	only its summary: arrays, ranges, bounds, time steps
Bring all data ticked in Properties, or a filter below it that runs here	the whole output

The Information panel always describes the full data on the server. Python filters (Programmable Filter, Python Calculator) and plugin filters run on this computer, on the data brought over. The server keeps results, so changing one filter only re-runs that filter there.

Setting up a host

1. Make sure `ssh yourhost` works without a password prompt.
2. Put `aurafx-server` on the host: **File › Open on Server… › Install server** copies the one shipped with the app to `~/.local/bin` (Linux hosts of the same architecture; the packages run on glibc 2.35 or newer), or build it there with `cargo build --release -p aurafx-server`, or point `AURAFX_SERVER` at it in the host's setup lines.
3. **Check server** shows its version and GPU.

Hosts come from the Simulations panel and from `~/.ssh/config`. “This computer” runs the server locally, which is handy for trying it out.

Opening files

File › Open on Server…: pick the host, browse to the file, double-click it. A cloud icon marks objects that run on a server; hover it, or look at the top of Properties, to see what was transferred and how long the server took.

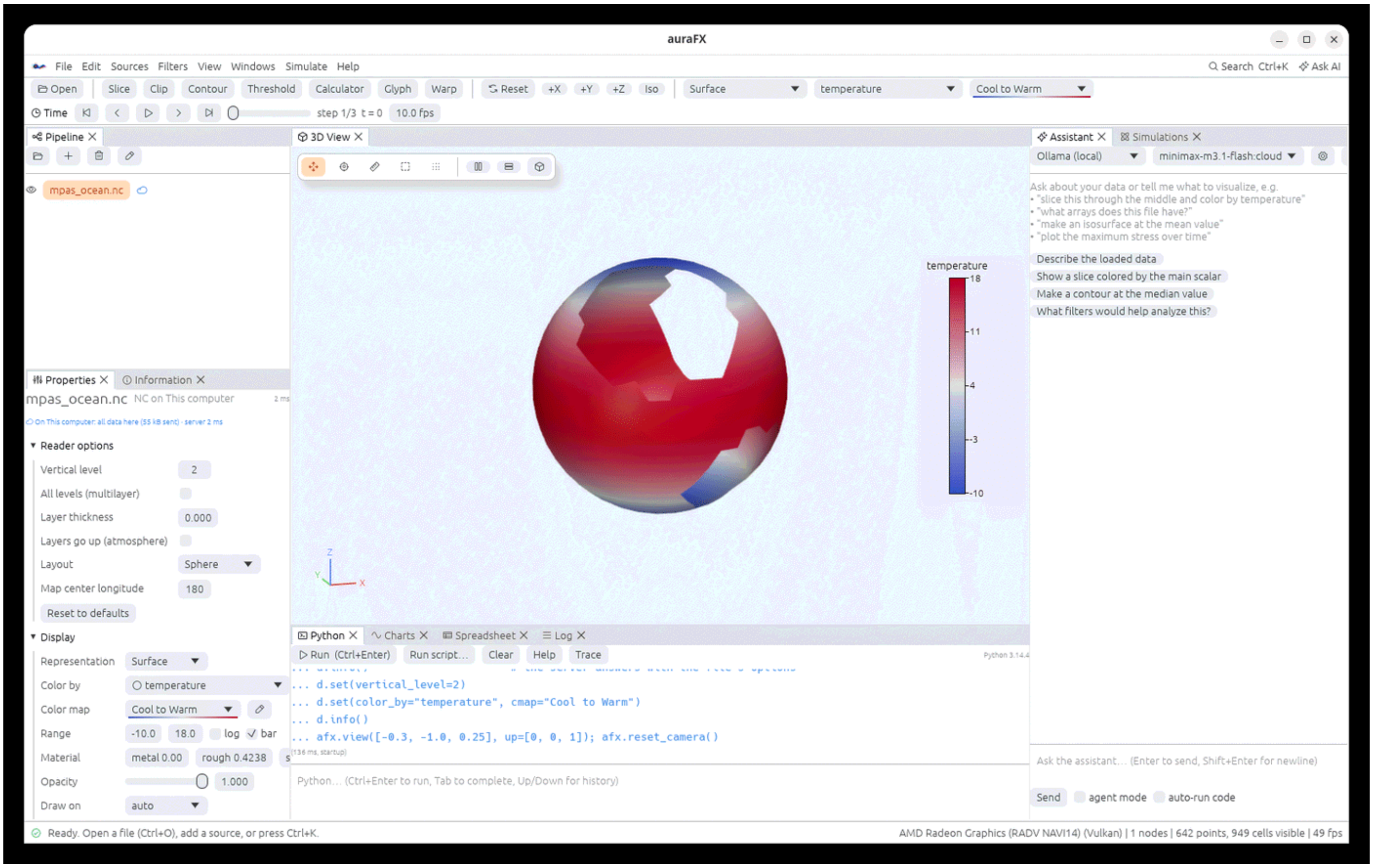
```
d = afx.load("/scratch/run42/out.e", host="cluster")   # a Simulations host, ssh
alias or user@machine
d = afx.load("ssh://cluster/scratch/run42/out.e")       # the same
s = d.slice(normal=(0, 0, 1))                            # runs on the server
s.set(all_data=True)                                     # bring the whole output
over
afx.servers()                                            # connections and the
machines' details
```

Reader options on the server

Files on a server have the same **Reader options** as local ones (an MPAS file’s vertical level and layout, a NetCDF file’s grid, a CAM-SE file’s connectivity file…). The server looks at the file and reports its options with its first answer, so they appear under Properties a moment after the file opens. Changing one has the server read the file again with it; on a parallel server, every process reads its piece with it. Paths in options, such as a CAM-SE connectivity file, are paths on the server.

```
d = afx.load("/scratch/ocean/output.nc", host="cluster")
d.set(vertical_level=10)           # read on the server
```

From Python, an option the file does not have is refused once the server has said which ones it has; before that, the server ignores names it does not know. Saved states keep the options.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
temperature
Cool to Warm
Time
step 1/3 t = 0
10.0 fps
Pipeline
3D View
Assistant
Simulations
mpas_ocean.nc
Properties
Information
temperature
18
11
4
-3
-10
Z
Y
X
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send

mpas_ocean.nc NC on This computer 2 ms

On This computer: all data here (55 kB sent) · server 2 ms

▾ Reader options

Vertical level: 2

All levels (multilayer): ☐

Layer thickness: 0.000

Layers go up (atmosphere): ☐

Layout: Sphere

Map center longitude: 180

Reset to defaults

▾ Display

Representation: Surface

Color by: ○ temperature

Color map: Cool to Warm

Range: -10.0 18.0 ☐ log ☑ bar

Material: metal 0.00 rough 0.4238

Opacity: 1.000

Draw on: auto

```
... d.set(vertical_level=2)
... d.set(color_by="temperature", cmap="Cool to Warm")
... d.info()
... afx.view([-0.3, -1.0, 0.25], up=[0, 0, 1]); afx.reset_camera()
(136 ms, startup)
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ollama (local) · minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data

Show a slice colored by the main scalar

Make a contour at the median value

What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 642 points, 949 cells visible | 49 fps

Drawing on the server

Some outputs are too big even as a surface. Then the server draws the object with its own GPU and sends pictures (a color and a depth image per view, compressed), which the 3D view blends by depth with the local objects, so everything hides and is hidden correctly. While you move the camera the last picture stays up until the next one arrives.

Display › Draw on in Properties picks per object: **auto** (drawn here unless the surface is over 64 MB), **this computer**, or **the server** (`node.set(draw_on="server")`). Picking, rulers and selections need the geometry here, so switch to “this computer” for those. Translucent objects, volumes and Surface LIC are always drawn here. The view’s lights and shadows travel with each request; screen effects (ambient occlusion, Eye-Dome Lighting, FXAA, tone mapping) are applied here to the whole picture.

Parallel servers

Data too big for one process can be spread over several, like `mpirun pvserver` : **Open on Server › Processes**, or `afx.load(path, host="cluster", processes=8)` , starts eight worker processes on the host (`aurafx-server --workers node1:16,node2:16` spreads them over machines sharing a file system). Partitioned VTK files are read piece by piece; other files are read by every process and split. Cell-by-cell filters run on every piece with automatic values taken from the whole data; the pieces come back appended, as a merged surface, or as pictures composited by depth. Results match a single process exactly.

Not yet: MPI launchers (`srun`) for the workers, ghost cells, sharing one server between people, temporal filters on parallel servers.

Grains and crystals

auraFX reads electron backscatter diffraction (EBSD) maps, serial-section stacks and voxel microstructures, and works with crystal orientations. It colors them, converts them between representations and re-expresses them in a turned reference frame. It also crops, resamples and aligns voxel volumes. It finds the grains (features) and measures them: sizes, shapes, average orientations, misorientations, neighbours and boundaries. Texture analysis and building synthetic microstructures come in later releases.

EBSD maps

Open a map with **File › Open** or `afx.load("scan.ang")`.

Format	Extension	Notes
TSL/EDAX	`.ang`	square grids; hexagonal grids are not read yet
Oxford	`.ctf`	angles in degrees in the file, converted to radians
EBSD stacks	`.h5ebsd`	serial sections from TSL or Oxford (HKL) maps, read as one volume
EDAX OIM HDF5	`.h5`	square-grid scans; choose the scan with the **Scan** reader option
Bruker Esprit HDF5	`.h5`	angles in degrees in the file; phases from their space group
Microstructures	`.dream3d`	voxel volumes and meshes with their cell, feature and phase arrays; see below

`.h5` files are recognised by their contents, so other HDF5 files with that extension still open with their own readers.

A map becomes image data with one cell per measured point, the cells centered on the points (so the map’s corner lies half a step before the first point). Its arrays are:

- **Cell data:** `EulerAngles` (three Bunge angles φ1, Φ, φ2 in radians, as stored in the file) and `Phases` (the phase of each point, 0 where the pattern was not indexed). A `.ang` file adds `Image Quality`, `Confidence Index`, `SEM Signal` and `Fit`. A `.ctf` file adds `Bands`, `Error`, `MAD`, `BC` and `BS`.
- **Field data:** one entry per phase, entry 0 standing for “not indexed”. `CrystalStructures` holds the crystal class of each phase (see the numbers below), `MaterialName` its name and `LatticeConstants` a, b, c, α, β, γ.

A single-phase `.ang` scan writes phase 0 for every point. auraFX counts those points as phase 1, unless their confidence index is negative.

An `.h5ebsd` stack becomes one volume with a cell layer per slice, the slices `Z Resolution` apart. The stack's sample and Euler transformations are not applied. They are kept as the field arrays `SampleTransformationAngle`, `SampleTransformationAxis`, `EulerTransformationAngle` and `EulerTransformationAxis`. Apply them with Rotate Orientations (and Transform for the sample) if you need them.

Hexagonal axes. Oxford files describe hexagonal crystals with the [10-10] direction along x. auraFX, `.ang` files and the color key put [2-1-10] along x. By default the `.ctf` reader adds 30° to φ2 of hexagonal and trigonal phases to match. To keep the angles as stored, set the reader option **Hexagonal axes** to *As stored* in the Properties panel, or in Python:

```
m = afx.load("titanium.ctf", hexagonal_axes="As stored")
```

Crystal classes

Number	Class	Number	Class
0	Hexagonal 6/mmm	6	Orthorhombic mmm
1	Cubic m-3m	7	Tetragonal 4/m
2	Hexagonal 6/m	8	Tetragonal 4/mmm
3	Cubic m-3	9	Trigonal -3
4	Triclinic -1	10	Trigonal -3m
5	Monoclinic 2/m	999	Unknown

Face-centered and body-centered cubic metals (nickel, aluminium, iron) are Cubic m-3m. Hexagonal close-packed metals (titanium, magnesium, zirconium) are Hexagonal 6/mmm.

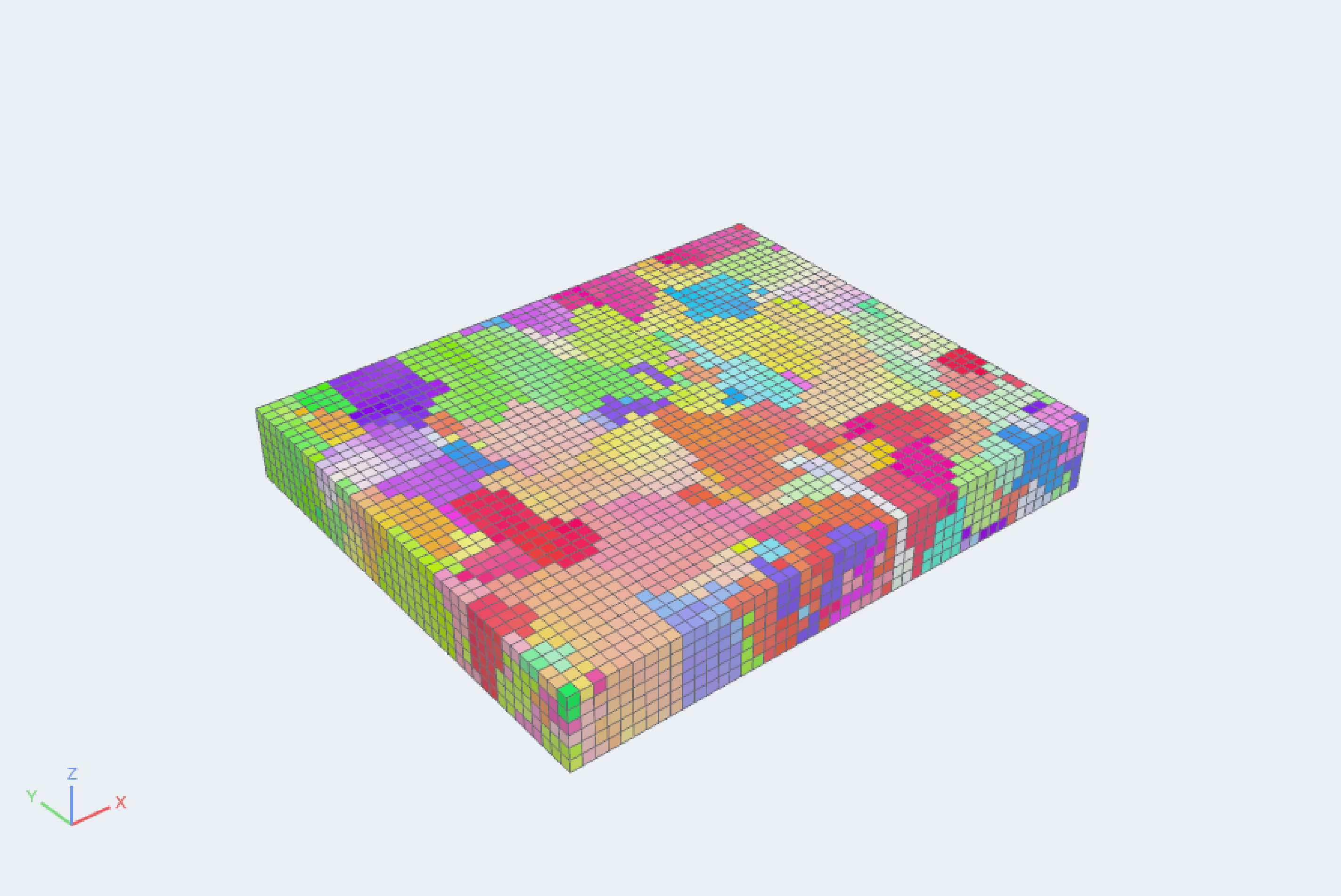
Microstructure files (.dream3d)

A `.dream3d` file holds one or more data containers, each a geometry with arrays. auraFX reads image (voxel), rectilinear, vertex, edge, triangle, quadrilateral, tetrahedral and hexahedral geometries.

- Cell arrays (voxels, triangles, …) become cell data, and vertex arrays become point data.
- Feature arrays (one row per feature, such as `AvgQuats`, `Volumes` or `Centroids`) and phase arrays (`CrystalStructures`, `LatticeConstants`, `MaterialName`) become field data.
- Neighbor lists are not read.

- A file with several data containers opens as blocks. The **Data container** reader option picks one.
- Large volumes load faster with only the arrays you need. Give their names in the **Arrays** reader option, separated by commas.

```
v = afx.load("volume.dream3d", arrays="EulerAngles, Phases, FeatureIds")
```



Orientation conventions

An orientation turns the sample axes into the crystal axes. A direction with sample coordinates *s* has crystal coordinates *g s*, where *g* is the orientation matrix. Euler angles are Bunge angles in radians: *g* = Rz(φ2) Rx(Φ) Rz(φ1). Quaternion arrays store (x, y, z, w), with w ≥ 0. Orientation matrices are stored row by row (9 components).

Orientation Colors

Filters › Grains › Orientation Colors adds an 8-bit RGB array named `IPFColor`. Each point is colored by one of two keys:

- **Inverse pole figure** (the default). The reference direction, z by default (the map normal), is turned into crystal coordinates. The crystal's symmetry brings it into the standard triangle of its class, and its place there gives the color. For cubic crystals, [001] is red, [101] green and [111] blue. For hexagonal crystals, [0001] is red, [2-1-10] green and [10-10] blue. Choose x or y as the reference direction to see the orientation along the map's edges.
- **Euler angles**: φ1, Φ and φ2 scaled to red, green and blue.

The class of each point comes from its phase and `CrystalStructures`, as EBSD files give them. For data without phases, choose the class under **Crystal structure**. Points not indexed, of an unknown class, or where the optional **Mask** array is 0, are black. The orientation array can hold Euler angles (3 components), quaternions (4) or orientation matrices (9). If none is chosen, the filter uses `EulerAngles`, `Quats` or `OrientationMatrix`, whichever it finds first.

```
m = afx.load("scan.ang")
ipf = m.orientation_colors(direction=(0, 0, 1))      # key="Euler angles" for the other key
ipf.set(color_by="IPFColor")
```

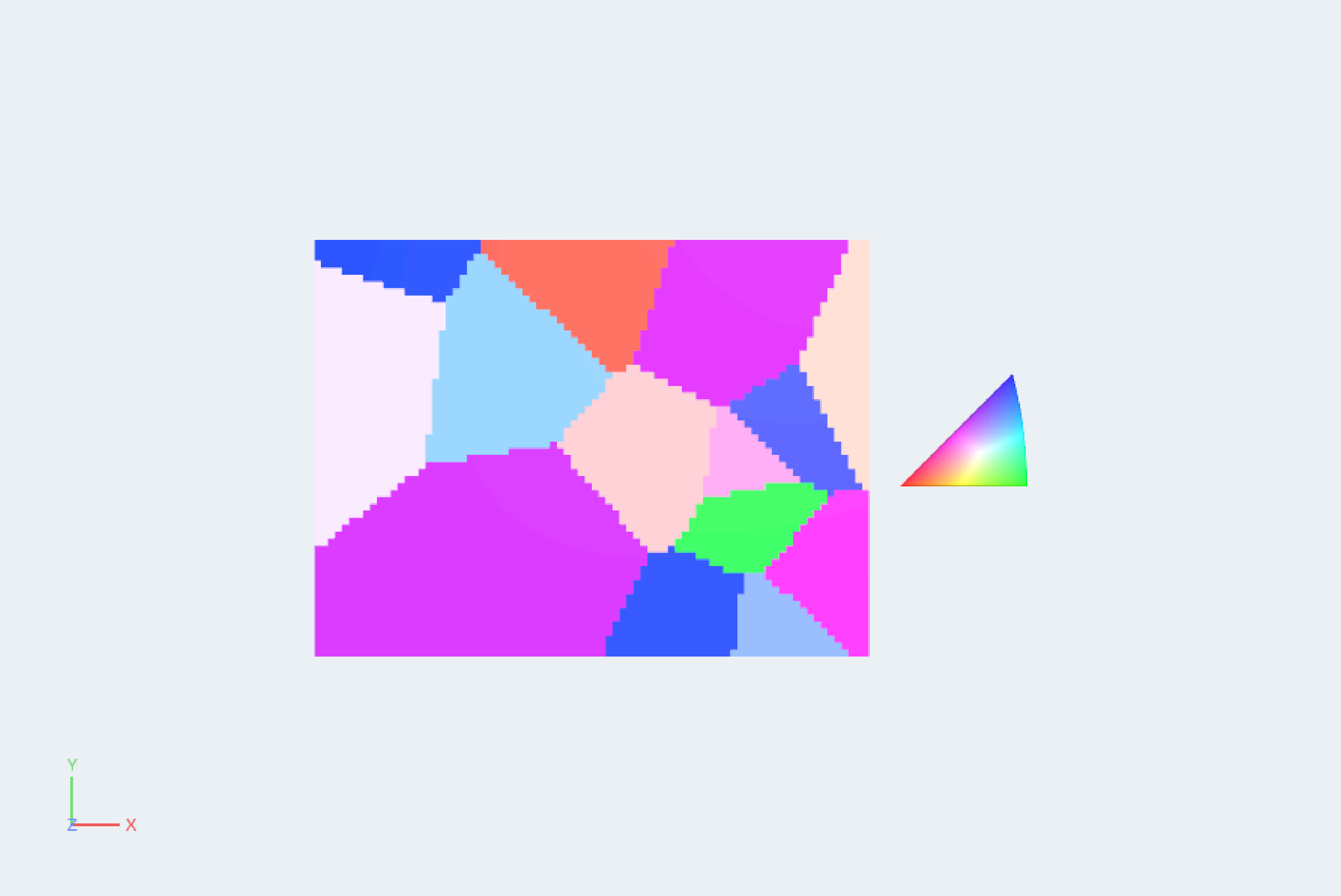
Arrays drawn as their own colors

An array of 8-bit values with 3 or 4 components, like `IPFColor`, is drawn with its own colors when you color by it, not through a color map. No legend is shown for it. To map such an array through a color map instead, clear **Colors: the array's own colors** under Color by in the Properties panel, or use `node.set(direct_colors=False)`. Choosing one component of the array also uses the color map.

IPF Color Key

Sources › IPF Color Key draws the color key of a crystal class: its standard triangle in stereographic projection, colored as Orientation Colors colors it. It lies around the origin in the xy plane, about half a unit across, so move and enlarge it with **Transform** to place it next to a map.

```
key = afx.ipf_color_key(structure="Cubic m-3m", resolution=60)
key.transform(scale=(20, 20, 1), translate=(42, 12, 0)).set(color_by="IPFColor")
```



Convert Orientations

Filters › Grains › Convert Orientations writes the orientations in another representation. It reads and writes Euler angles, quaternions, orientation matrices, axis-angle pairs (the unit axis and the angle in radians), Rodrigues vectors (the unit axis and tan(ω/2)) and homochoric vectors. With **Reduce to the fundamental zone**, each orientation is replaced by its symmetry equivalent with the smallest rotation angle (for cubic crystals, at most 62.8°). The output is named after the representation (`Quats`, `OrientationMatrix`, …) unless you give a name.

```
q = m.convert_orientations(input_type="Euler angles", output_type="Quaternions",
reduce=True)
```

Rotate Orientations

Filters › Grains › Rotate Orientations re-expresses the orientations after the reference frame is turned by an angle (degrees) about an axis. The points do not move; only the description of their orientations changes.

- **Sample** frame: use this when the map's axes differ from those you want, such as a scan taken with the sample turned 90° about its normal.
- **Crystal** frame: turns the crystal axes the orientations refer to.

```
r = m.rotate_orientations(frame="Sample", axis=(0, 0, 1), angle=90)
```

Crop

Filters › Grains › Crop keeps the cells of a voxel volume or EBSD map whose centres lie inside a box, given in sample coordinates (leave a side unset to keep everything on that side). The cells keep their place in space. With **Renumber features** (on by default), the remaining feature ids are numbered from 1 and the per-feature field arrays keep only the rows of those features. To crop by grid index instead, use **Extract Subset**.

```
c = v.crop_volume(min=(0, 0, 0), max=(100, 80, 20))
```

Change Resolution

Filters › Grains › Change Resolution resamples a voxel volume or EBSD map onto cells of another size. Each new cell takes the values of the old cell that holds its centre, so orientations, phases and feature ids stay valid. The volume keeps its corner and, rounded to whole cells, its size. Features that disappear are dropped and the rest renumbered, as in Crop. Point data is not carried over.

```
coarse = v.change_resolution(spacing=(2, 2, 2))
```

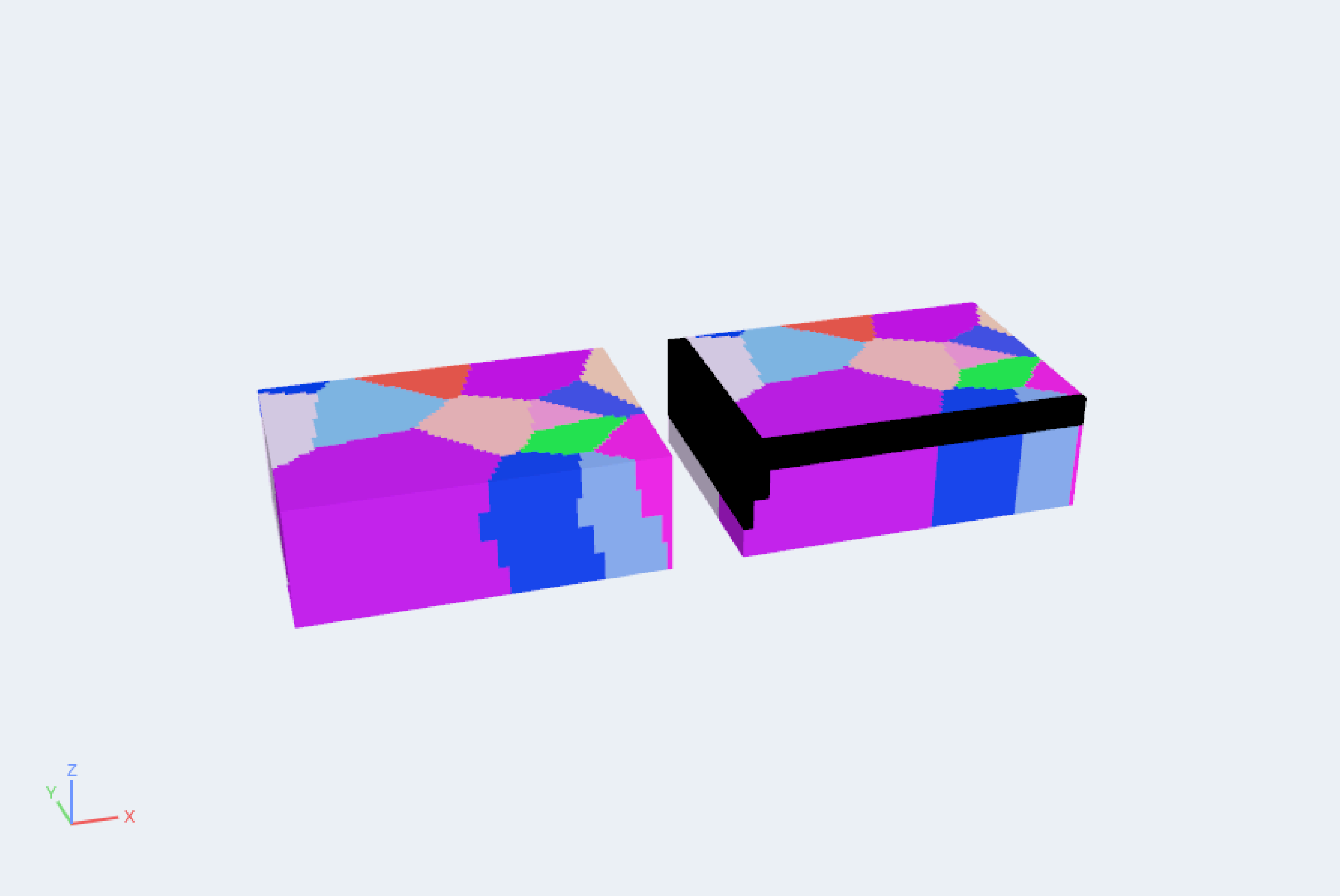
Align Sections

Filters › Grains › Align Sections moves each z slice of a serial-section volume in x and y, by whole cells, so it matches the slice below. Choose how slices are compared:

- **Misorientation** (the default): two cells match when their orientations differ by less than the **Tolerance** (5° by default), with the crystal class of their phase.
- **Feature ids**: two cells match when they belong to the same feature.
- **Given shifts**: no search. Each slice moves by the x and y cells you give, against the slice below.

The search starts from no move and steps to the neighbouring shift with fewer differing cells until none is better, at most **Largest shift** cells each way. A **Mask** array (a sample or good-data mask) limits the comparison to cells where it is not 0. Cells uncovered by a move are set to 0 (phase 0, not indexed). The total shift of each slice is kept as the field array `SectionShifts` (x, y cells). Point data is not carried over.

```
a = stack.align_sections(method="Misorientation", tolerance=5, mask="SampleVoxels")
a.align_sections(method="Given shifts", shifts=[2, -1, 3, 1, -1, 2])   # 3 slices after the first
```

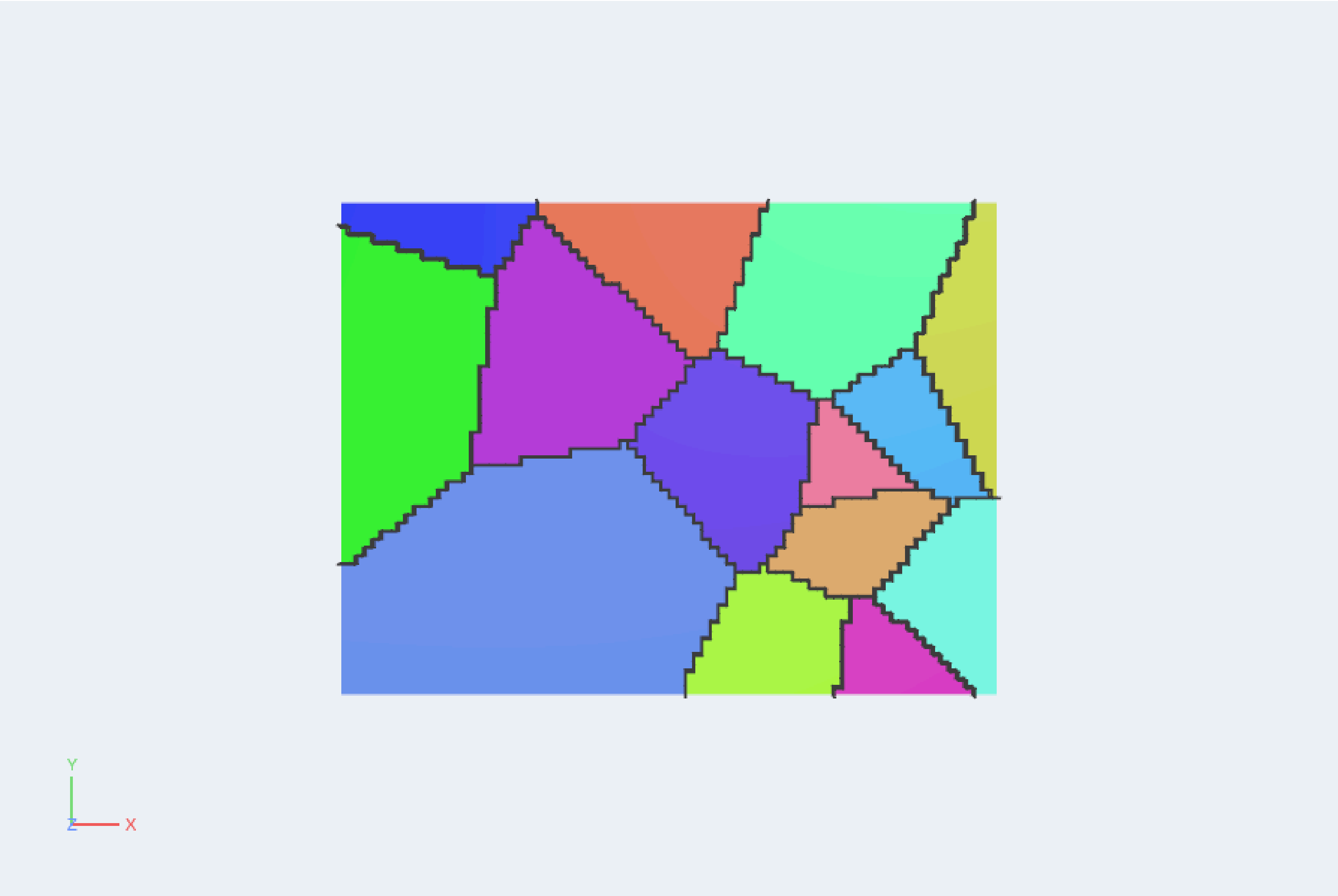


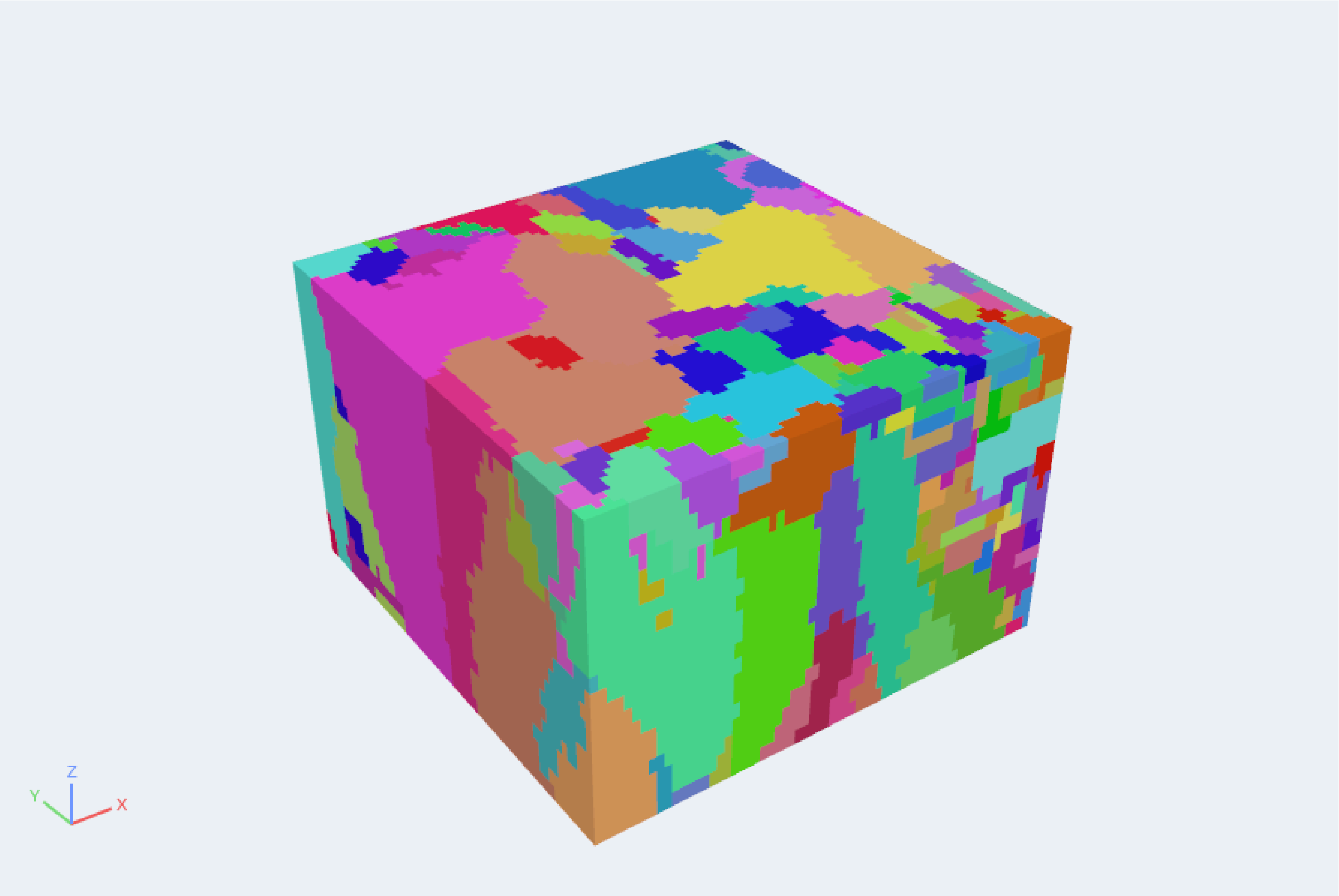
Features (grains)

A feature is a connected set of cells, cells sharing a face, that belong together: usually a grain. **Segment Features** finds them and writes the cell array `FeatureIds` (0 for cells in no feature). The other feature filters then add per-feature results as field arrays, one row per feature id (row 0 stands for id 0). All of them are under **Filters › Grains** and work on voxel volumes and on flat EBSD maps.

A typical sequence for an EBSD volume:

```
v = afx.load("scan.dream3d")
g = (v.segment_features(tolerance=2, mask="SampleVoxels")
      .fill_bad_data(min_defect=7)
      .minimum_size(min_cells=7)
      .feature_sizes().feature_centroids().feature_shapes()
      .average_orientations().reference_misorientations()
      .kernel_average_misorientation(kernel=(2, 2, 1))
      .feature_neighbors())
table = g.feature_table()           # one row per grain, for the Spreadsheet, charts and CSV
g.feature_colors().set(color_by="FeatureColors")
```





Segment Features

Neighbouring cells join the same feature when they pass the test of **Group by**:

- **Misorientation**: same phase, and misorientation below the **Tolerance** in degrees.
- **C-axis alignment**: same phase, and crystal c axes within the tolerance either way round. This is for hexagonal crystals.
- **Scalar**: values of the chosen cell array differ by at most the tolerance.

Cells outside the **Mask** (cells where it is 0), and cells of no known crystal class, stay at 0. Features are numbered in the order their first cell appears (x fastest, then y, then z). **Shuffle ids** numbers them in a shuffled order instead, which gives better colors when coloring by id. The field arrays `Active` and `FeaturePhases` (the phase of each feature) are added, and per-feature arrays from an earlier segmentation are removed.

Cleaning up

- **Fill Bad Data**: clusters of cells without a feature that are smaller than **Smallest kept defect** are filled from the surrounding features. Each cell takes every cell array from the

neighbour whose feature most of its neighbours belong to, sweep after sweep. Larger clusters (pores, the outside of the sample) stay 0.
- **Minimum Size**: features with fewer cells than **Fewest cells** are removed and filled from their neighbours the same way. Feature 0 can win, so a small feature against the outside may become outside. The remaining features are renumbered from 1 in their old order, and per-feature field arrays keep their rows.

Per-feature results

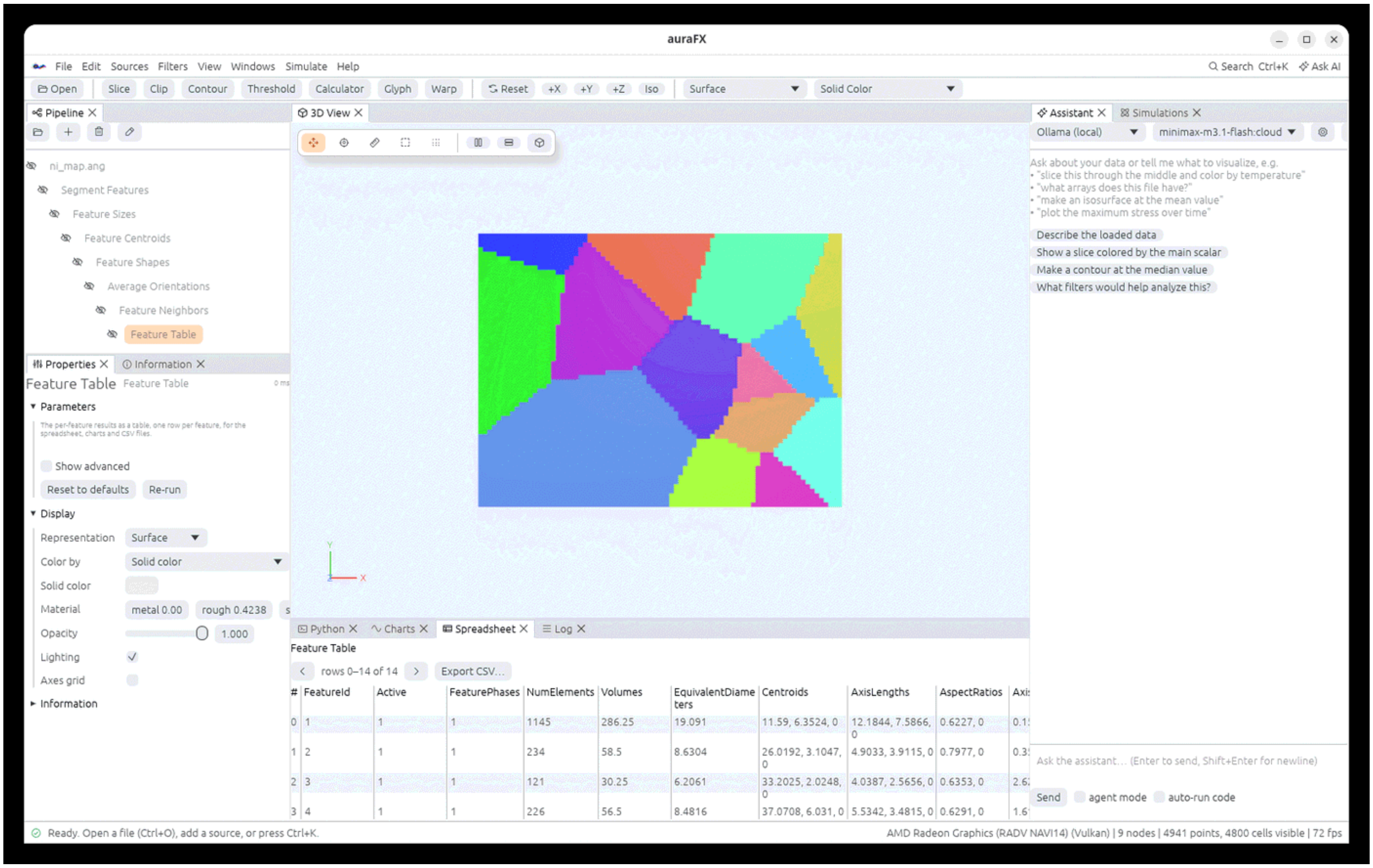
Filter	Field arrays (per feature)	Cell arrays
Feature Sizes	`NumElements`, `Volumes`, `EquivalentDiameters` (sphere of equal volume; circle of equal area on flat maps)	
Feature Centroids	`Centroids` (mean of the cell centres)	
Feature Shapes	`AxisLengths` (semi-axes a ≥ b ≥ c of the ellipsoid with the same moments of inertia), `AspectRatios` (b/a, c/a), `AxisEulerAngles` (Bunge angles of the axes: the rows of the rotation are the a, b and c directions), `Omega3s` (1 for an ellipsoid, smaller for other shapes)	
Average Orientations	`AvgQuats` (x, y, z, w), `AvgEulerAngles`	
Reference Misorientations	`FeatureAvgMisorientations`	`FeatureReferenceMisorientations` (degrees from the feature's average orientation)
Kernel Average Misorientation		`KernelAverageMisorientations` (degrees)
Feature Neighbors	`NumNeighbors`, `SurfaceFeatures` (1 when touching the edge of the volume or cells without a feature), `AvgMisorientations` (mean	`BoundaryCells` (face neighbours in another feature)

Filter	Field arrays (per feature)	Cell arrays
	misorientation to the neighbouring features of the same crystal class, when average orientations exist)	
Distance to Boundaries		`GBEuclideanDistances`, `TJEuclideanDistances`, `QPEuclideanDistances` (distance between cell centres to the nearest boundary, triple-line and quadruple-point cell; -1 where there is none)
Merge Twins	`ParentIds`	`ParentIds`

- **Feature Shapes** uses the exact second moments of the cells: each cell counted at its centre with its own inertia. On flat maps it gives the ellipse of the same moments: a and b, b/a, and the angle of the long axis in the plane as the first Euler angle.
- **Average Orientations** adds, cell by cell, the symmetry equivalent nearest the running average (the identity for the first cell), then normalises.
- **Kernel Average Misorientation** averages a cell's misorientation to the cells of the same feature within **Kernel** cells along x, y and z (1 1 1 means the 26 neighbours). The cell itself counts, with 0. Cells without a feature get 0.
- **Merge Twins** gives neighbouring cubic m-3m features a common parent when their average orientations differ by 60° about a ⟨111⟩ axis, within the **Axis** and **Angle** tolerances. Chains of twins share one parent. Parents are numbered in the order of their first feature.

Boundaries, colors and the table

- **Feature Boundaries** gives the cell faces between different features as a surface. Each face has `FeatureIdA` and `FeatureIdB`, the `Misorientation` (degrees) between the two cells, and `Twin` (1 on twin boundaries of cubic m-3m crystals). **Include the outer surface** adds the faces against cells without a feature and the edge of the volume. Draw it as **Wireframe** over a flat map to see the boundaries as lines.
- **Feature Colors** gives each feature a distinct, stable color (`FeatureColors`, drawn as its own colors).
- **Feature Table** turns the per-feature field arrays into a table, one row per feature with a `FeatureId` column, for the Spreadsheet, charts and **Export CSV**.


auraFX
File Edit Sources Filters View Windows Simulate Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Solid Color
Pipeline
3D View
Assistant
Simulations
ni_map.ang
Segment Features
Feature Sizes
Feature Centroids
Feature Shapes
Average Orientations
Feature Neighbors
Feature Table
Properties
Information
Parameters
Show advanced
Reset to defaults
Re-run
Display
Information
Y
X
Python
Charts
Spreadsheet
Log
Export CSV…
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Send
agent mode
auto-run code

Feature Table Feature Table 0 ms

The per-feature results as a table, one row per feature, for the spreadsheet, charts and CSV files.

☐ Show advanced

Representation: Surface

Color by: Solid color

Material: metal 0.00, rough 0.4238

Opacity: 1.000

Lighting: ☑

Axes grid: ☐

Ollama (local)

minimax-m3.1-flash:cloud

Ask about your data or tell me what to visualize, e.g.

- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Feature Table

rows 0–14 of 14

#	FeatureId	Active	FeaturePhases	NumElements	Volumes	EquivalentDiameters	Centroids	AxisLengths	AspectRatios	Axi
0	1	1	1	1145	286.25	19.091	11.59, 6.3524, 0	12.1844, 7.5866, 0	0.6227, 0	0.1
1	2	1	1	234	58.5	8.6304	26.0192, 3.1047, 0	4.9033, 3.9115, 0	0.7977, 0	0.3
2	3	1	1	121	30.25	6.2061	33.2025, 2.0248, 0	4.0387, 2.5656, 0	0.6353, 0	2.6
3	4	1	1	226	56.5	8.4816	37.0708, 6.031, 0	5.5342, 3.4815, 0	0.6291, 0	1.6

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 9 nodes | 4941 points, 4800 cells visible | 72 fps

Limits

- Hexagonal-grid `.ang` and EDAX files, Oxford `.h5oina` files, and `.dream3d` neighbor lists are not read yet.
- Cubochoric vectors are not among the representations.
- Neighbor lists (each feature's neighbours and shared areas) are not kept as arrays; Feature Boundaries holds the same information face by face.
- Merge Twins handles cubic m-3m crystals only.
- The feature filters copy the volume they work on: a volume of 80 million cells needs several gigabytes of memory. Segmentation of such a volume takes about 10 seconds.
- The orientation mathematics is checked against independent references: rotations from SciPy, disorientation angles by brute force over every crystal class, and the color key by a separate implementation. On a window of a real volume, the inverse pole figure colors agree with those stored in the file within one level of 255. The verification notes (`docs/reference/verification.md` in the source tree) list the comparisons.

Geometry and meshing

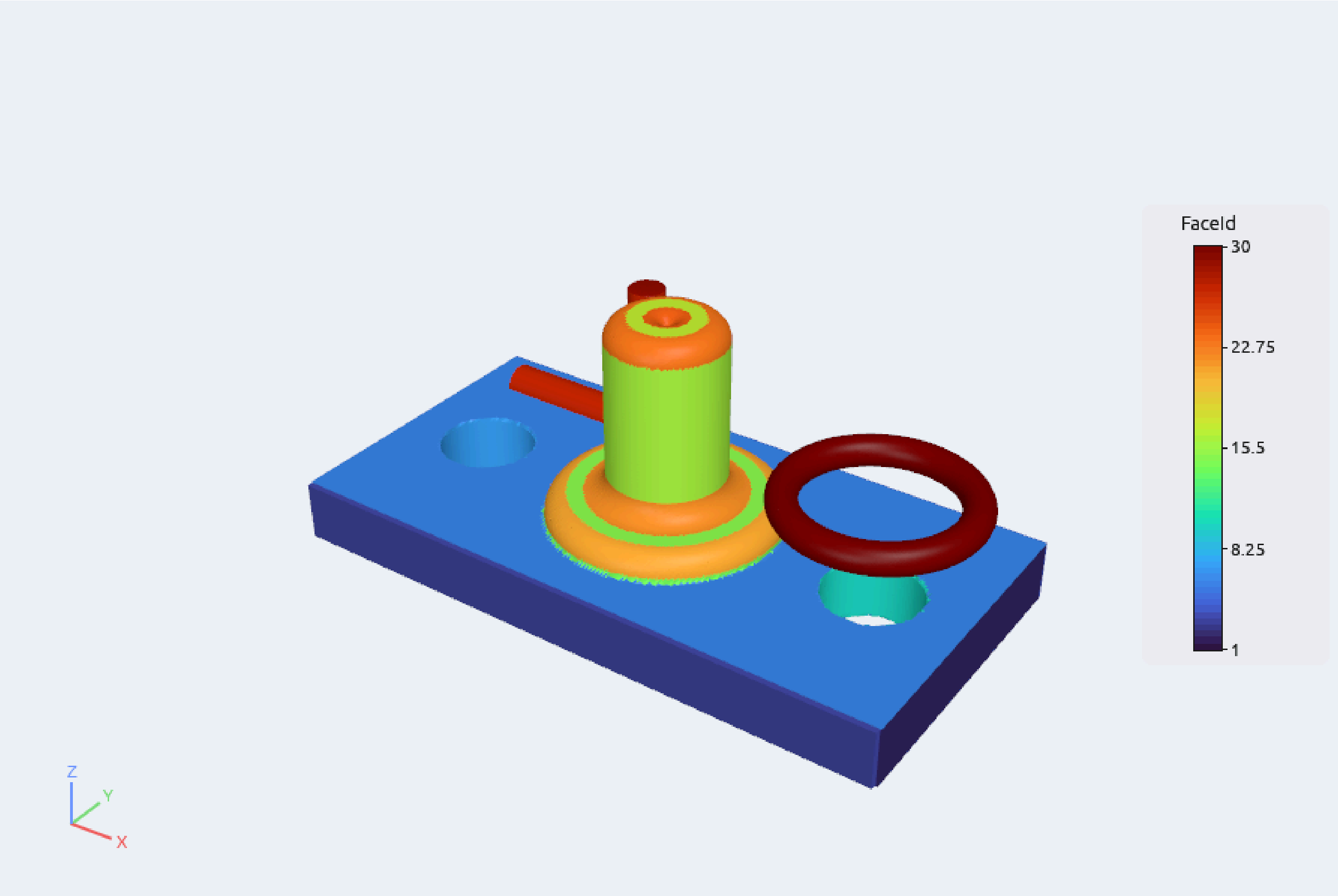
auraFX builds solid geometry from short commands: primitives, booleans, outlines extruded, revolved or swept, and imported surfaces. Each body keeps named faces, so later steps can refer to the top of a plate or the side of a hole. This chapter covers the geometry. Meshing it into hexahedra and other elements comes in the following releases.

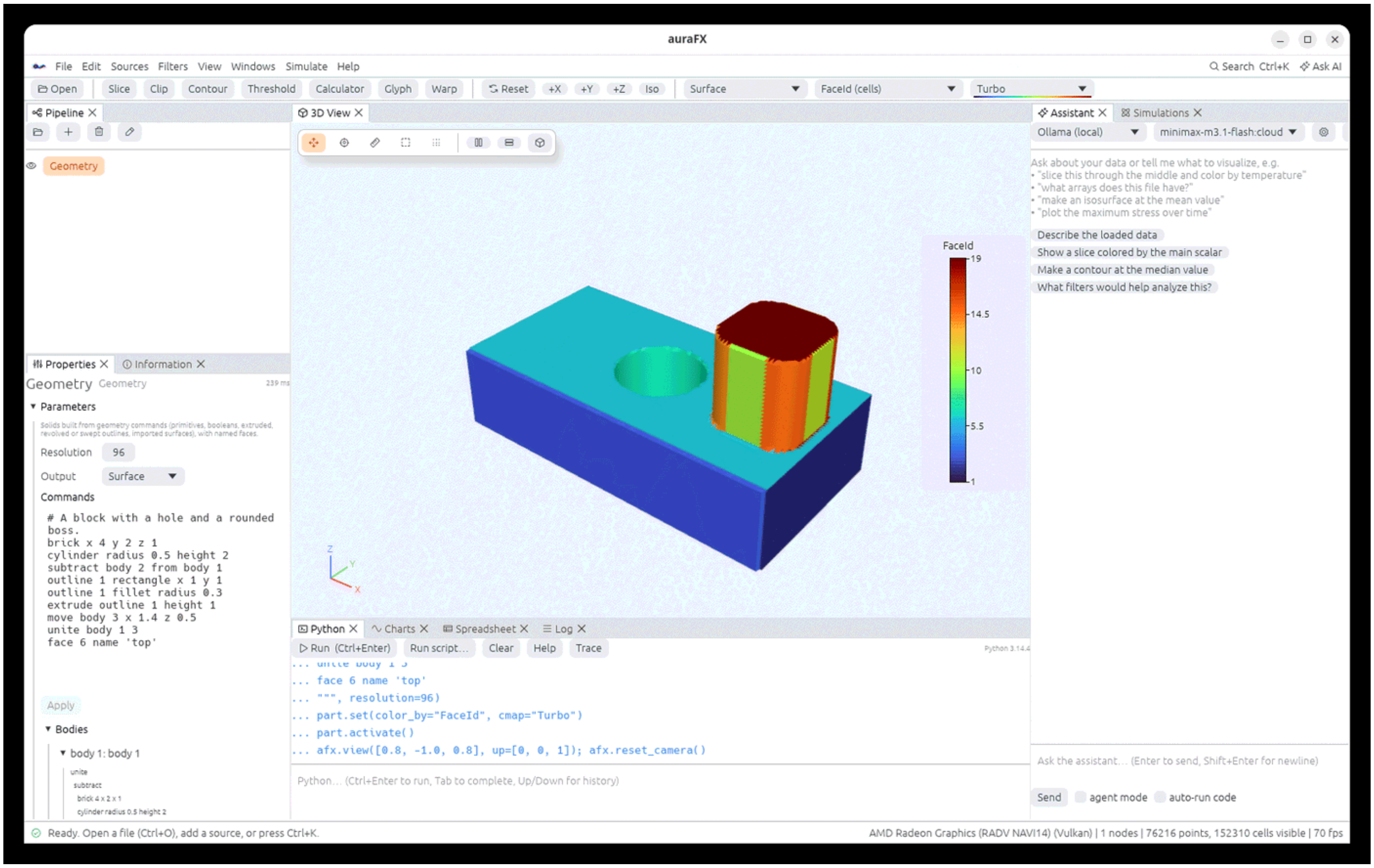
The Geometry source

Add it with **Sources › Geometry**. Its Properties show:

- **Commands**: the command list, edited in the box and applied with **Apply**.
- **Bodies**: a tree of the bodies the commands make. It shows how each body was built and lists its faces with their numbers and names. A mistake in a command shows here, with the line it is on.
- **Resolution**: cells along the longest side of each body when its surface is drawn.
- **Output**: *Surface* gives the bodies' surfaces. *Distance field* gives the signed distance to the nearest body on a grid (negative inside), with the body and face nearest each point.

The surface output carries the cell arrays `BodyId` and `FaceId`. Its field arrays are `FaceNames` (indexed by face number), `BodyIds`, `BodyNames`, and each body's `Volume` and `SurfaceArea`. Color by `FaceId` to see the faces.


FaceId
30
22.75
15.5
8.25
1
Z
Y
X


auraFX
File
Edit
Sources
Filters
View
Windows
Simulate
Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Pipeline
Geometry
3D View
Assistant
Simulations
Properties
Information
FaceId
19
14.5
10
5.5
1
Z
Y
X
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script…
Clear
Help
Trace
Python 3.14.4
Send
Apply

Surface | FaceId (cells) | Turbo

Ollama (local) | minimax-m3.1-flash:cloud

Geometry Geometry 239 ms

Parameters

Solids built from geometry commands (primitives, booleans, extruded, revolved or swept outlines, imported surfaces), with named faces.

Resolution: 96

Output: Surface

Commands

```
# A block with a hole and a rounded
boss.
brick x 4 y 2 z 1
cylinder radius 0.5 height 2
subtract body 2 from body 1
outline 1 rectangle x 1 y 1
outline 1 fillet radius 0.3
extrude outline 1 height 1
move body 3 x 1.4 z 0.5
unite body 1 3
face 6 name 'top'
```

Bodies

- body 1: body 1
  - unite
    - subtract
      - brick 4 x 2 x 1
      - cylinder radius 0.5 height 2

```
... face 6 name 'top'
... """, resolution=96)
... part.set(color_by="FaceId", cmap="Turbo")
... part.activate()
... afx.view([0.8, -1.0, 0.8], up=[0, 0, 1]); afx.reset_camera()
```

Python… (Ctrl+Enter to run, Tab to complete, Up/Down for history)

Ask about your data or tell me what to visualize, e.g.
- "slice this through the middle and color by temperature"
- "what arrays does this file have?"
- "make an isosurface at the mean value"
- "plot the maximum stress over time"

Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?

Ask the assistant… (Enter to send, Shift+Enter for newline)

☐ agent mode ☐ auto-run code

Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.

AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 76216 points, 152310 cells visible | 70 fps

Commands

Write one command per line, or separate commands with `;`. Text after `#` is a comment. Bodies, outlines and faces are numbered from 1 in the order they are made. Sizes are in model units and angles in degrees.

Primitives

All primitives are centred on the origin, and round ones stand along z.

Command	Makes
`brick x 4 y 2 z 1` (or `brick 2` for a cube)	a box; faces `x-`, `x+`, `y-`, `y+`, `z-`, `z+`
`sphere radius 1`	a sphere
`cylinder radius 0.5 height 2`	a cylinder; faces `side`, `bottom`, `top`
`cone radius 1 top 0.4 height 2`	a cone or frustum (top 0 for a point)
`torus major 2 minor 0.5`	a torus in the xy plane
`prism sides 6 radius 1 height 2`	a regular prism; faces `side 1 …`, `bottom`, `top`
`superquadric x 1 y 0.8 z 0.6 e1 0.5 e2 0.7`	a superellipsoid (exponents 1 give an ellipsoid; smaller ones square it off)
`harmonic radius 1 coefficient 2 0 0.3 coefficient 3 2 0.1`	a star-shaped body with radius $r (1 + \Sigma c Y_{lm})$, real spherical harmonics, for particle and grain shapes
`import 'part.stl' scale 0.001`	a closed surface from a file (STL, PLY, OBJ, VTK polygonal data)

Outlines and profile solids

An outline is a closed 2-D curve. Build it, then extrude, revolve or sweep it.

Command	Does
`outline 1 polyline 0 0 2 0 2 1 0 1`	straight edges through the points, closed back to the start
`outline 1 rectangle x 2 y 1 center 0 0`	a rectangle

Command	Does
`outline 1 circle radius 1 center 0 0`	a circle
`outline 1 ellipse x 2 y 1`	an ellipse
`outline 1 regular sides 6 radius 1`	a regular polygon
`outline 1 spline 0 0 2 0 2 1 0 1`	a smooth closed curve through the points
`outline 1 add line 3 1 3 2`	straight edges appended to the outline
`outline 1 add arc center 0 0 radius 1 start 90 end -90`	an arc appended (counterclockwise when end > start)
`outline 1 fillet radius 0.2`	every corner between straight edges rounded
`extrude outline 1 height 2`	the outline from z = 0 to the height
`revolve outline 1 angle 360`	the outline turned about z; x is the distance from the axis and y the height
`sweep outline 1 path 0 0 0 4 0 0 4 3 0`	the outline carried along a path of points, kept normal to it

Faces of profile solids are named after the outline's curves: `edge 1`, `edge 2`, … for straight edges, then `arc 1`, `fillet 1`, `circle`, `spline 1`. Extrusions add `bottom` and `top`, partial revolutions `start` and `end`, and sweeps `start` and `end`.

Moving and combining bodies

Command	Does
`move body 2 x 1 y 0 z 0.5`	translates
`rotate body 2 angle 30 about z center 0 0 0`	rotates (`about x`, `y`, `z` or a direction `1 1 0`)
`scale body 2 1.5` or `scale body 2 x 2 y 1 z 1`	scales about the origin
`unite body 1 2 3`	one body (numbered as the first) from several
`subtract body 2 3 from body 1`	removes bodies 2 and 3 from body 1 (add `keep` to keep them)
`intersect body 1 2`	the common part
`copy body 1`	a copy as a new body
`delete body 2`	removes a body
`body 1 name 'bracket'`	names a body

Command	Does
`face 6 name 'inlet'`	names a face

A face keeps its name through booleans, so the faces of a hole are still `cylinder 2 side` after the cylinder is subtracted, unless you rename them.

Python

`afx.mesh.geometry()` builds geometry command by command. Each command is checked at once, so a mistake raises an error naming the line.

```
g = afx.mesh.geometry()
g.cmd("brick x 10 y 5 z 2")
g.cmd("cylinder radius 1 height 4; subtract body 2 from body 1")
for b in g.bodies():                   # id, name, history, faces, bounds
    print(b["id"], b["name"], b["faces"])
part = g.result(resolution=96)         # adds the Geometry source to the pipeline
print(g.journal())                     # every command so far
```

The Geometry source is also an ordinary source: `afx.geometry(commands="sphere radius 1")`. Edits made in the command box are recorded in the trace (**Trace** in the Python panel), so the journal can be replayed.

Signed Distance Field

Filters › Mesh › Signed Distance Field turns a closed surface, such as an STL file or the Geometry output, into an image grid with a `Distance` point array. The array is negative inside. **Resolution** sets the cells along the surface's longest side, and **Margin** adds space around it as a fraction of its size. Contour it at 0 to get the surface back, or threshold it to keep the inside.

```
d = afx.load("part.stl").signed_distance(resolution=128, margin=0.1)
```

Limits

- Surfaces are drawn by contouring each body's distance field, so sharp edges are rounded over about one cell. Raise **Resolution** for crisper edges. The `Volume` and `SurfaceArea` of smooth bodies are within about 0.5 % at a resolution of 96.

- The distances of superquadrics, harmonic shapes, partial revolutions, sweeps and non-uniform scaling are approximate away from the surface. The surface itself is exact.
- Sweeps along sharply bent paths join the pieces with a mitre, not a rounded bend.
- Imported surfaces must be closed. Their inside is found from the direction the triangles face.

Settings and keyboard shortcuts

Settings

Edit › Settings… (Ctrl+,):

Setting	Meaning
Theme	dark, light or system; the 3D background per theme
UI scale	enlarges the whole interface
Orientation axes, Time label	shown in the 3D views
Filter threads	cores used by the parallel filters (0 = all); takes effect at the next start; `AURAFX_THREADS` overrides it
Default color map	for new objects
AI	provider, model, server URL, API key, auto-run and agent mode
Plugins	the Plugins window (also **Edit › Plugins…**)
Lighting	in **View › Lighting**; the choices are saved with the settings

Settings live in `~/.config/aurafx/settings.toml` (Linux) or the app's config folder on other systems; the window layout in `~/.local/share/aurafx/app.ron`.

Keyboard shortcuts

Keys	Action
Ctrl+O	open a file
Ctrl+S	save state
Ctrl+Shift+S	save a screenshot
Ctrl+K, Ctrl+Space	command palette
Ctrl+Z, Ctrl+Y	undo, redo
Ctrl+Enter	run the code in the Python console
Ctrl+,	settings

Keys	Action
R	reset the camera
Esc, P, M, S, D	view modes: navigate, pick, ruler, select cells, select points
Delete	delete the selected object
Space	play / pause time
Double-click in the view	fit the data

Help › Keyboard shortcuts shows this list in the app.

Environment variables

Variable	Use
`AURAFX_THREADS`	thread limit for filters
`AURAFX_PLUGIN_PATH`	extra plugin folders
`AURAFX_SERVER`	path of `aurafx-server` on a host (in its setup lines)
`AURAFX_DATA_DIR`	where settings and state are kept
`OLLAMA_HOST`, `ANTHROPIC_API_KEY`	AI providers
`PYO3_PYTHON` (at build time)	which Python to embed